

# **Apuntes – Clase 10**

## **Paginación**

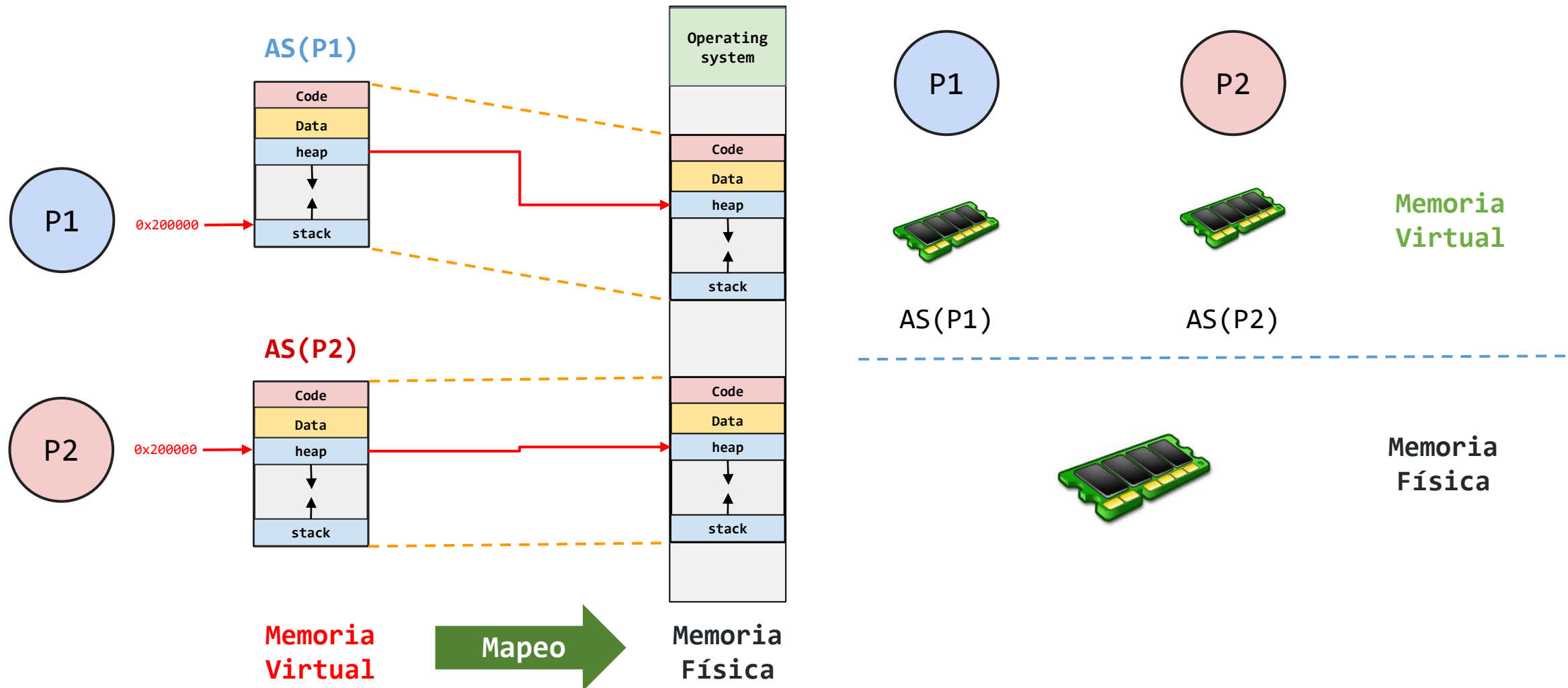
### **Sistemas Operativos**

Universidad de Antioquia

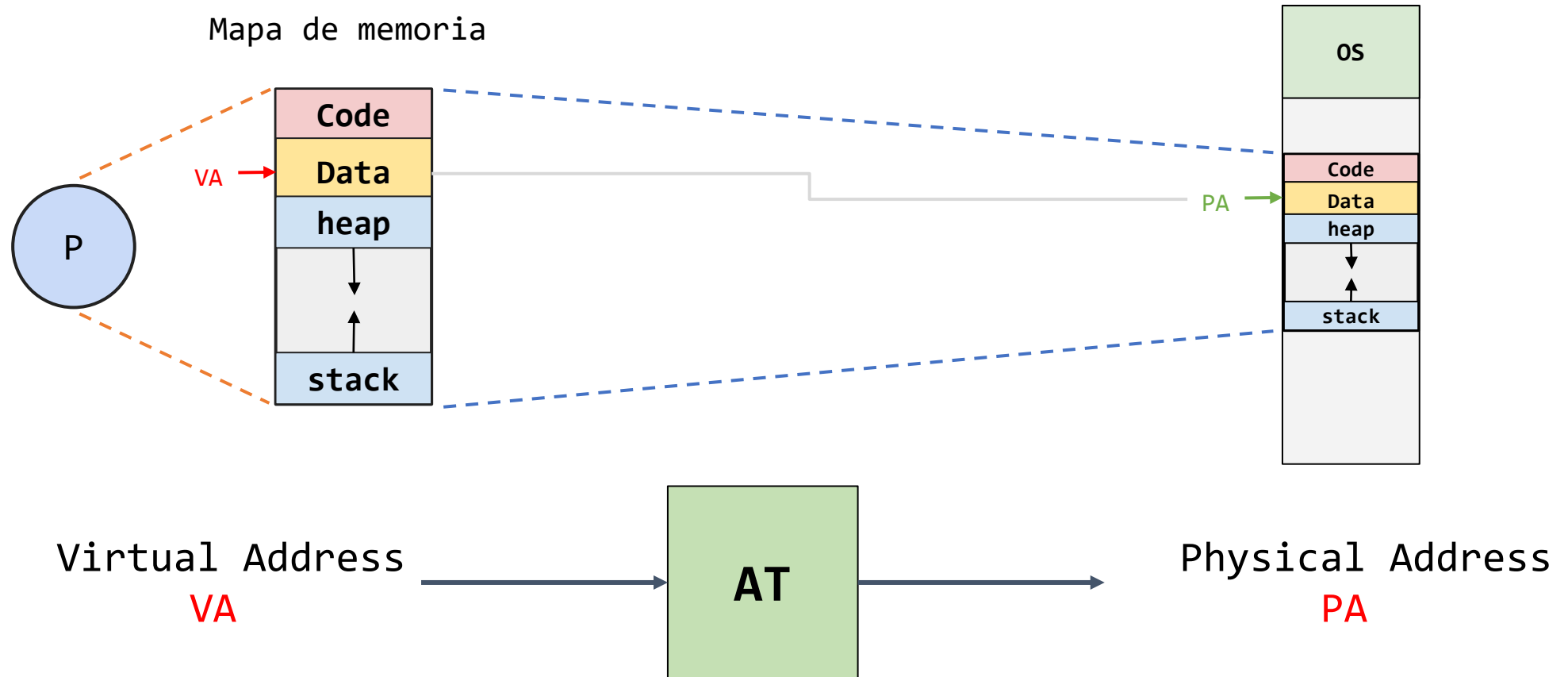
Facultad Ingeniería

Ingeniería de Sistemas

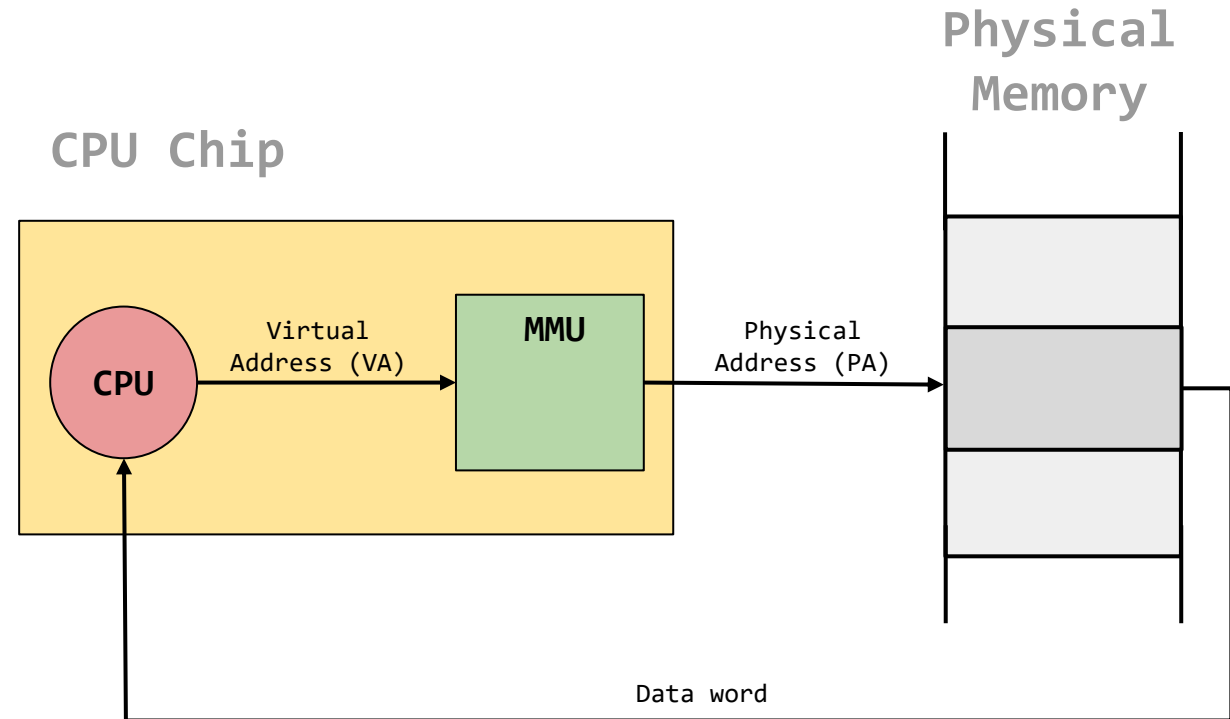
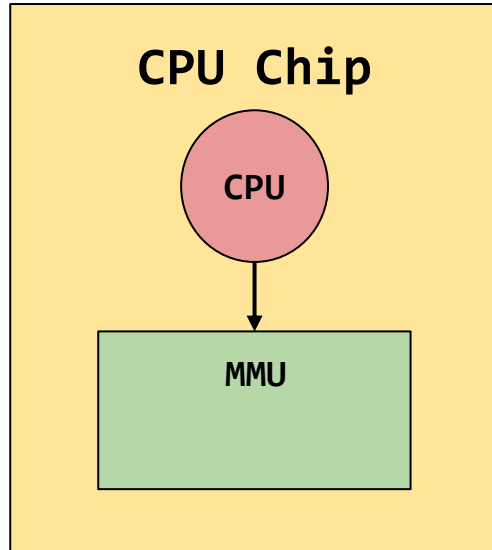
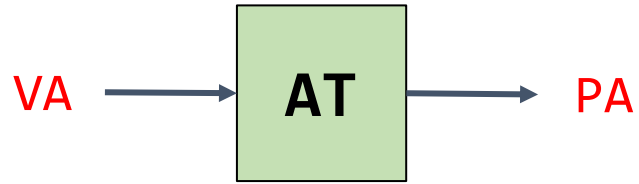
# Virtualización de memoria



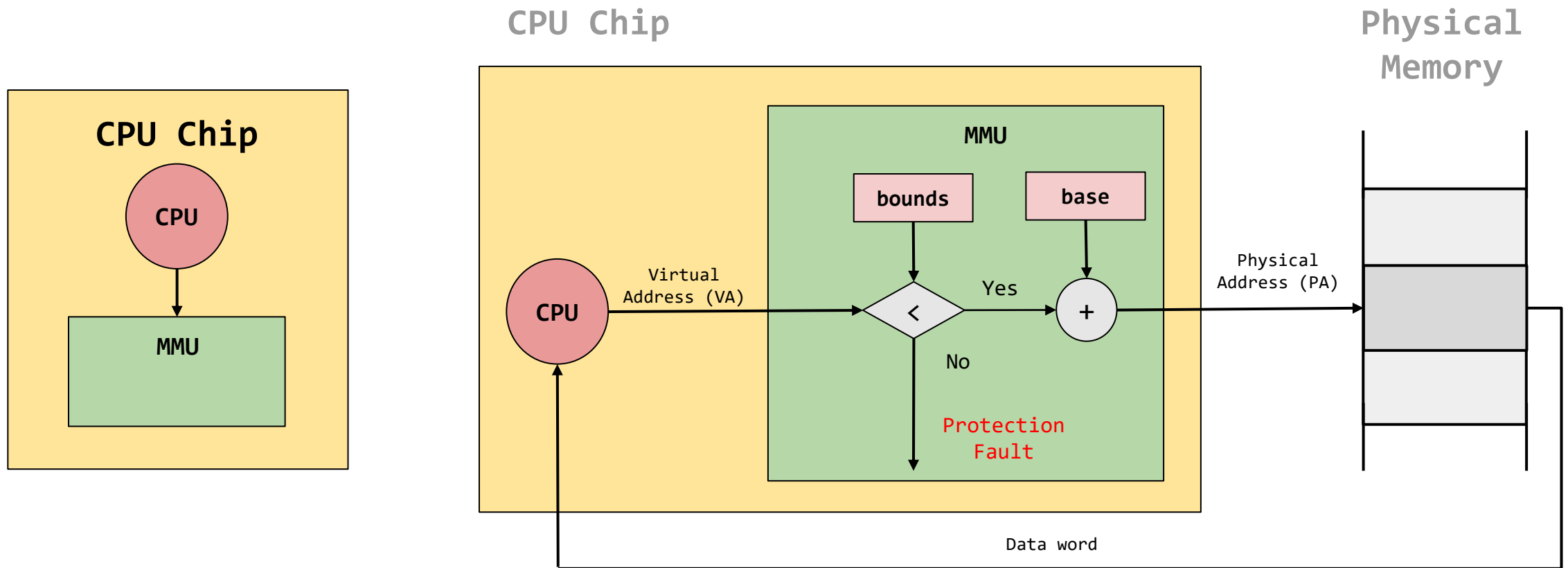
# Address Translation (Mapeo)



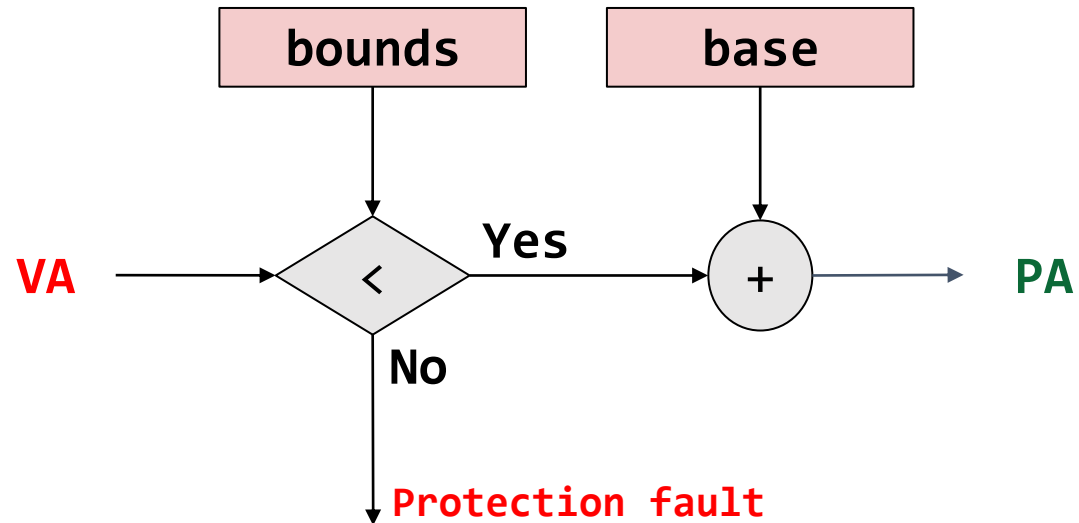
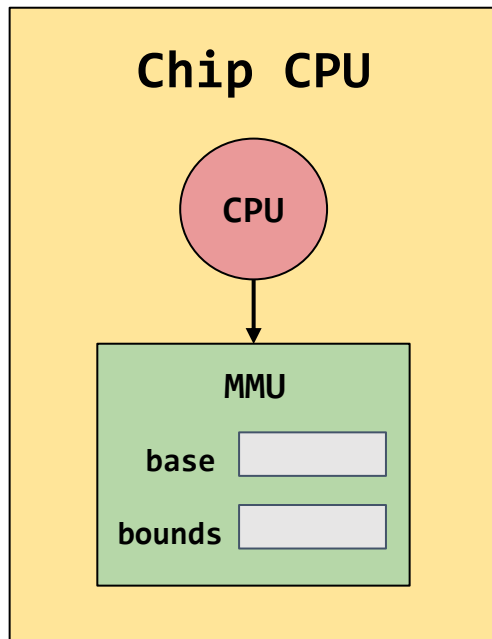
# Memory Management Unit (MMU)



# Traducción de direcciones – Dynamic Relocation (base & bounds)



# Repaso Traducción de direcciones – Dynamic Relocation (base & bounds)



1

$$0 \leq VA < \text{bounds}$$

2

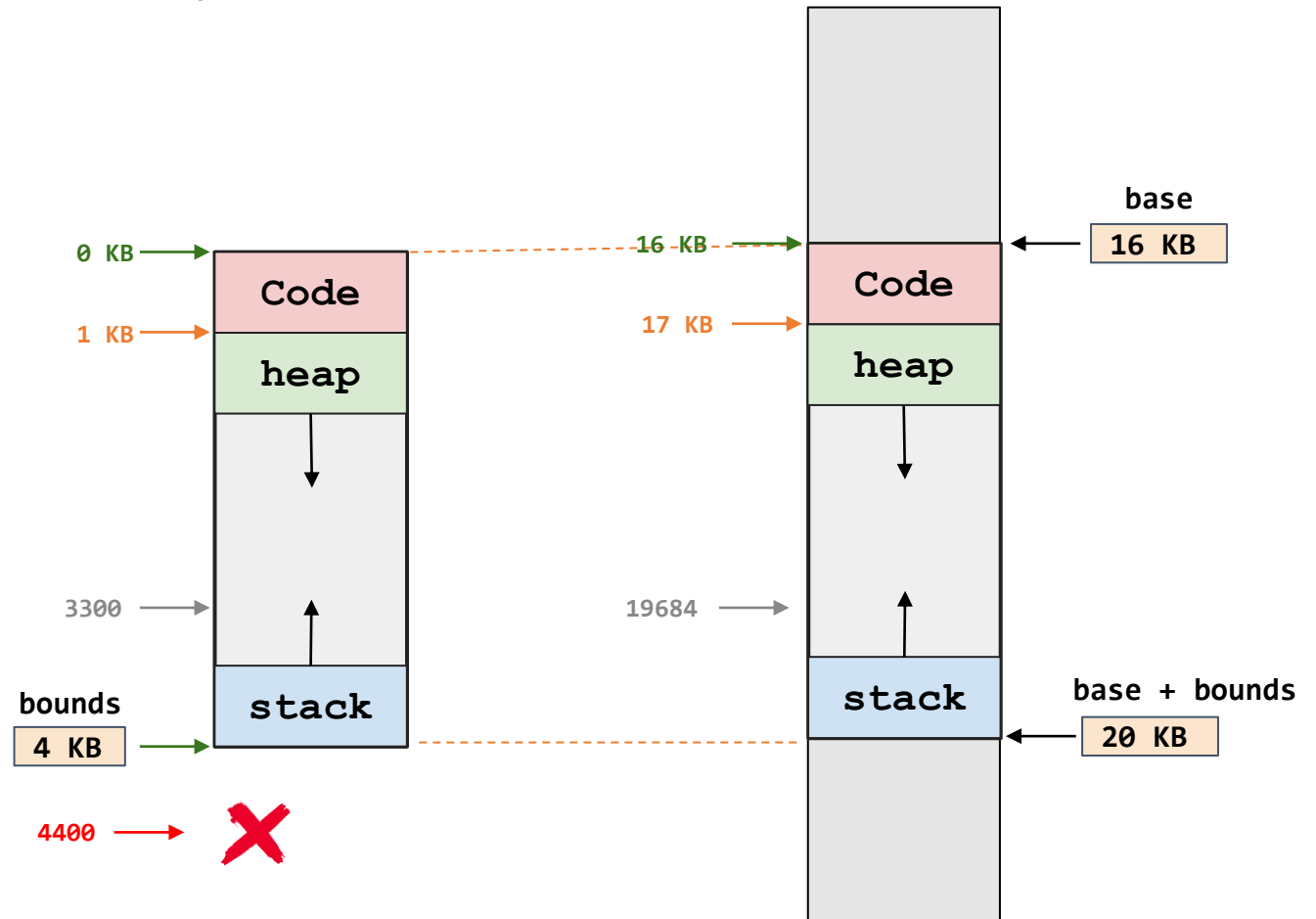
$$PA = VA + \text{base}$$

# Repaso Traducción de direcciones – Dynamic Relocation (base & bounds)

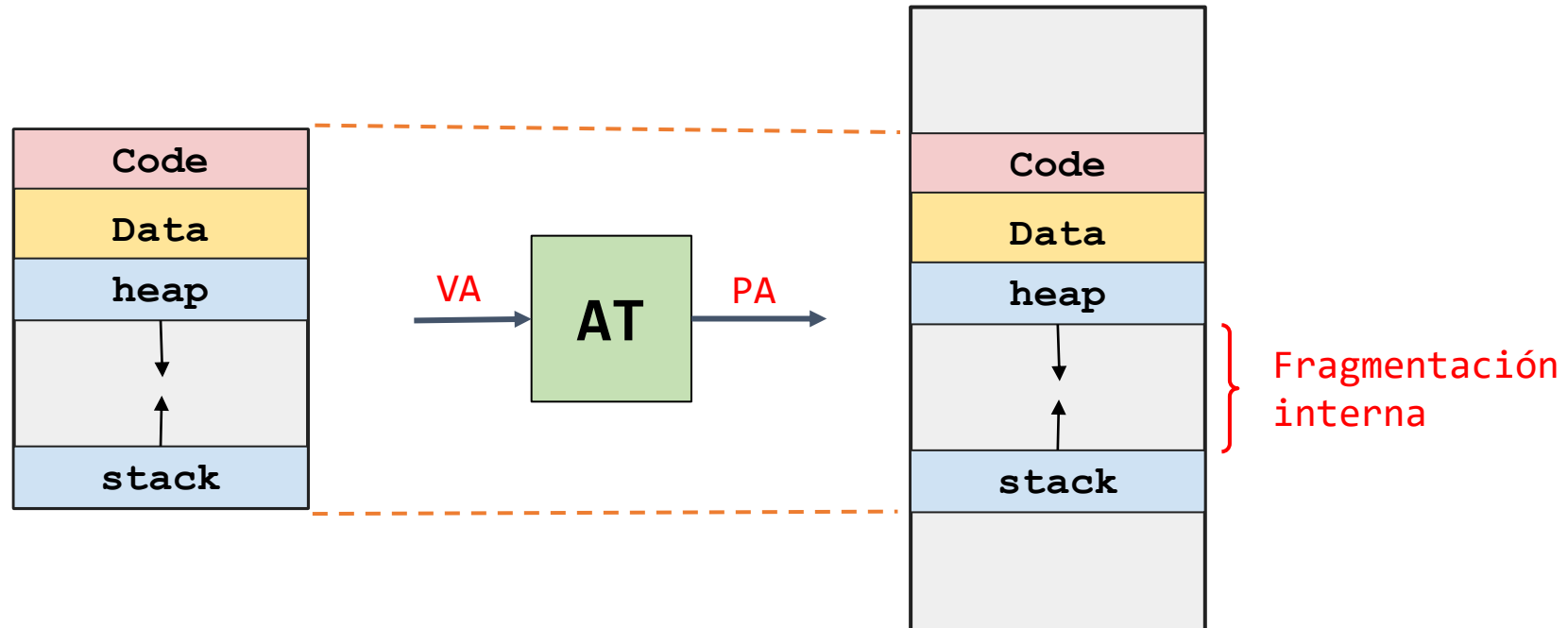
## Ejemplo traducción:

- $\text{Size(AS)} = 4\text{KB}$
- $\text{Size(PM)} = 64\text{KB}$
- $\text{base} = 16\text{ K}$
- $\text{bounds} = 4\text{K}$

| Virtual Address (VA) | Physical Address (PA)    |
|----------------------|--------------------------|
| 0                    | 16KB                     |
| 1 KB                 | 17KB                     |
| 3300                 | 19684                    |
| 4400                 | Dirección fuera de rango |

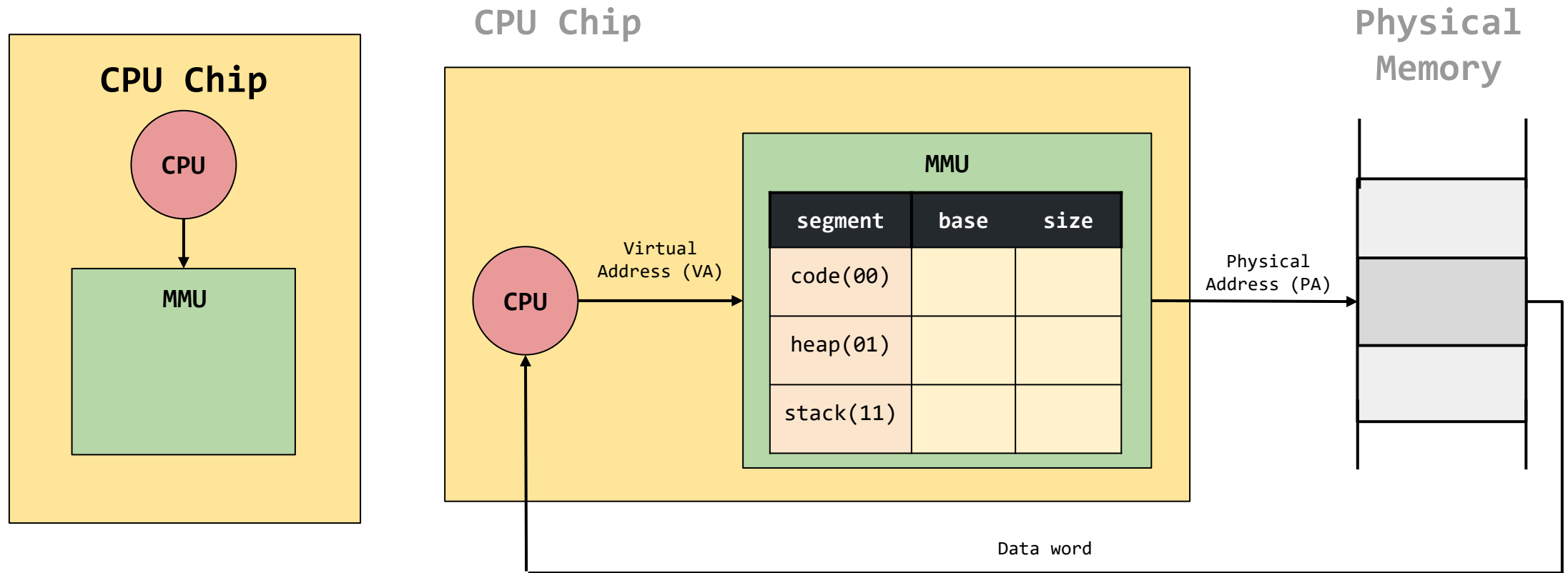


# Fragmentación interna

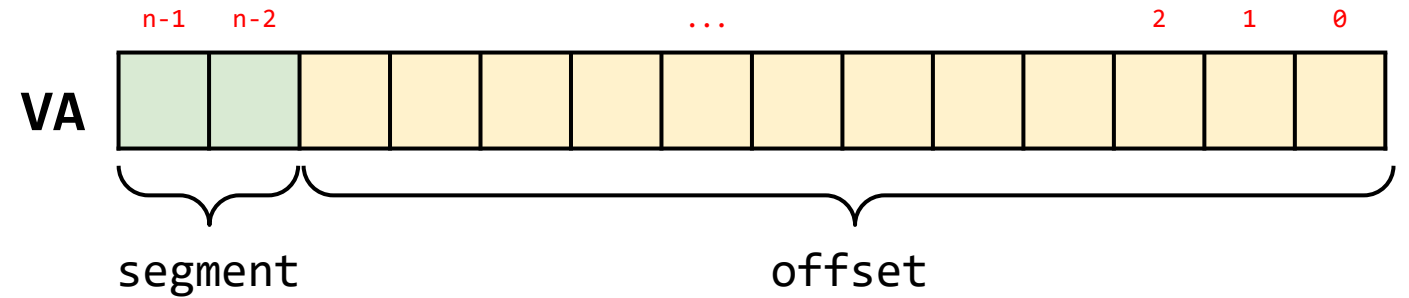
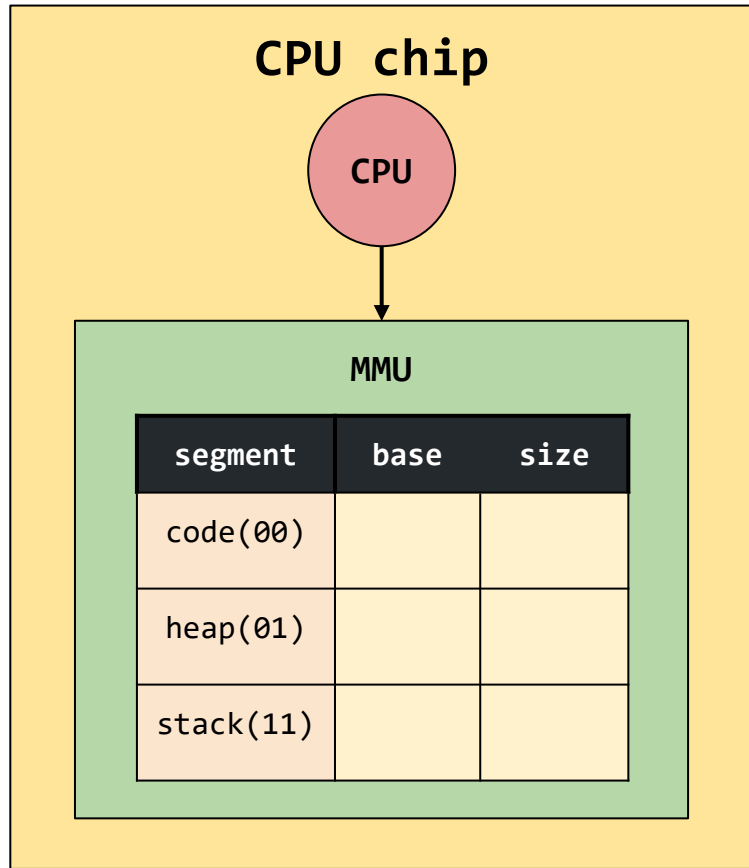




# Traducción de direcciones - Segmentation

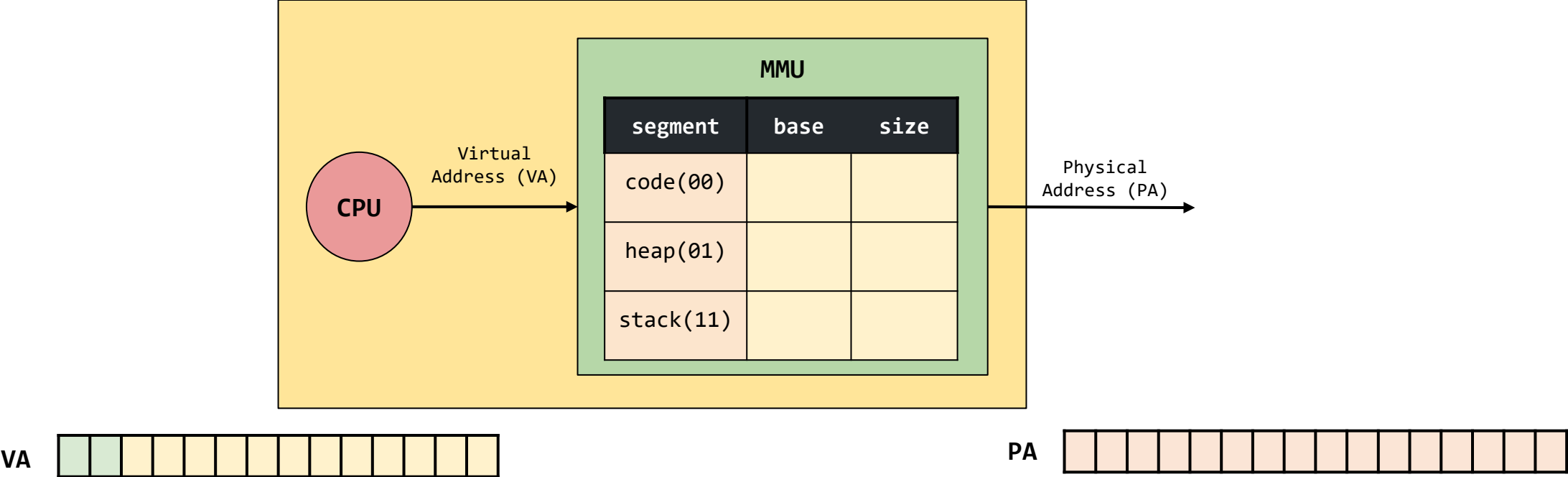


# Traducción de direcciones - Segmentation



| segment   | base | size | Grows Positive? | Protection   |
|-----------|------|------|-----------------|--------------|
| code(00)  |      |      |                 | Read-Execute |
| heap(01)  |      |      |                 | Read-Write   |
| stack(11) |      |      |                 | Read-Write   |

# Traducción de direcciones - Segmentation



| segment | offset                               | Physical Address          |
|---------|--------------------------------------|---------------------------|
| code    | offset = VA                          | PA = offset + base[code]  |
| heap    | offset = VA - VA[heap]               | PA = offset + base[heap]  |
| stack   | Offset[stack] = offset - offset[max] | PA = offset + base[stack] |

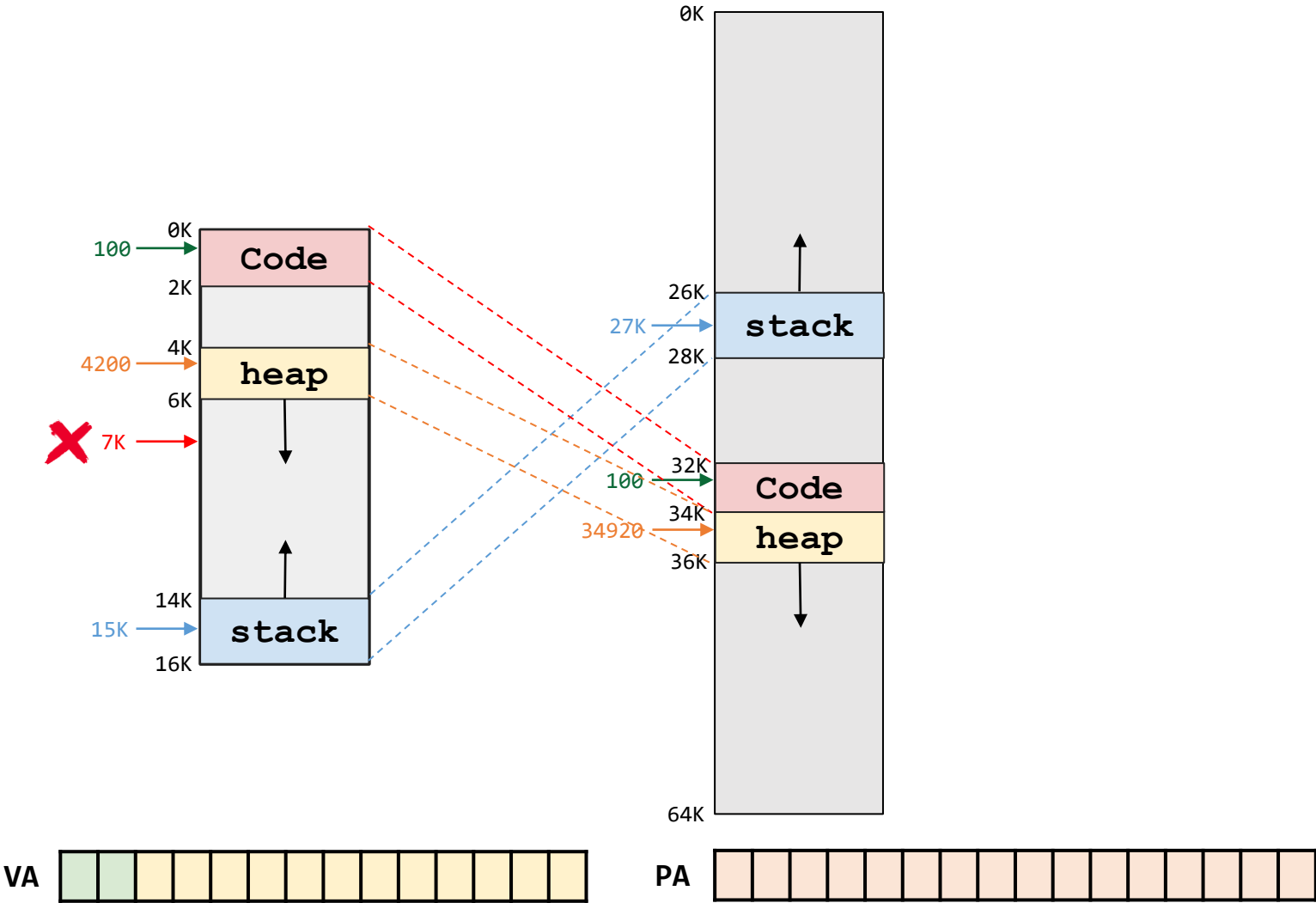
# Traducción de direcciones - Segmentation

## Datos - Ejemplo:

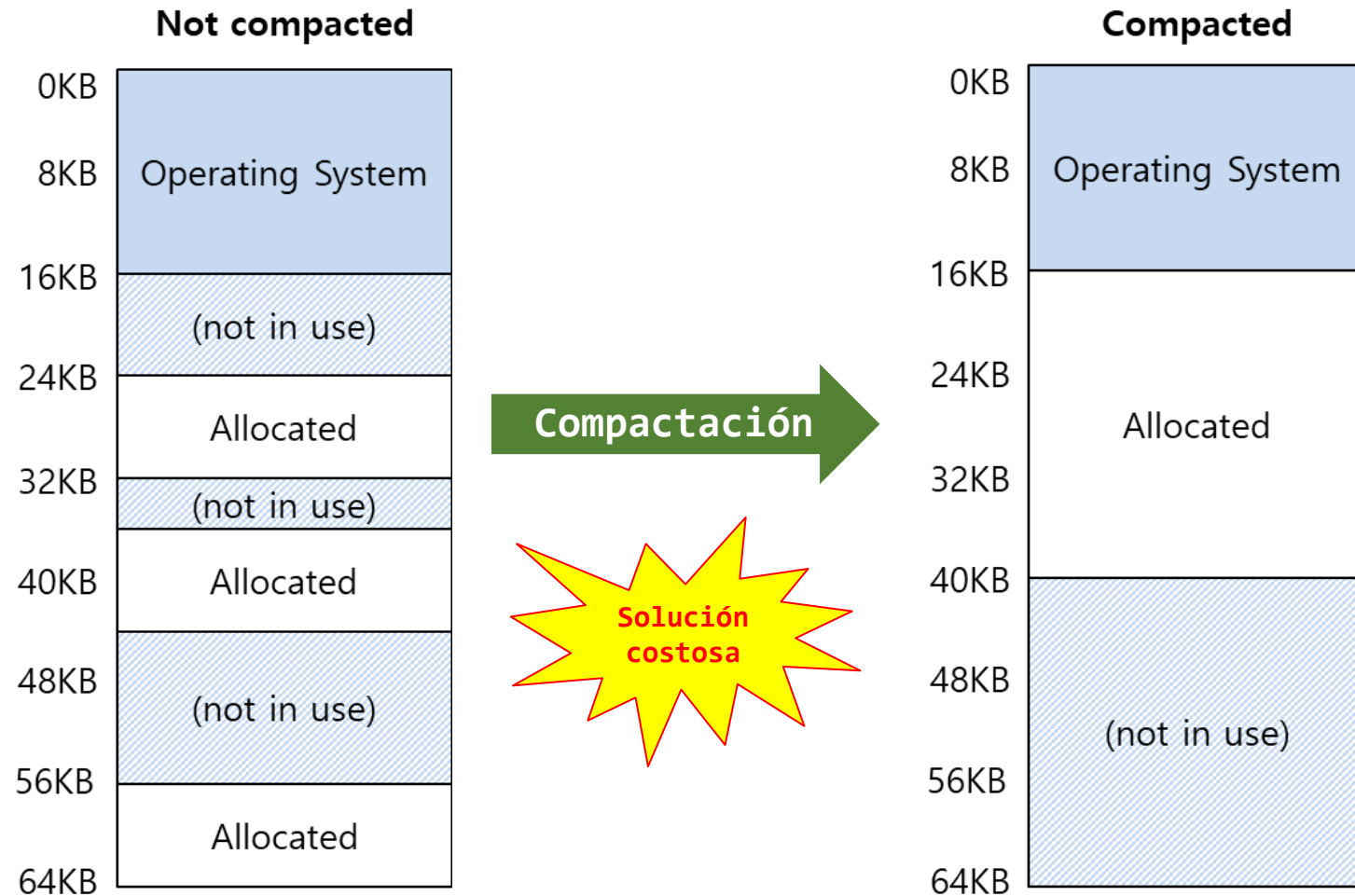
- Size(AS) = 16KB
- Size(PM) = 64KB

| bits | segment | base | size |
|------|---------|------|------|
| 00   | code    | 32K  | 2K   |
| 01   | heap    | 34K  | 2K   |
| 11   | stack   | 28K  | 2K   |
| ---  | ---     | ---  | ---  |

| Segment | Virtual Address (VA) | Physical Address (PA) |
|---------|----------------------|-----------------------|
| code    | 100                  | 32868                 |
| heap    | 4200                 | 34920                 |
| ---     | 7K                   | ---                   |
| stack   | 15K                  | 27K                   |



# Fragmentación externa



# Contextualización

## ¿Cómo virtualizar la memoria con páginas?

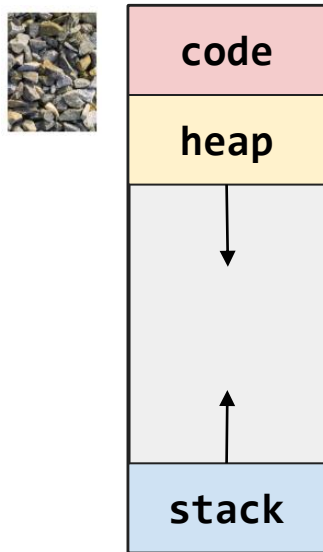
- ¿Cómo virtualizar la memoria con páginas para evitar los problemas de la segmentación?
- ¿Cuáles son las técnicas básicas?
- ¿Cómo hacer que estas técnicas trabajen correctamente para alcanzar un mínimo **overhead** (espacio y tiempo)?

# Contextualización

## Preliminares

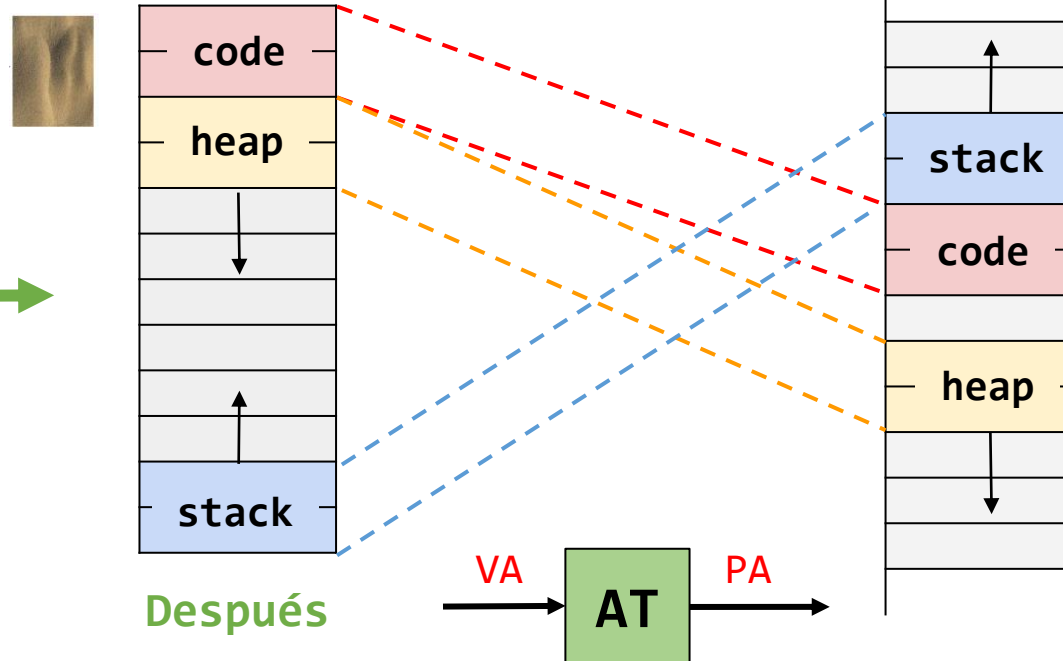
- En la segmentación la forma en cómo se divide la memoria maneja una granularidad grano grueso (pocas divisiones, continuas, tamaño variable).
- El objetivo al usar paginación es hacer más fina la división (grano fino) de la memoria (mayor cantidad de divisiones, tamaño fijo).

Segmentación



Antes

Paginación

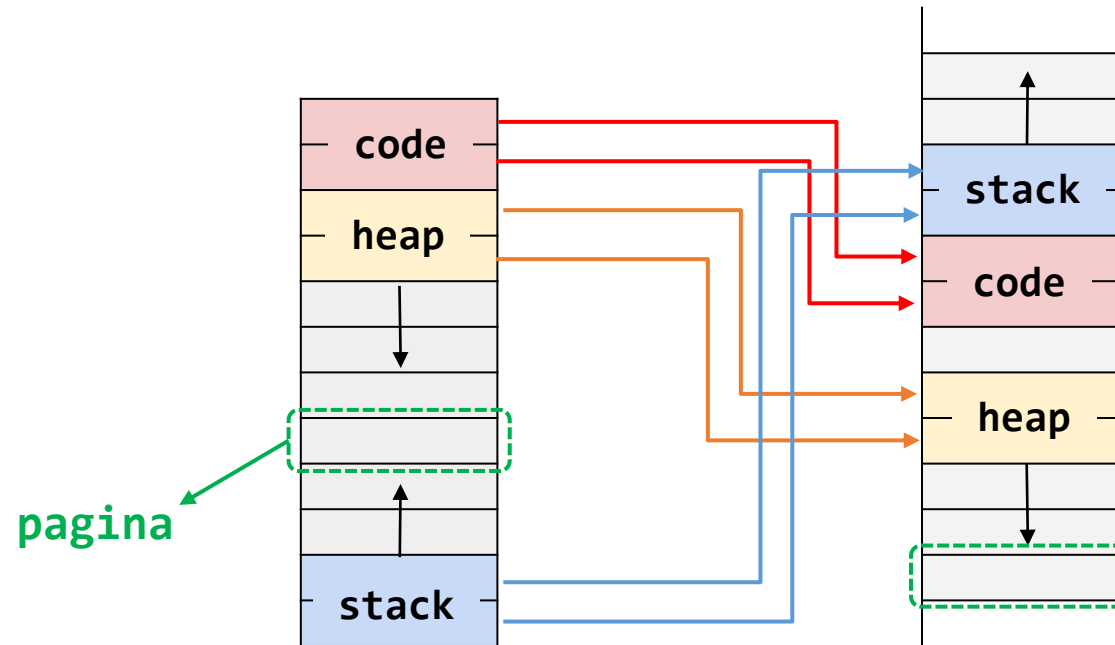


Después

# Paginación – Conceptos claves

## Aspectos claves

- Dividir el espacio de direcciones (**address space**) en unidades de tamaño fijo conocidas como **páginas**.
- Cada página es independientemente mapeada en memoria física.
- Elimina la necesidad de que los segmentos del espacio de direcciones sea continuo.

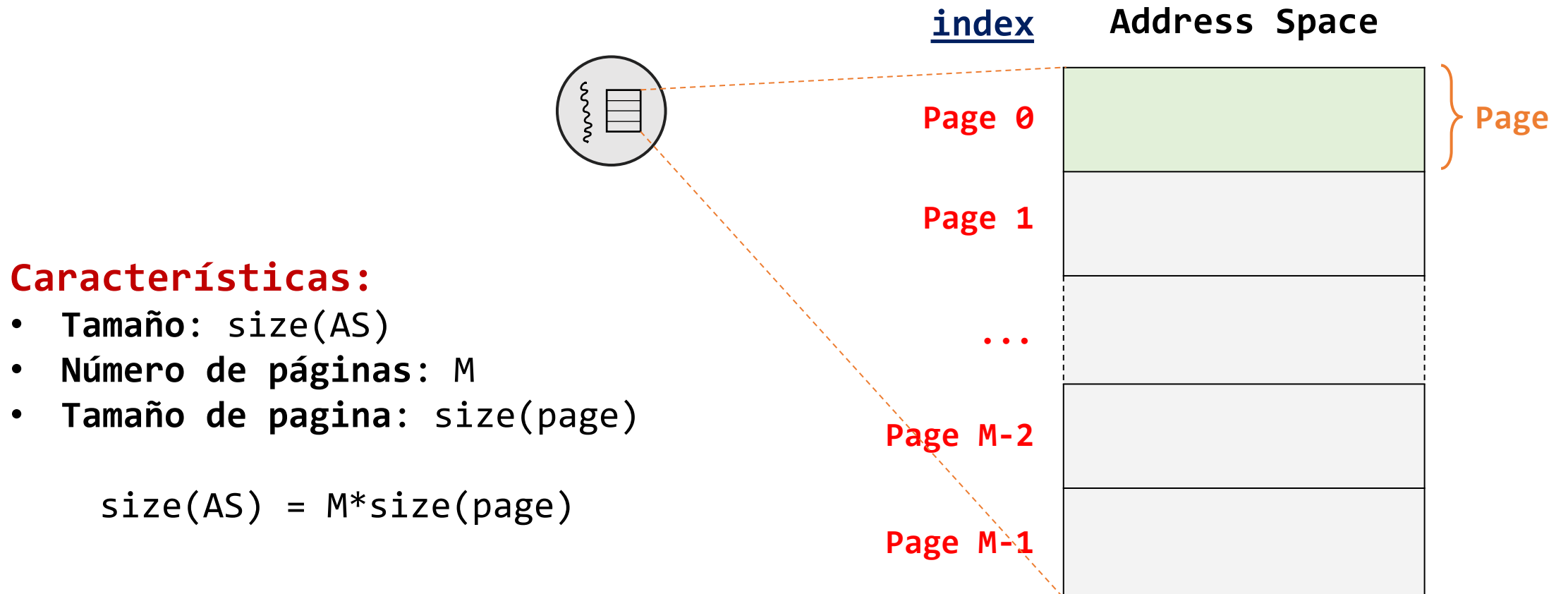




# Paginación – Conceptos claves

## Page (Página)

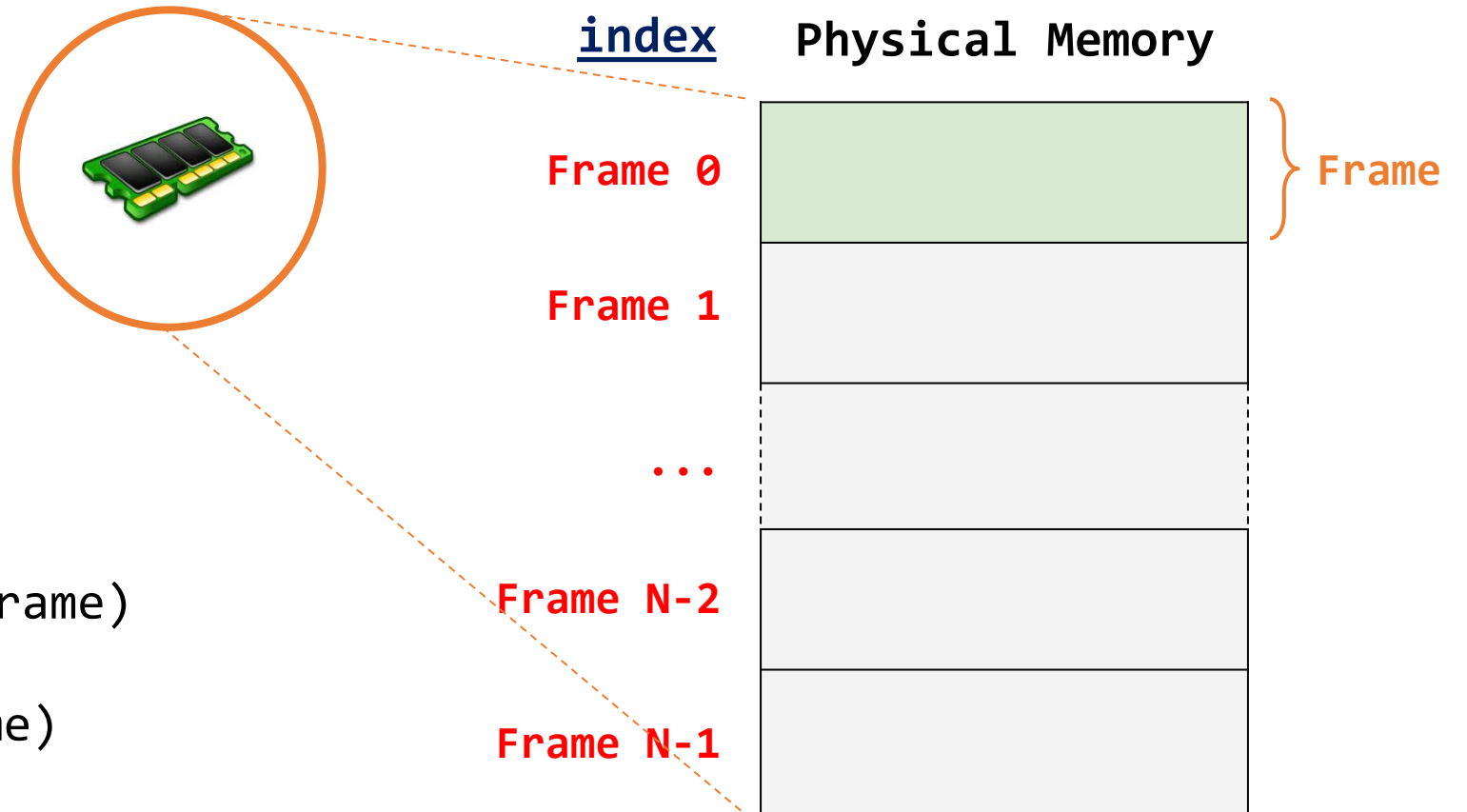
- Una pagina (page) es un bloque de memoria virtual (Address Space) de tamaño fijo.



# Paginación – Conceptos claves

## Page Frame (Marco)

- **Un marco (frame)** es un bloque de memoria física de tamaño fijo el cual se encuentra asociado a una página (page) de la memoria virtual.



### Características:

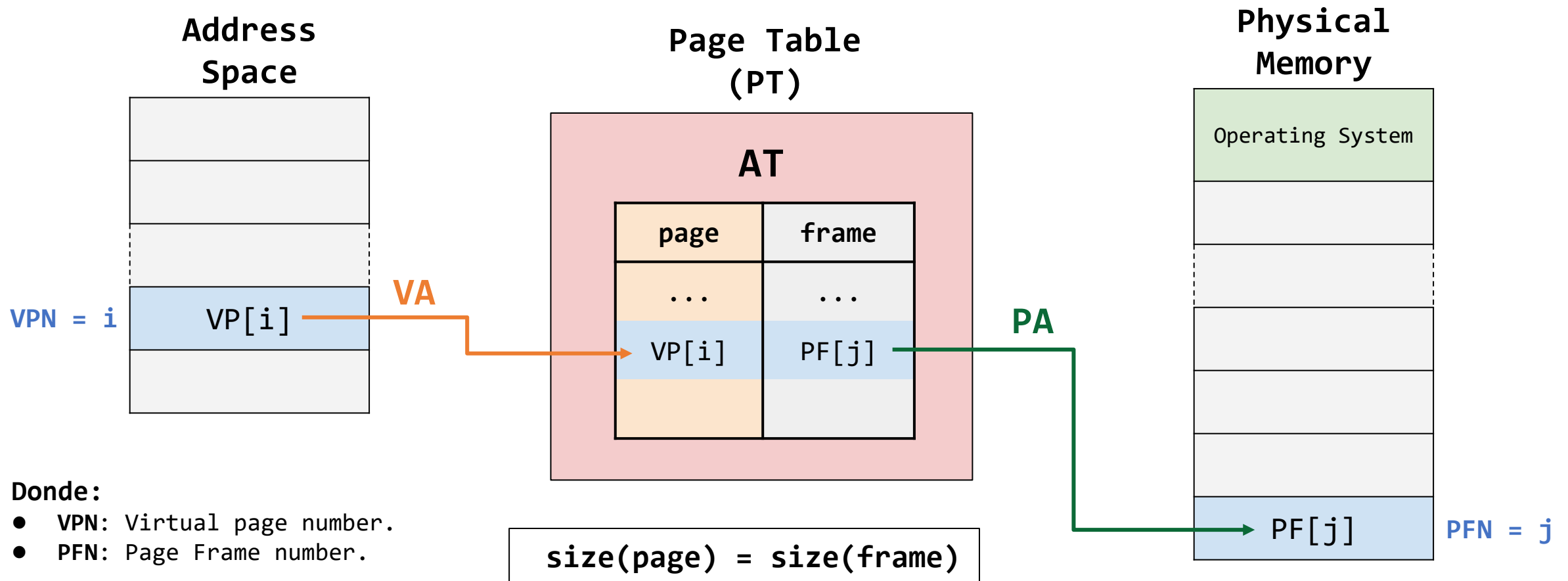
- Tamaño:  $\text{size}(\text{PM})$
- Número de frames:  $N$
- Tamaño de frame:  $\text{size}(\text{frame})$

$$\text{size}(\text{PM}) = N * \text{size}(\text{frame})$$

# Paginación – Conceptos claves

## Page Tables

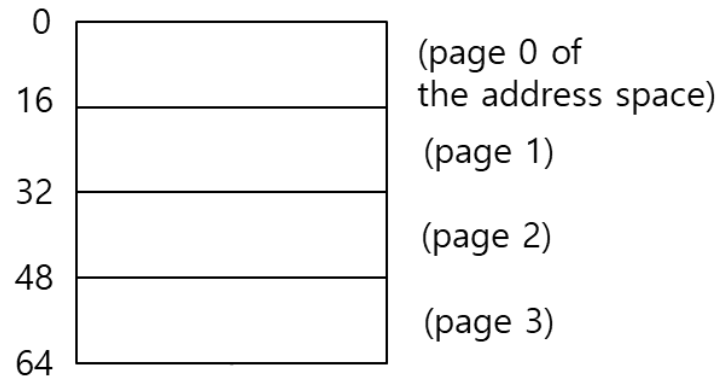
- Permite la traducción de direcciones virtuales (**VA**) a direcciones físicas (**PA**) mapeando páginas (**pages**) en memoria virtual a marcos (**frames**) en memoria física.



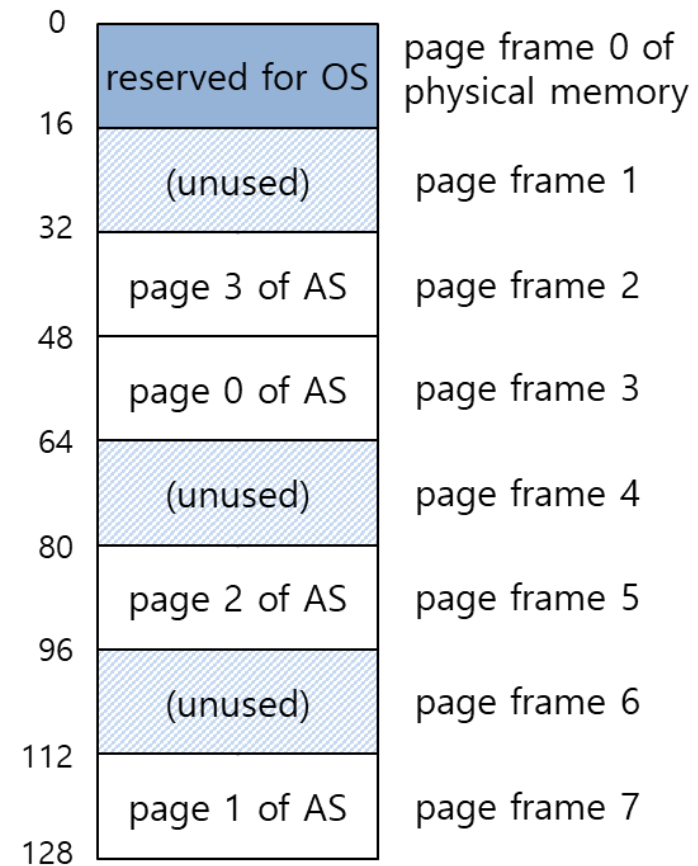
# Paginación - Ejemplo

## Ejemplo (Muy ideal)

- Memoria física de 128 bytes con frames (marcos de página) de 16 bytes.
- Espacio de direcciones (Address Space) de 64 bytes con páginas de 16 bytes.



**A Simple 64-byte Address Space**



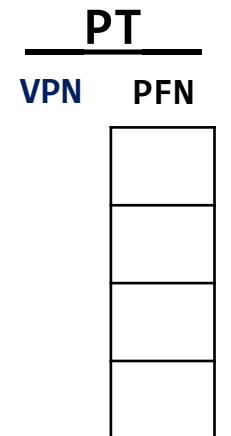
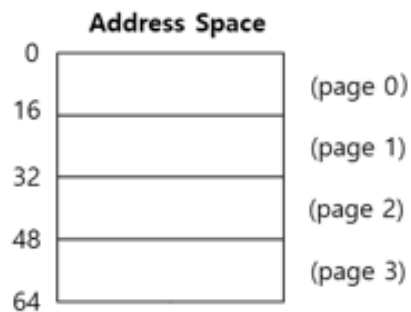
**64-Byte Address Space Placed In Physical Memory**

# Paginación - Ejemplo

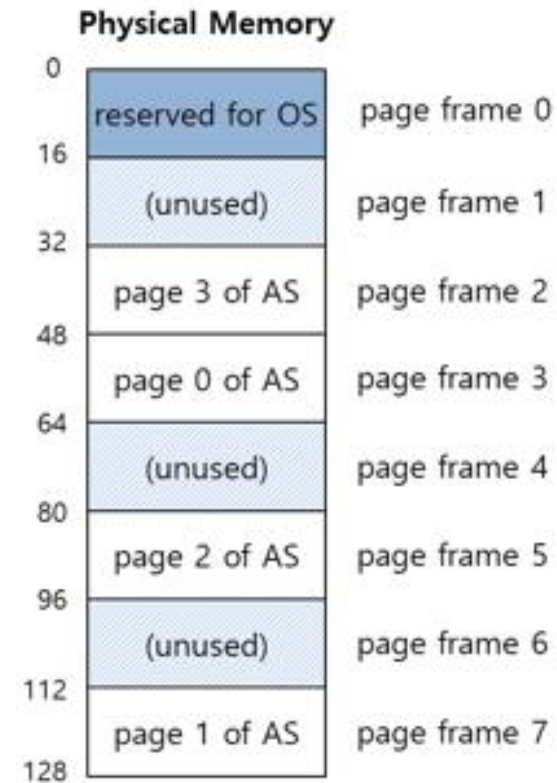
## Preguntas

Teniendo en cuenta la información del ejemplo anterior responda:

- ¿Cuál es el número de frames en que se divide la memoria física?
- ¿Cuál es el número de páginas (pages) en que se divide la memoria virtual?
- Teniendo en cuenta los diagramas asociados a la memoria virtual y física. ¿Cómo es la tabla de pagina?



$$PFN = AT[VPN]$$



# Paginación - Ejemplo

## Preguntas

Teniendo en cuenta la información del ejemplo anterior:

- `size(PM)` = 128 B
- `size(frame)` = 16 B
- `size(AS)` = 64 B
- `size(page)` = 16 B

## Se pide:

- `num_frames` = ?
- `num_pages` = ?
- `PT[]` = ?

# Paginación - Ejemplo

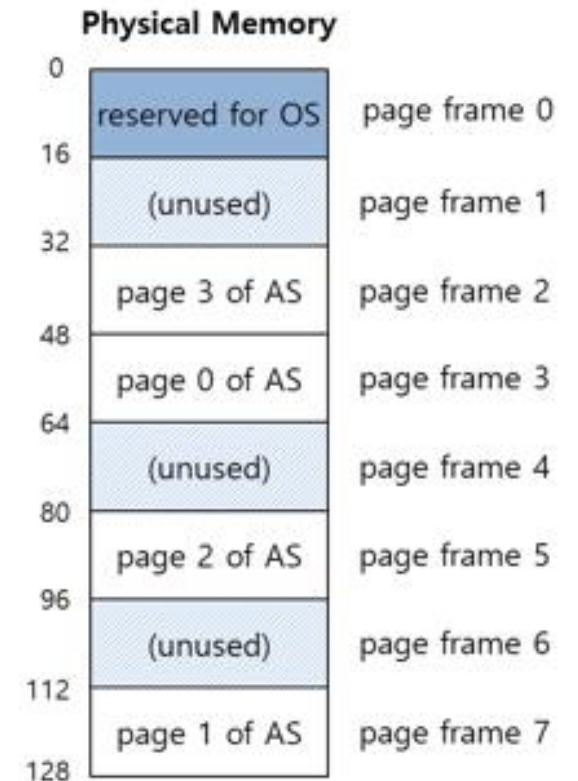
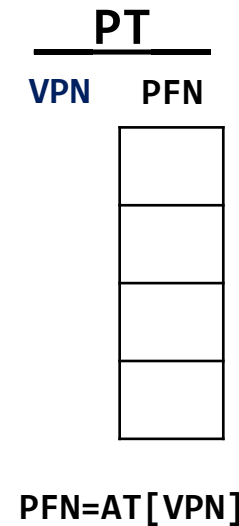
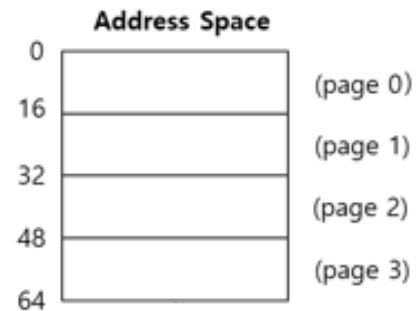
## Preguntas

Teniendo en cuenta la información del ejemplo anterior:

- $\text{size}(\text{PM}) = 128 \text{ B}$
- $\text{size}(\text{frame}) = 16 \text{ B}$
- $\text{size}(\text{AS}) = 64 \text{ B}$
- $\text{size}(\text{page}) = 16 \text{ B}$

Se pide:

- $\text{num\_frames} = ?$
- $\text{num\_pages} = ?$
- $\text{PT}[] = \{?, ?, ?, ?\}$



# Paginación - Ejemplo

## Preguntas

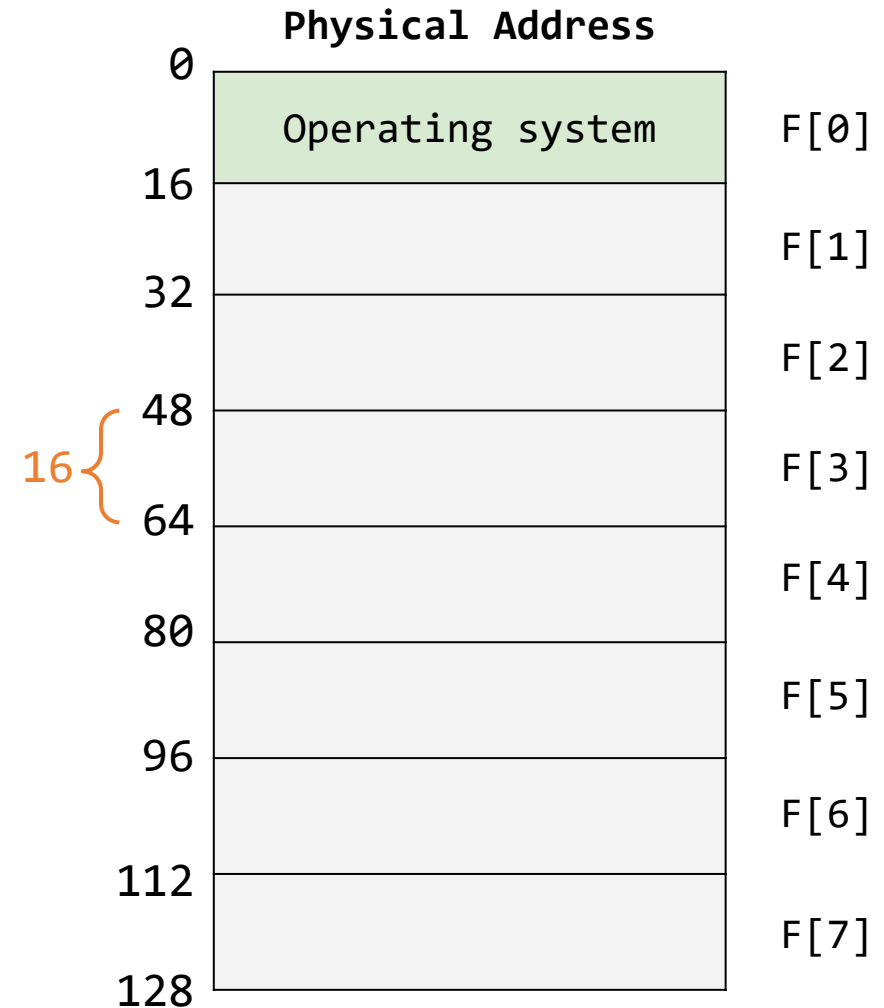
- ¿Cuál es el número de frames en que se divide la memoria física?
  - `size(PM) = 128 B`
  - `size(frame) = 16 B`

`num_frames = size(PM)/size(frame)`

`num_frames = 128 Bytes/16 Bytes`

`num_frames = 8`

**Solución:** 8 frames de 16 Bytes cada uno.





# Paginación - Ejemplo

## Preguntas

- ¿Cuál es el número de páginas (pages) en que se divide la memoria virtual?
  - `size(AS) = 64 B`
  - `size(page) = 16 B`

`num_pages = size(AS)/size(page)`

`num_pages = 64 Bytes/16 Bytes`

`num_pages = 4`

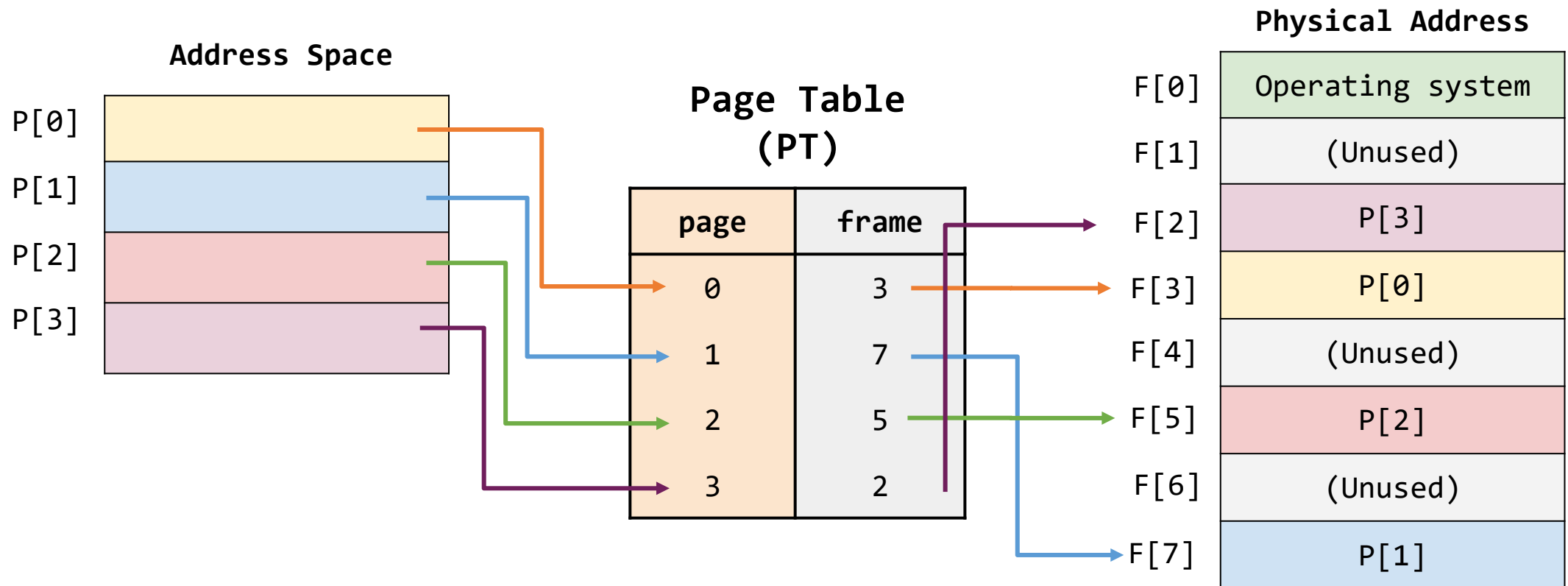
**Solución:** 4 páginas de 16 Bytes cada una.



# Paginación - Ejemplo

## Preguntas

- Teniendo en cuenta los esquemas asociados a las memorias física y virtual, ¿Cómo sería la **tabla de página** asociada?

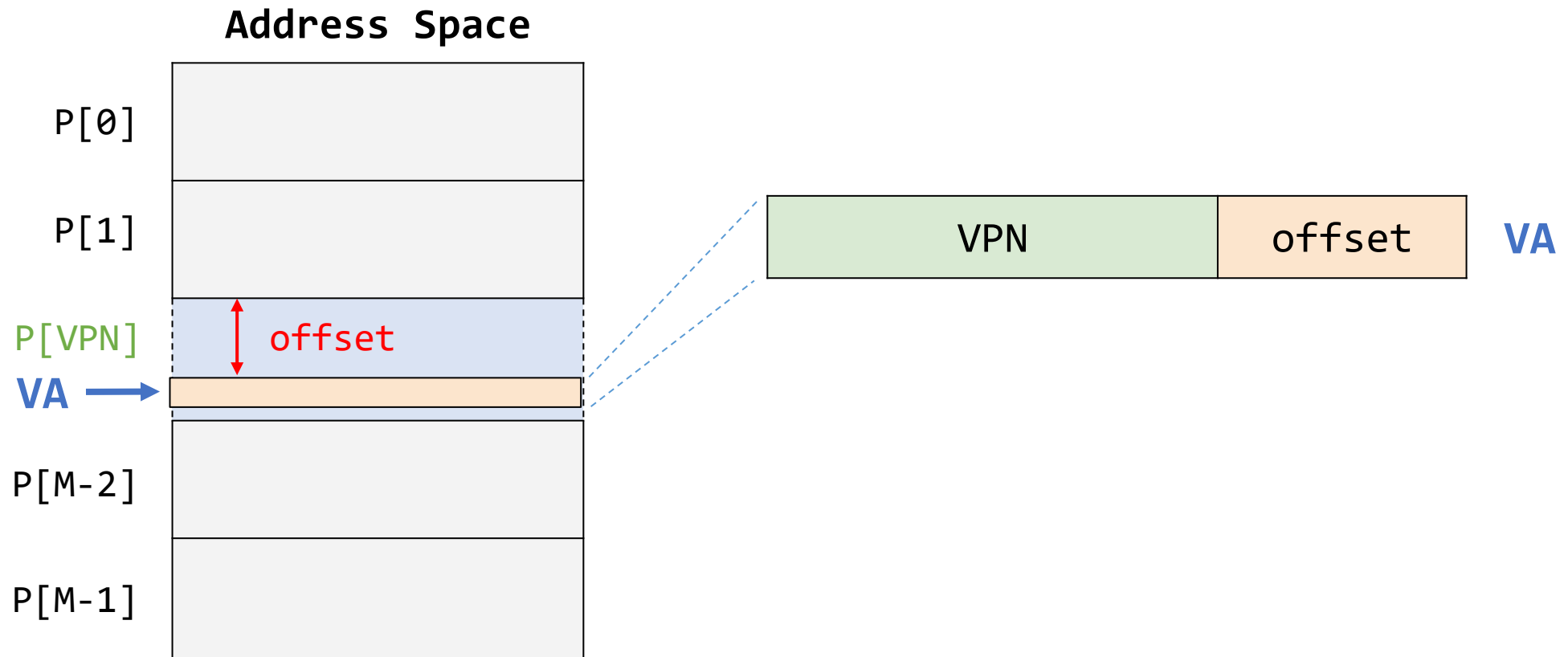


# Traducción de direcciones – Direcciones Virtuales

## Direcciones virtuales

Una dirección virtual está compuesta por dos partes:

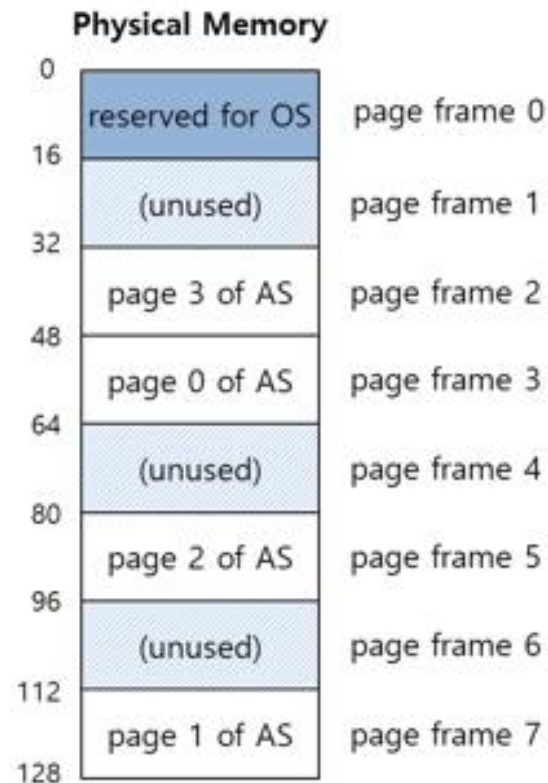
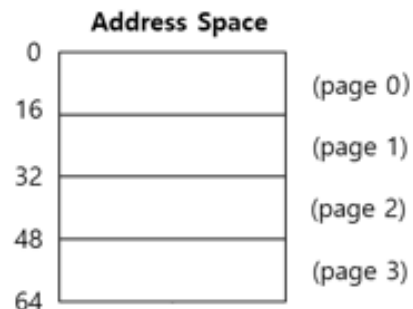
- **VPN (Virtual Page number)**: Índice asociado a una página en particular.
- **Offset**: Desplazamiento dentro de la página seleccionada.



# Traducción de direcciones – Direcciones Virtuales

## Ejemplo - Enunciado del ejemplo original

- Memoria física de 128 bytes con frames (marcos de página) de 16 bytes.
- Espacio de direcciones (Address Space) de 64 bytes con páginas de 16 bytes.



### Datos:

- $\text{size}(\text{PM}) = 128 \text{ B}$
- $\text{size}(\text{frame}) = 16 \text{ B}$
- $\text{size}(\text{AS}) = 64 \text{ B}$
- $\text{size}(\text{page}) = 16 \text{ B}$
- $\text{num\_pages} = 4$
- $\text{num\_frames} = 8$

### Preguntas:

1. ¿Cuántos bits se necesitan para direccionar la memoria virtual?
2. ¿Cuántos bits se necesitan para referirse a cada página?
3. ¿Cuántos bits se necesitan para acceder a cada las direcciones de cada página?
4. ¿Cuál es el VPN y el offset de la dirección virtual 21 en el AS de 64 B?

# Traducción de direcciones – Direcciones Virtuales

## Ejemplo - Enunciado del ejemplo original

### Datos:

- `size(PM)` = 128 B
- `size(frame)` = 16 B
- `size(AS)` = 64 B
- `size(page)` = 16 B
- `pages` = 4
- `frames` = 8

### Preguntas:

1. ¿Cuántos bits se necesitan para direccionar la memoria virtual?
2. ¿Cuántos bits se necesitan para referirse a cada página?
3. ¿Cuántos bits se necesitan para acceder a cada las direcciones de cada página?

# Traducción de direcciones – Direcciones Virtuales

## Ejemplo - Enunciado del ejemplo original

### Datos:

- `size(PM) = 128 B`
- `size(frame) = 16 B`
- `size(AS) = 64 B`
- `size(page) = 16 B`
- `pages = 4`
- `frames = 8`

### Preguntas:

4. ¿Cuál es el VPN y el offset de la dirección virtual 21 en el AS de 64 B?

# Traducción de direcciones – Direcciones Virtuales

## Ejemplo - Enunciado del ejemplo original

### Datos:

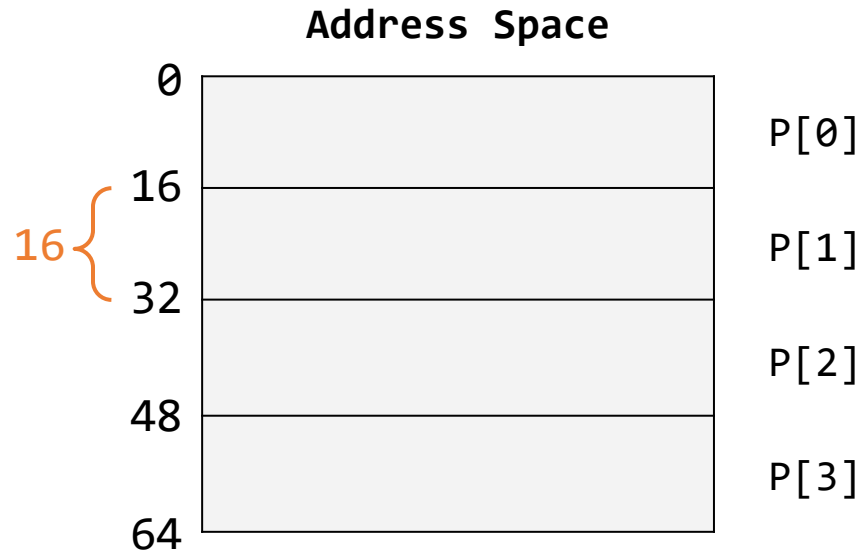
- `size(PM)` = 128 B
- `size(frame)` = 16 B
- `size(AS)` = 64 B
- `size(page)` = 16 B
- `pages` = 4
- `frames` = 8

### Preguntas:

1. ¿Cuántos bits se necesitan para acceder a cada las direcciones de cada página?
2. ¿Cuál es el VPN y el offset de la dirección virtual 21 en el AS de 64 B?

# Traducción de direcciones – Direcciones Virtuales

## Ejemplo - Enunciado del ejemplo original – Pregunta 1



$$n = \log_2(\text{size})$$

### Datos:

- $\text{size}(\text{AS}) = 64 \text{ B}$
- $\text{size}(\text{page}) = 16 \text{ B}$
- Número de páginas = 4 (Obtenido previamente)

### Pregunta:

1. ¿Cuántos bits se necesitan para direccionar la memoria virtual?

- $\text{size}(\text{AS}) = 64 \text{ B}$
- $n = ?$

$$2^n = \text{size}(\text{AS})$$

$$2^n = 64$$

$$2^n = 2^6$$

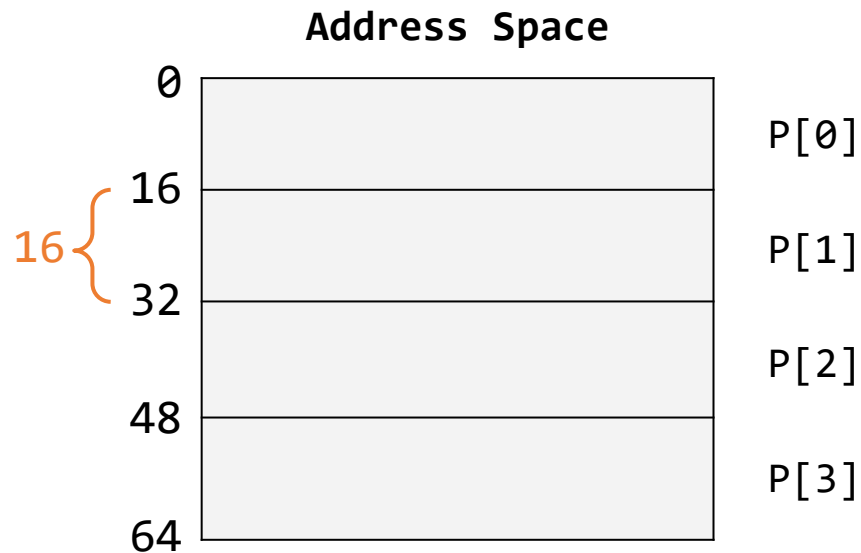
$$n = \log_2(2^6)$$

$$n = 6$$



# Traducción de direcciones – Direcciones Virtuales

## Ejemplo - Enunciado del ejemplo original – Pregunta 2



$$n = \log_2(\text{size})$$

### Datos:

- $\text{size}(\text{AS}) = 64 \text{ B}$
- $\text{size}(\text{page}) = 16 \text{ B}$
- Número de páginas = 4 (Obtenido previamente)

### Solución pregunta 2:

2. ¿Cuántos bits se necesitan para referirse a cada página?

- $\text{num\_pages} = 4$
- $n(\text{VPN}) = ?$

$$2^{n(\text{VPN})} = \text{num\_pages}$$

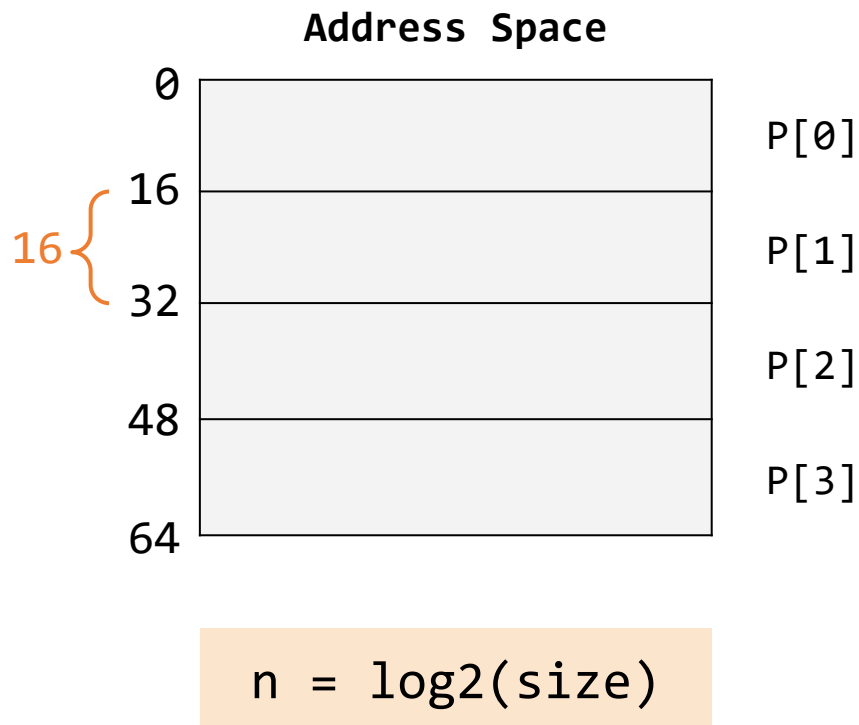
$$2^{n(\text{VPN})} = 4$$

$$2^{n(\text{VPN})} = 2^2$$

$$n(\text{VPN}) = 2$$

# Traducción de direcciones – Direcciones Virtuales

## Ejemplo - Enunciado del ejemplo original – Pregunta 3



### Datos:

- $\text{size}(\text{AS}) = 64 \text{ B}$
- $\text{size}(\text{page}) = 16 \text{ B}$
- Número de páginas = 4 (Obtenido previamente)

### Solución pregunta 3:

3. ¿Cuántos bits se necesitan para acceder a cada las direcciones de cada página?
- $\text{size}(\text{page}) = 16$
  - $n(\text{offset}) = ?$

$$2^{n(\text{offset})} = \text{size}(\text{page})$$

$$2^{n(\text{offset})} = 16$$

$$2^{n(\text{offset})} = 2^4$$

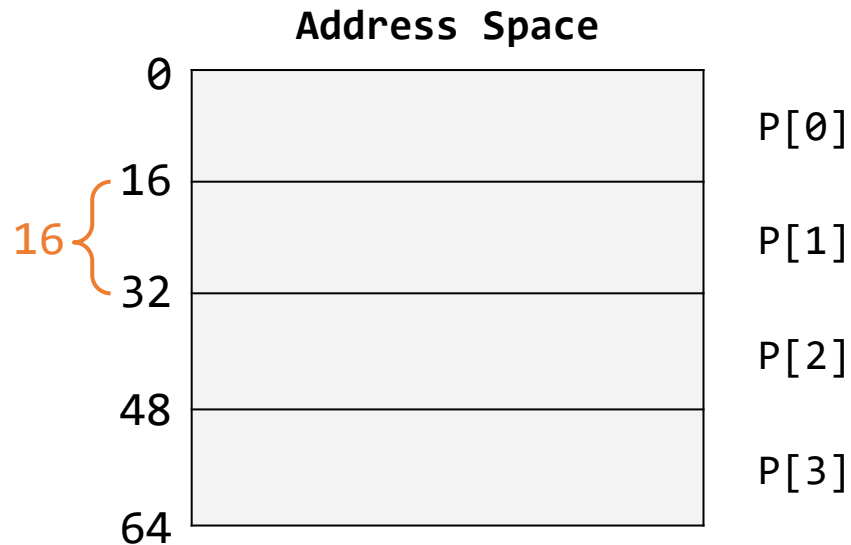
$$n(\text{offset}) = 4$$

# Traducción de direcciones – Direcciones Virtuales

## Ejemplo - Enunciado del ejemplo original – Resumen de lo que llevamos

### Resumen de los resultados:

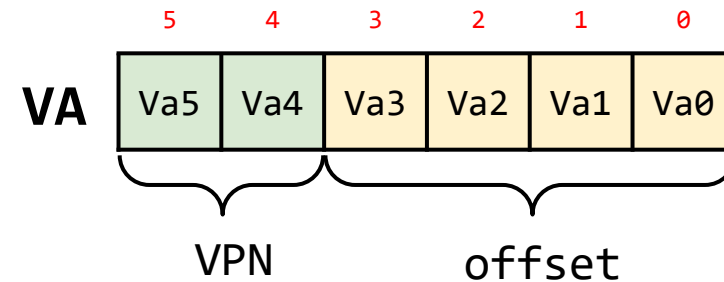
1.  $n(\text{VA}) = 6$
2.  $n(\text{VPN}) = 2$
3.  $n(\text{offset}) = 4$



$$n = \log_2(\text{size})$$

| Page size | Virt Addr. bits (VA) | High bits (VPN) | Low bits (offset) |
|-----------|----------------------|-----------------|-------------------|
| 16 bytes  | 6                    | 2               | 4                 |

**Conclusión:** Teniendo en cuenta los resultados anteriormente obtenidos, la dirección virtual tendrá la siguiente forma:



# Traducción de direcciones – Direcciones Virtuales

## Ejemplo - Enunciado del ejemplo original – Pregunta 4

### Solución pregunta 4:

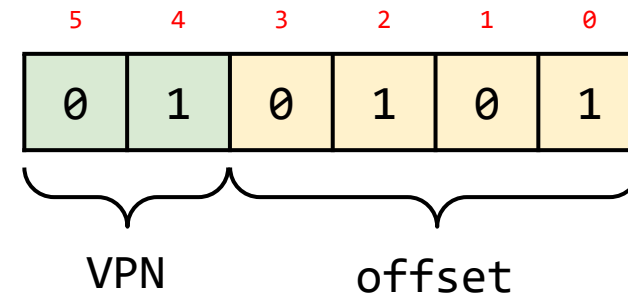
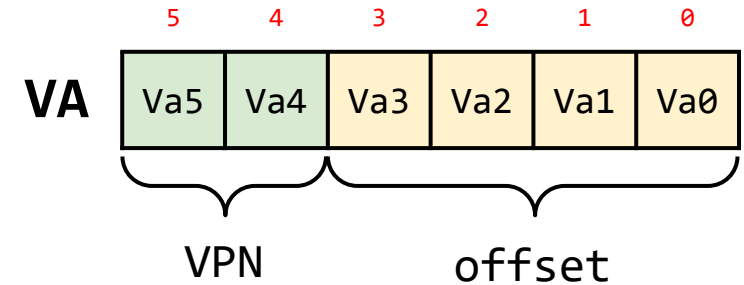
4. ¿Cuál es el VPN y el offset de la dirección virtual 21 en el AS de 64 B?

- VA = 21
- VPN = ?
- offset = ?

VA = 21 = 10101 = 010101 = [01|0101] = [VPN|offset]

- VPN = 01 = 1
- Offset = 0101 = 5

VA = 21 = 0x15 = 010101



# Traducción de direcciones – Direcciones Virtuales

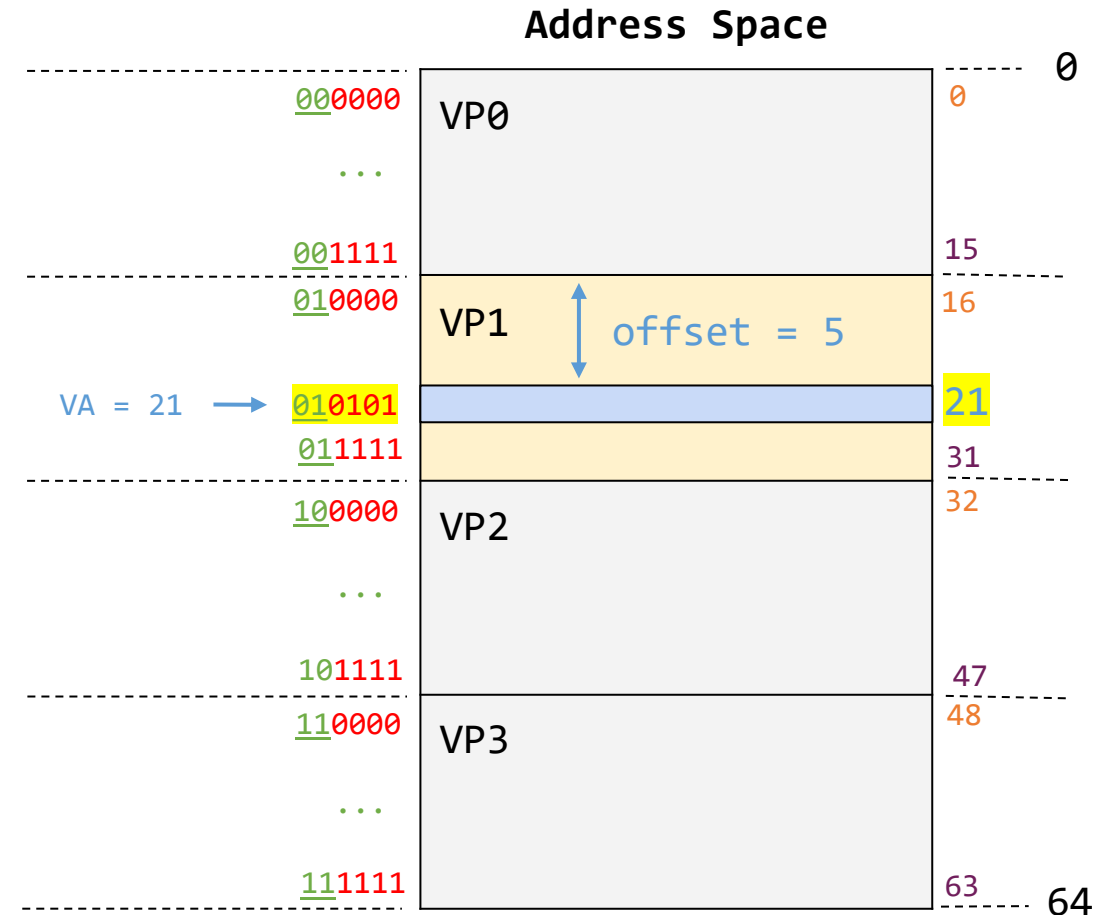
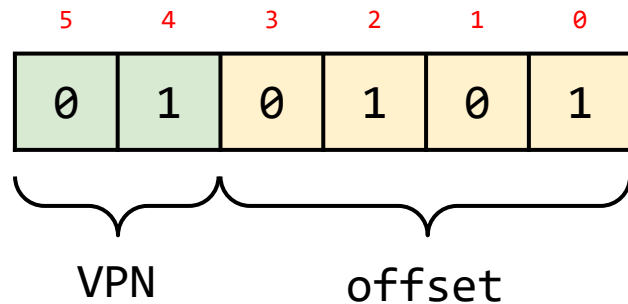
## Ejemplo - Enunciado del ejemplo original – Pregunta 4

### Solución pregunta 4:

4. ¿Cuál es el VPN y el offset de la dirección virtual 21 en el AS de 64 B?

- VA = 21
- VPN = ?
- offset = ?

$$VA = 21 = 0x15 = \textcolor{green}{0}\textcolor{red}{1}\textcolor{red}{0}\textcolor{red}{1}\textcolor{red}{0}\textcolor{red}{1}$$



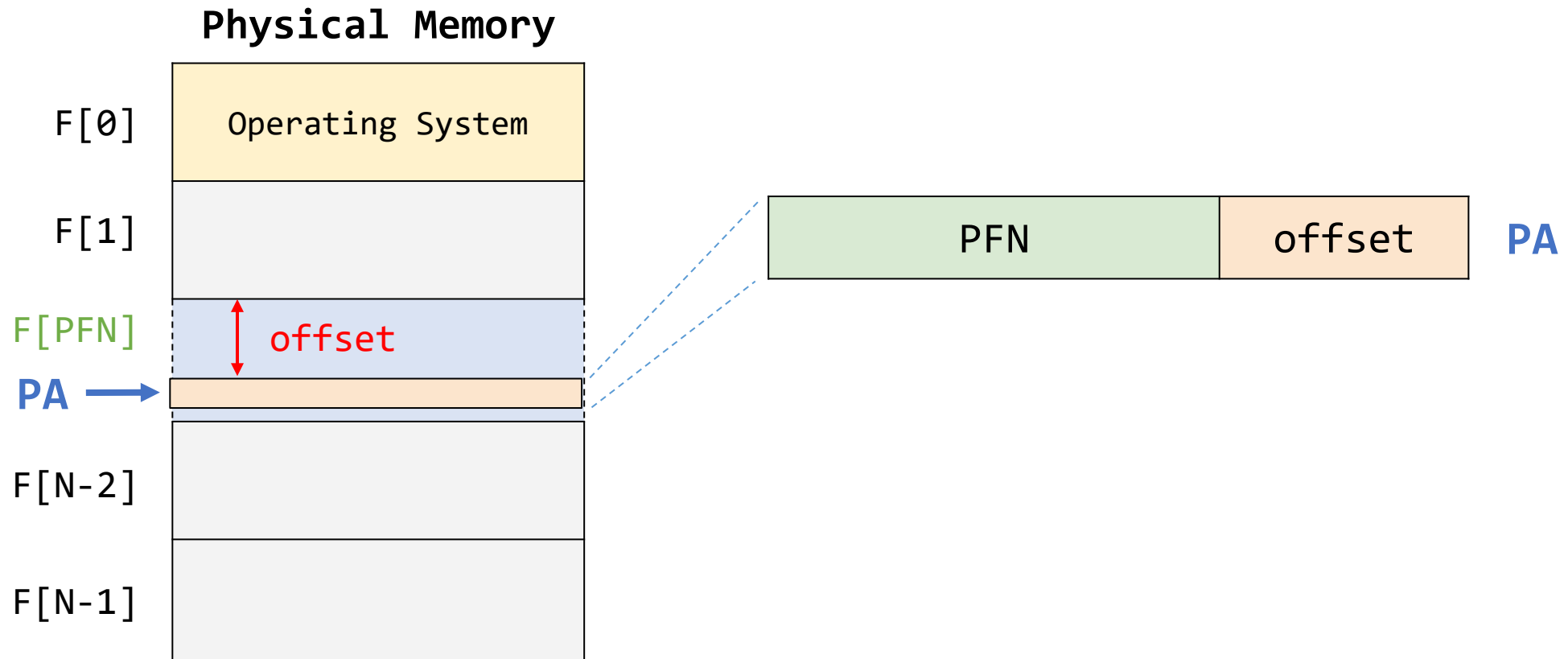
Notación:  $P[i] = VP_i$

# Traducción de direcciones – Direcciones físicas

## Direcciones físicas

Una dirección física está compuesta por dos partes:

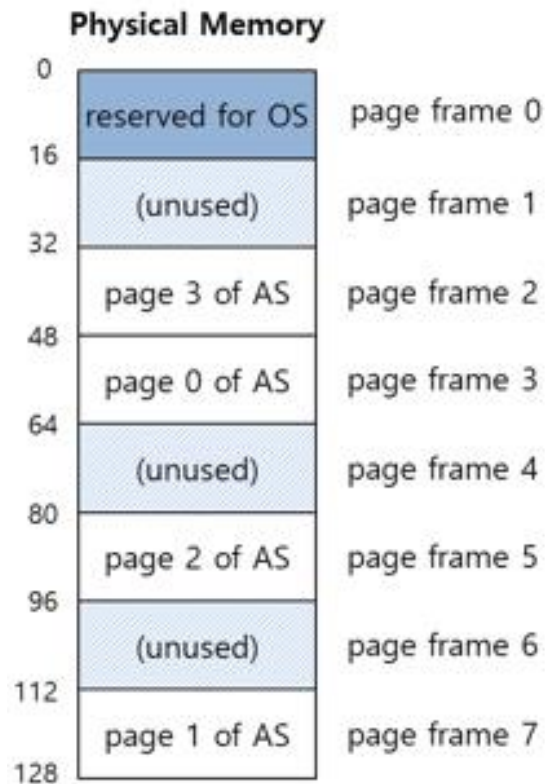
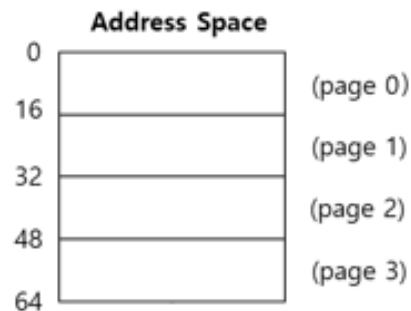
- **PFN (Page Frame number)**: Frame dentro del que se encuentra la dirección física.
- **Offset**: Desplazamiento dentro del frame seleccionado.



# Traducción de direcciones – Direcciones físicas

## Ejemplo - Enunciado del ejemplo original

- Memoria física de 128 bytes con frames (marcos de página) de 16 bytes.
- Espacio de direcciones (Address Space) de 64 bytes con páginas de 16 bytes.



## Datos:

- $\text{size}(\text{PM}) = 128 \text{ B}$
- $\text{size}(\text{frame}) = 16 \text{ B}$
- $\text{size}(\text{AS}) = 64 \text{ B}$
- $\text{size}(\text{page}) = 16 \text{ B}$
- $\text{num\_pages} = 4$
- $\text{num\_frames} = 8$

## Preguntas:

1. ¿Cuántos bits se necesitan para direccionar la memoria física?
2. ¿Cuántos bits se necesitan para hacer referencia a los frames?
3. ¿Cuántos bits se necesitan para el offset?
4. ¿Cómo es el formato de la dirección para este ejemplo?

# Traducción de direcciones – Direcciones Virtuales

## Ejemplo - Enunciado del ejemplo original

### Datos:

- `size(PM)` = 128 B
- `size(frame)` = 16 B
- `size(AS)` = 64 B
- `size(page)` = 16 B
- `pages` = 4
- `frames` = 8

### Preguntas:

1. ¿Cuántos bits se necesitan para direccionar la memoria física?
2. ¿Cuántos bits se necesitan para hacer referencia a los frames?
3. ¿Cuántos bits se necesitan para el offset?



# Traducción de direcciones – Direcciones Virtuales

## Ejemplo - Enunciado del ejemplo original

### Datos:

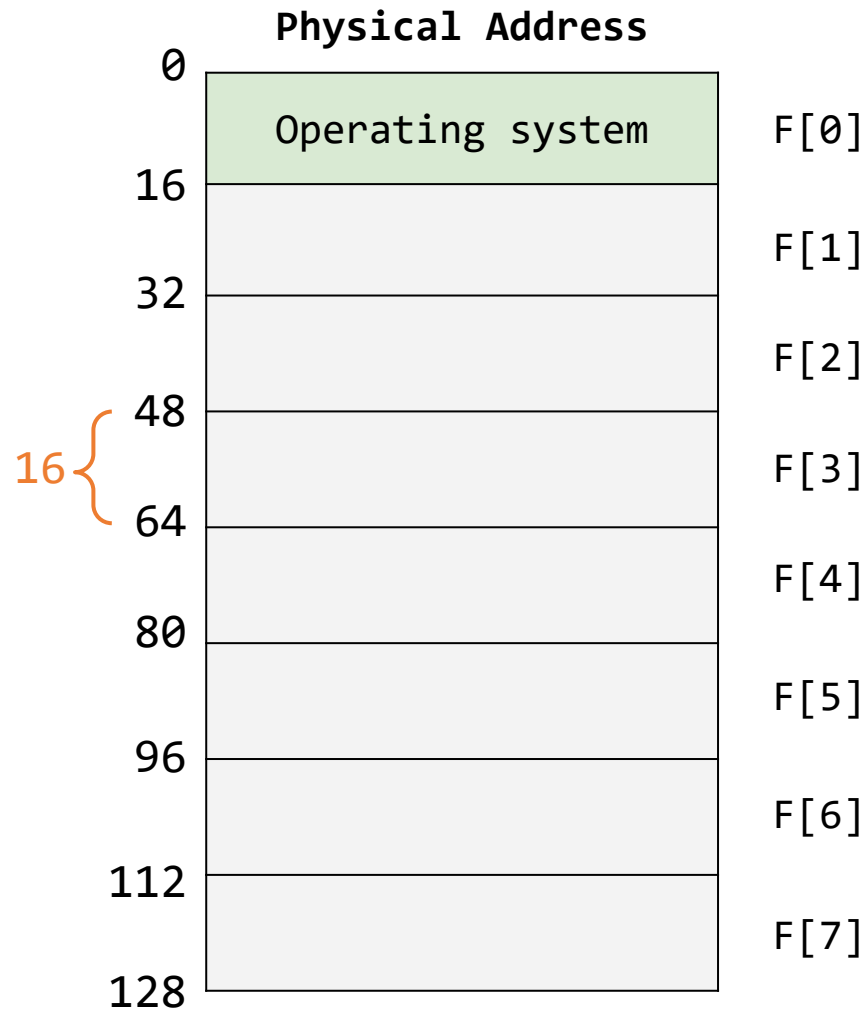
- `size(PM)` = 128 B
- `size(frame)` = 16 B
- `size(AS)` = 64 B
- `size(page)` = 16 B
- `pages` = 4
- `frames` = 8

### Preguntas:

4. ¿Cómo es el formato de la dirección para este ejemplo?

# Traducción de direcciones – Direcciones físicas

## Ejemplo - Enunciado del ejemplo original – Pregunta 1



### Datos:

- $\text{size}(\text{PM}) = 128 \text{ B}$
- $\text{size}(\text{page}) = 16 \text{ B}$
- Número de frames = 8 (Obtenido previamente)

### Pregunta:

1. ¿Cuántos bits se necesitan para direccionar la memoria física?

- $\text{size}(\text{PM}) = 128 \text{ B}$
- $n = ?$

$$2^n = \text{size}(\text{PM})$$

$$2^n = 128$$

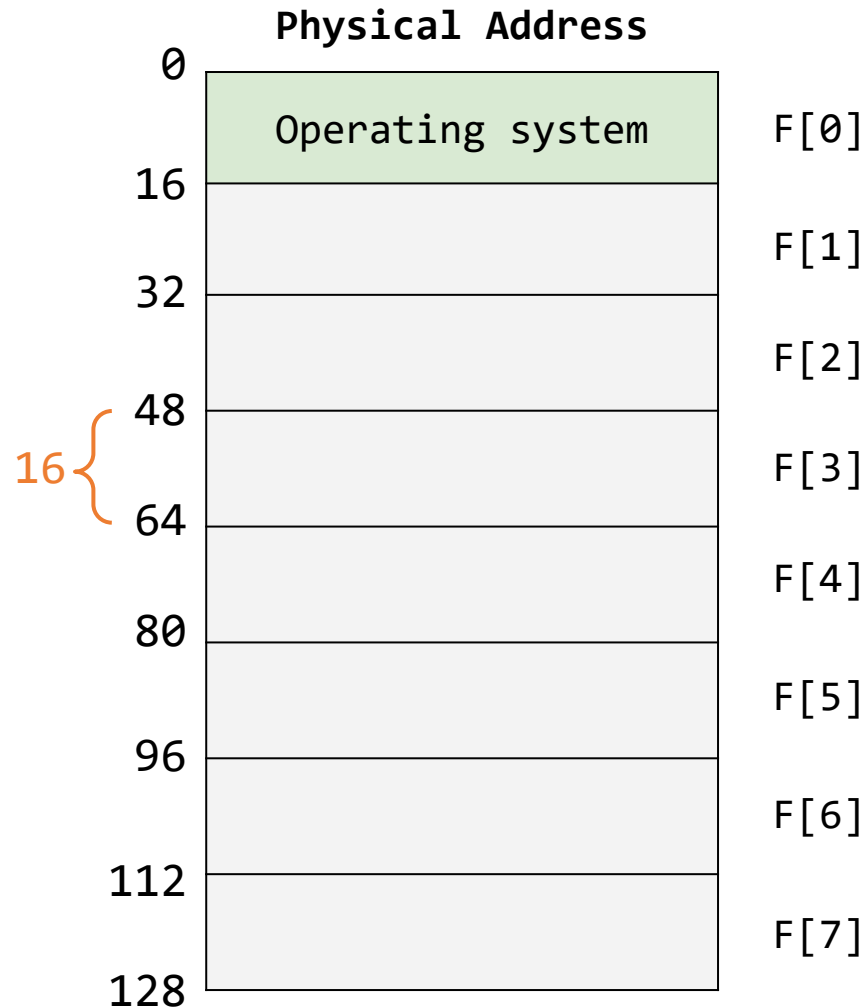
$$2^n = 2^7$$

$$n = \log_2(2^7)$$

$$n = 7$$

# Traducción de direcciones – Direcciones físicas

## Ejemplo - Enunciado del ejemplo original – Pregunta 2



### Datos:

- $\text{size}(\text{PM}) = 128 \text{ B}$
- $\text{size}(\text{page}) = 16 \text{ B}$
- Número de frames = 8 (Obtenido previamente)

### Pregunta:

2. ¿Cuántos bits se necesitan para hacer referencia a los frames?

- $\text{num\_frames} = 8$
- $n(\text{PFN}) = ?$

$$2^{n(\text{PFN})} = \text{num\_frames}$$

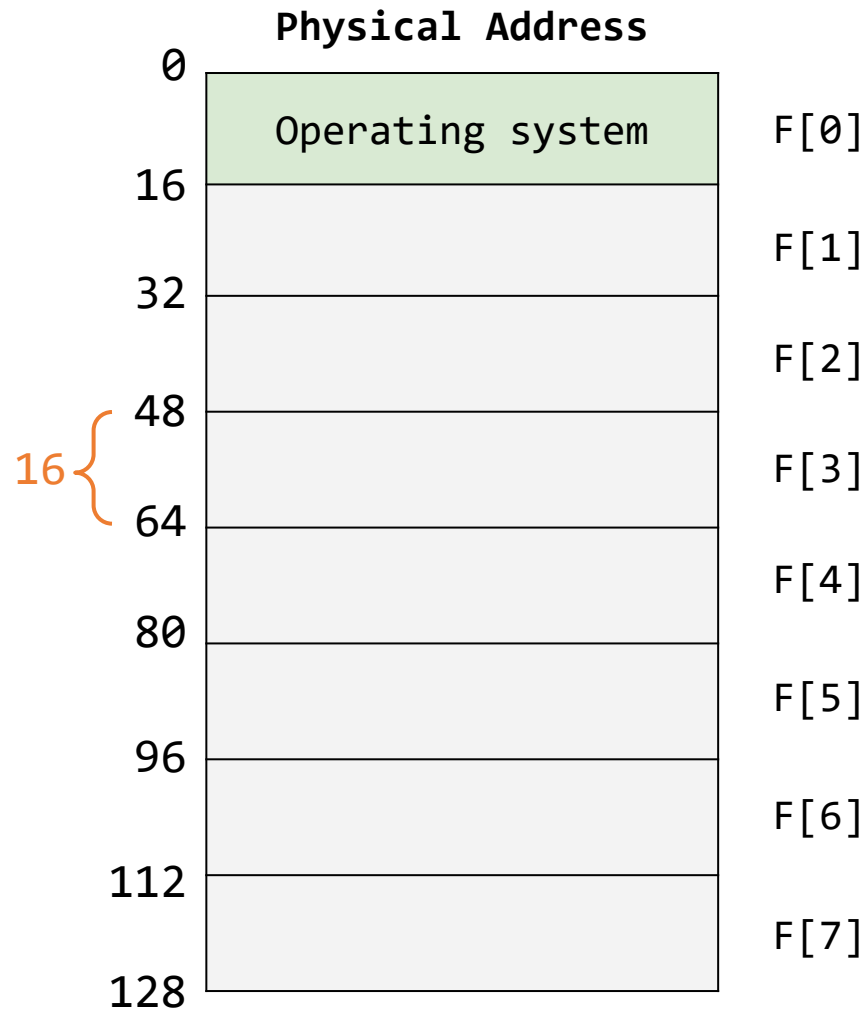
$$2^{n(\text{PFN})} = 8$$

$$2^{n(\text{PFN})} = 2^3$$

$$n(\text{PFN}) = 3$$

# Traducción de direcciones – Direcciones físicas

## Ejemplo - Enunciado del ejemplo original – Pregunta 3



### Datos:

- $\text{size}(\text{PM}) = 128 \text{ B}$
- $\text{size}(\text{page}) = 16 \text{ B}$
- Número de frames = 8 (Obtenido previamente)

### Pregunta:

3. ¿Cuántos bits se necesitan para el offset?

- $n = 7$
- $n(\text{PFN}) = 3$
- $n(\text{offset}) = ?$

$$n = n(\text{PFN}) + n(\text{offset})$$

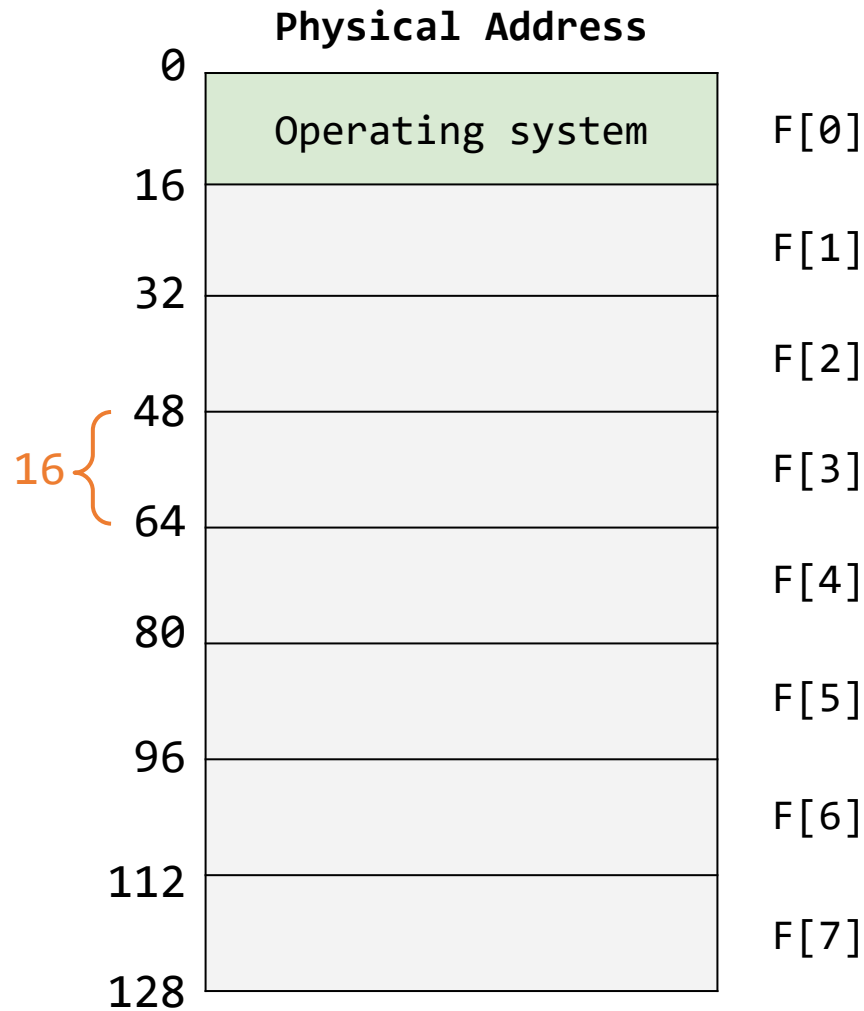
$$n(\text{offset}) = n - n(\text{PFN})$$

$$n(\text{offset}) = 7 - 3$$

$$n(\text{offset}) = 4$$

# Traducción de direcciones – Direcciones físicas

## Ejemplo - Enunciado del ejemplo original – Pregunta 4

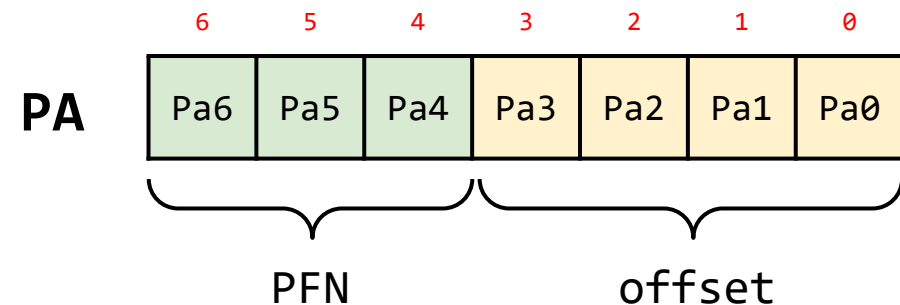


### Resumen de los resultados:

1.  $n(\text{PA}) = 7$
2.  $n(\text{PFN}) = 3$
3.  $n(\text{offset}) = 4$

| Page size | Phys Addr. bits (PA) | High bits (PFN) | Low bits (offset) |
|-----------|----------------------|-----------------|-------------------|
| 16 bytes  | 7                    | 3               | 4                 |

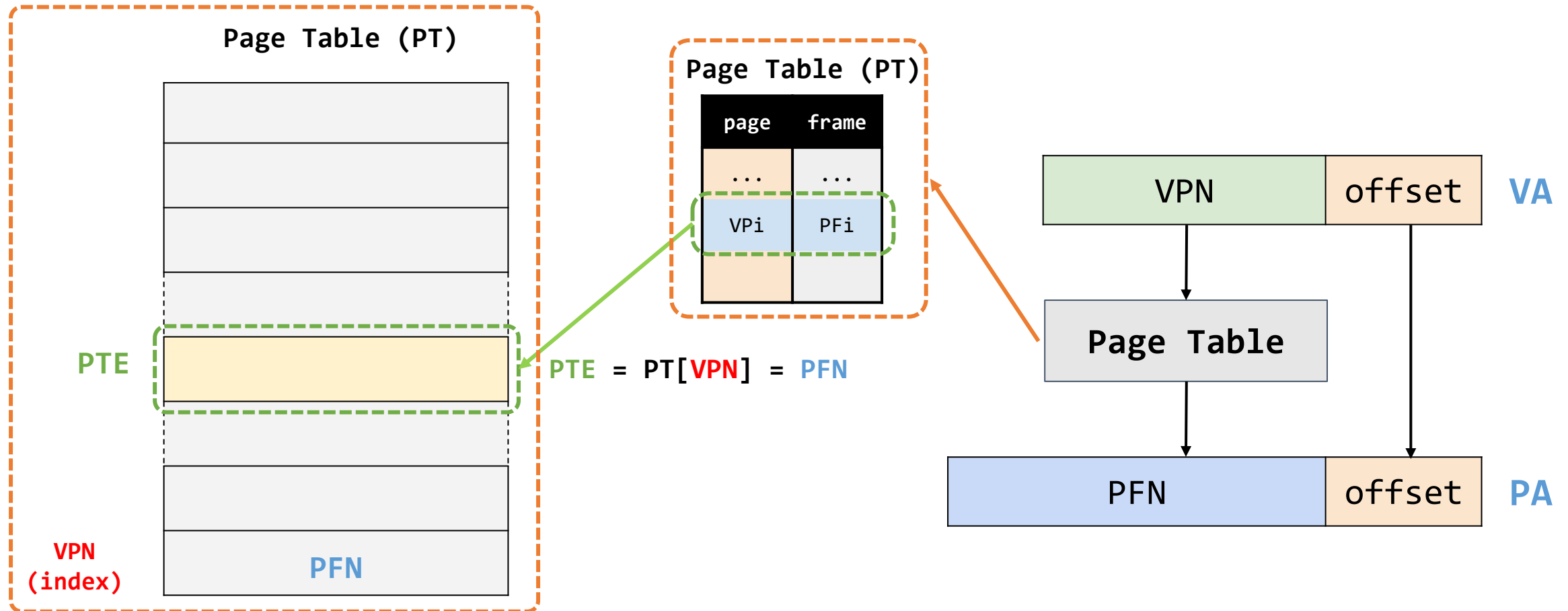
**Conclusión:** Teniendo en cuenta los resultados anteriormente obtenidos, la dirección física tendrá la siguiente forma:



# Traducción de direcciones – Tabla de pagina

## Tabla de página (Page Table)

Array que traduce direcciones virtuales a físicas.



**PTE (Page Table Entry):** Hay una por cada página en el AS.

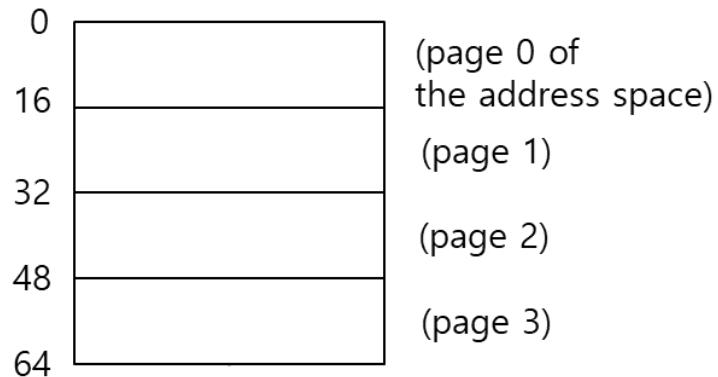
# Traducción de direcciones – Tabla de pagina

## Ejemplo - Enunciado del ejemplo original

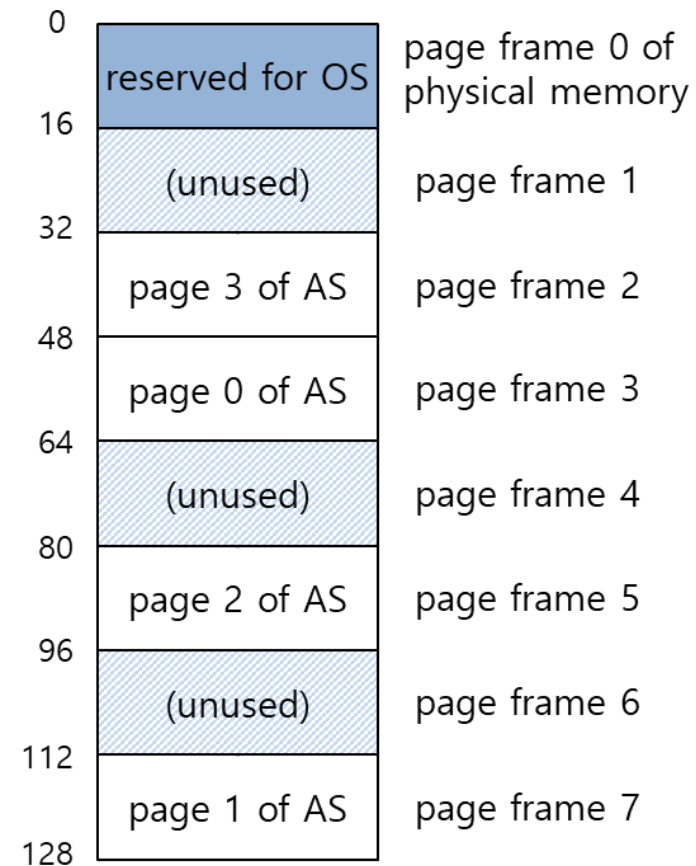
- Memoria física de 128 bytes con frames (marcos de página) de 16 bytes.
- Espacio de direcciones (Address Space) de 64 bytes con páginas de 16 bytes.

### Preguntas:

1. ¿Bosqueje la tabla de página?
2. ¿Cuál es la dirección física asociada a la dirección virtual 21?



**A Simple 64-byte Address Space**



**64-Byte Address Space Placed In Physical Memory**

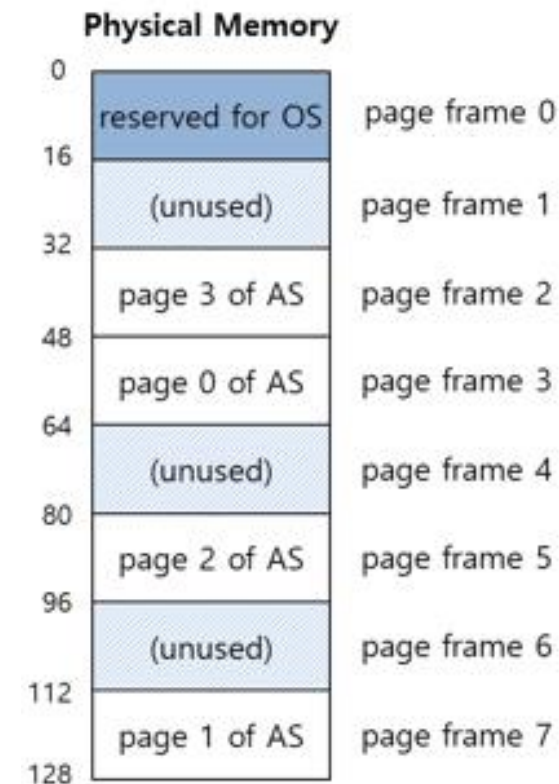
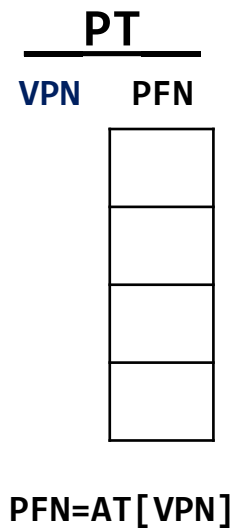
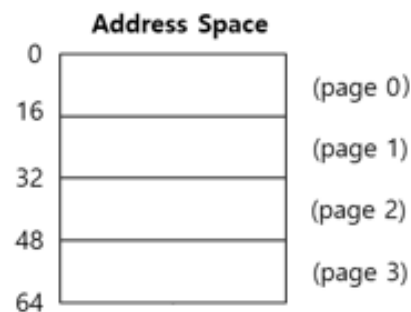
# Traducción de direcciones – Tabla de pagina

## Ejemplo - Enunciado del ejemplo original - Pregunta 1

- Memoria física de 128 bytes con frames (marcos de página) de 16 bytes.
- Espacio de direcciones (Address Space) de 64 bytes con páginas de 16 bytes.

### Preguntas:

1. ¿Bosqueje la tabla de página?





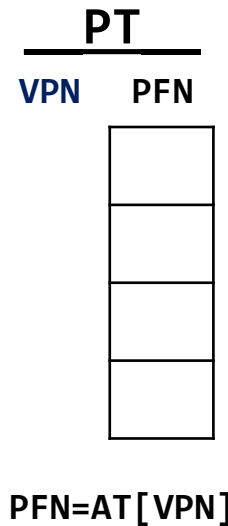
# Traducción de direcciones – Tabla de pagina

## Ejemplo - Enunciado del ejemplo original – Pregunta 2

- Memoria física de 128 bytes con frames (marcos de página) de 16 bytes.
- Espacio de direcciones (Address Space) de 64 bytes con páginas de 16 bytes.

### Preguntas:

2. ¿Cuál es la dirección física asociada a la dirección virtual 21?

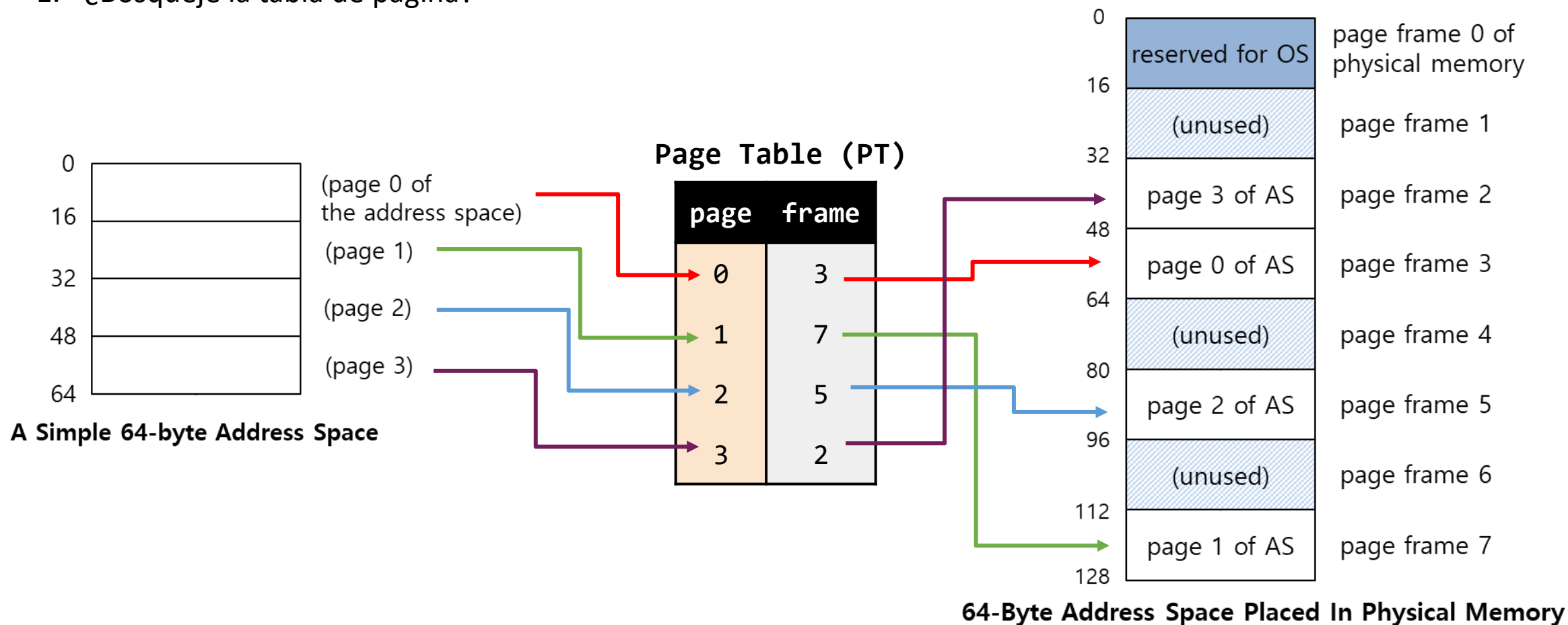


# Traducción de direcciones – Tabla de pagina

## Ejemplo - Enunciado del ejemplo original

### Preguntas

1. ¿Bosqueje la tabla de página?

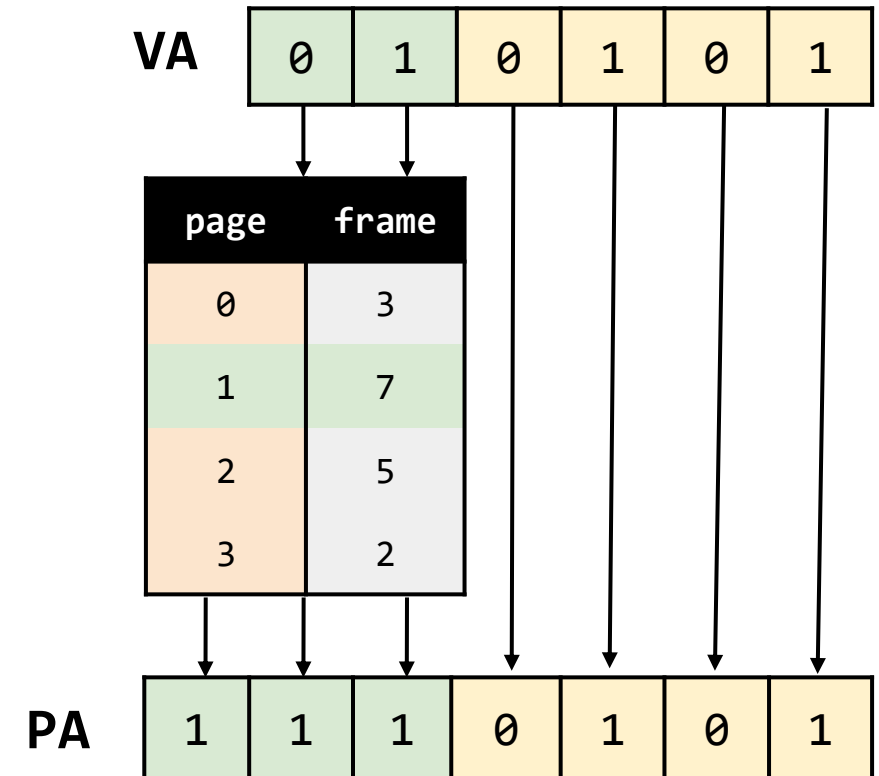
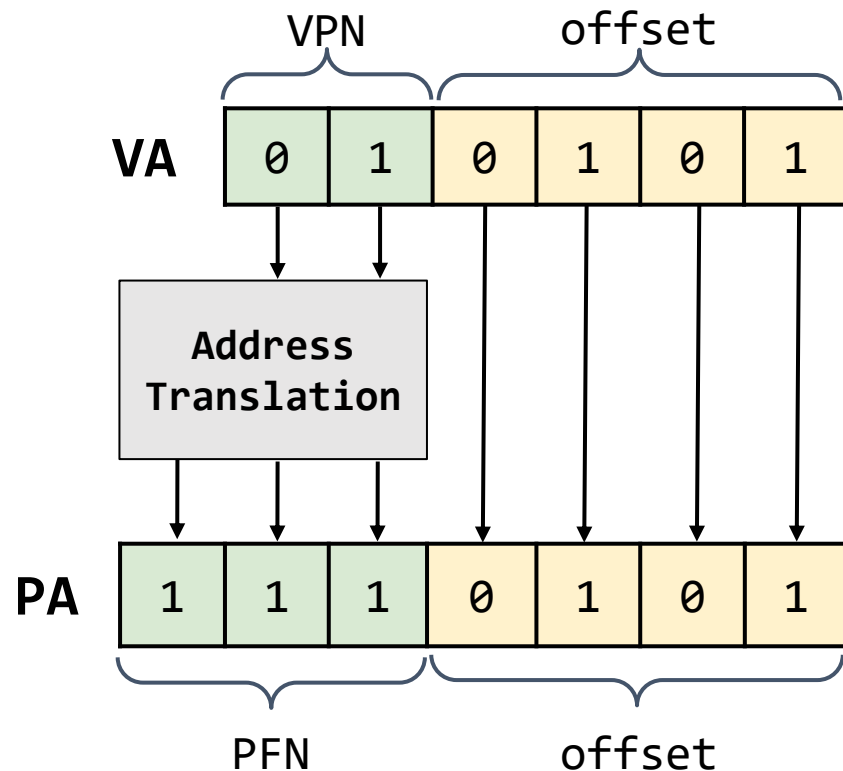


# Traducción de direcciones – Tabla de pagina

## Ejemplo - Enunciado del ejemplo original

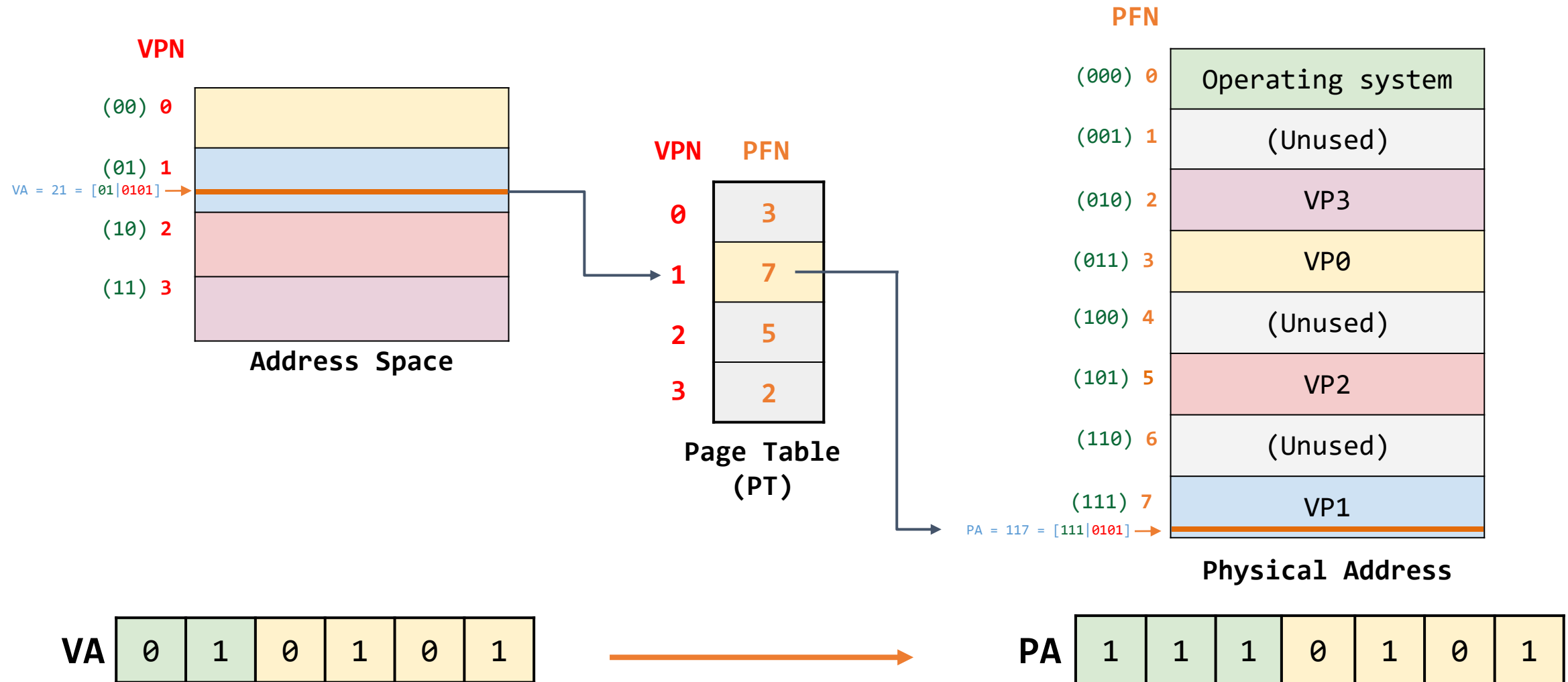
### Preguntas:

2. ¿Cual es la dirección física asociada a la dirección virtual 21?
- $VA = 21 = 0x15 = 010101$
  - $PA = ?$



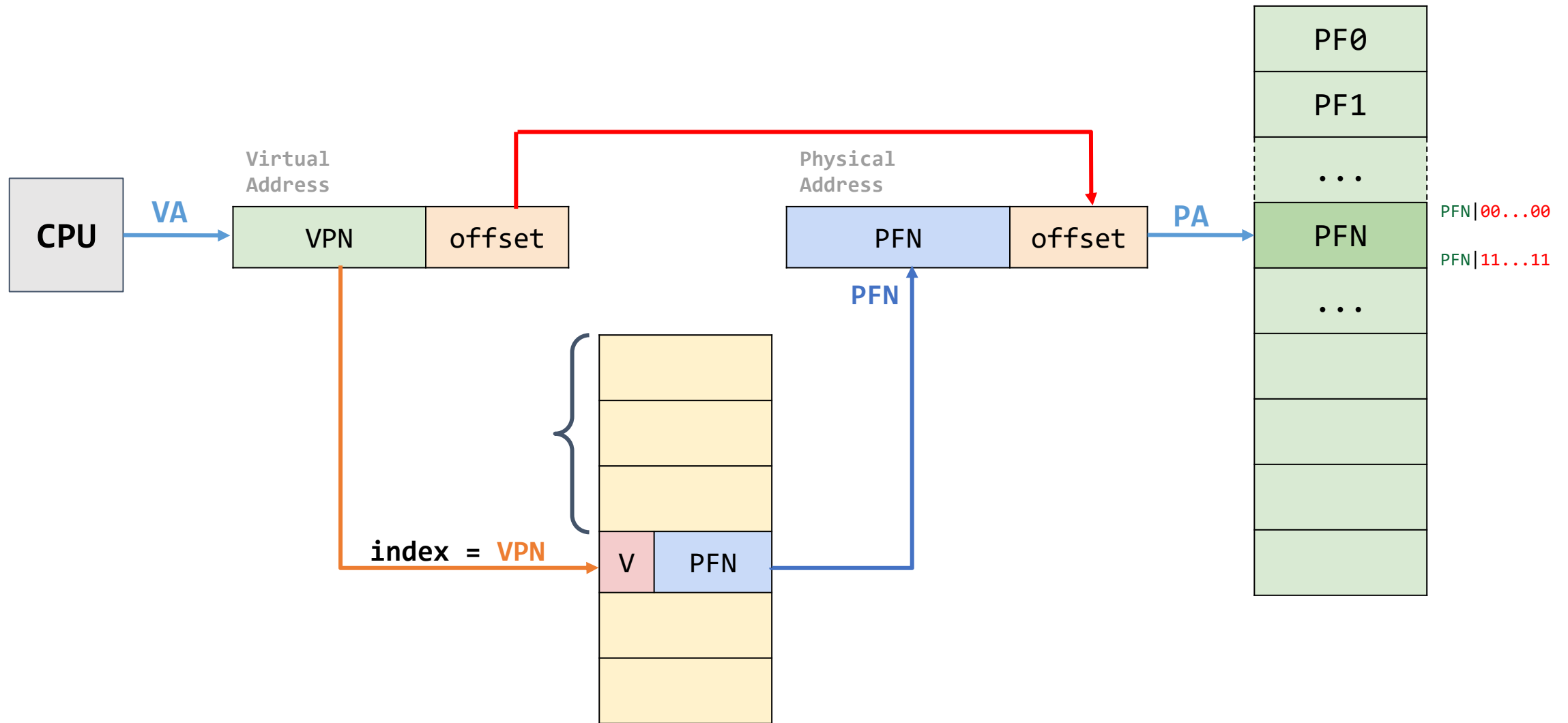
# Traducción de direcciones

## Ejemplo - Enunciado del ejemplo original



# Traducción de direcciones – HW de traducción

## Hardware para la traducción



# Traducción de direcciones

## Ejemplo

1. A continuación se muestra una memoria virtual y una física que almacenan datos de 1 byte. Teniendo en cuenta la información anterior y la siguiente figura, completar la siguiente información:

- a. Número de páginas: \_\_\_\_\_
- b. Número de frames: \_\_\_\_\_
- c. Tamaño de la página: \_\_\_\_\_
- d. Offset: \_\_\_\_\_ bits
- e. VPN: \_\_\_\_\_ bits
- f. PFN: \_\_\_\_\_ bits
- g. Número total de PTEs: \_\_\_\_\_

|    |   |
|----|---|
| 0  | a |
| 1  | b |
| 2  | c |
| 3  | d |
| 4  | e |
| 5  | f |
| 6  | g |
| 7  | h |
| 8  | i |
| 9  | j |
| 10 | k |
| 11 | l |
| 12 | m |
| 13 | n |
| 14 | o |
| 15 | p |

logical memory

|   |   |
|---|---|
| 0 | 5 |
| 1 | 6 |
| 2 | 1 |
| 3 | 2 |

page table

|    |                  |
|----|------------------|
| 0  |                  |
| 4  | i<br>j<br>k<br>l |
| 8  | m<br>n<br>o<br>p |
| 12 |                  |
| 16 |                  |
| 20 | a<br>b<br>c<br>d |
| 24 | e<br>f<br>g<br>h |
| 28 |                  |

physical memory

# Traducción de direcciones

## Ejemplo

2. Teniendo en cuenta la tabla de página (page table). Llenar la siguiente tabla:

| Virtual Address | Physical Address |
|-----------------|------------------|
| 2               |                  |
| 5               |                  |
| 6               |                  |
| 11              |                  |
| 12              |                  |

$$PA = PFN * \text{size}(\text{page}) + \text{offset}$$

|    |   |
|----|---|
| 0  | a |
| 1  | b |
| 2  | c |
| 3  | d |
| 4  | e |
| 5  | f |
| 6  | g |
| 7  | h |
| 8  | i |
| 9  | j |
| 10 | k |
| 11 | l |
| 12 | m |
| 13 | n |
| 14 | o |
| 15 | p |

logical memory

|   |   |
|---|---|
| 0 | 5 |
| 1 | 6 |
| 2 | 1 |
| 3 | 2 |

page table

|    |                  |
|----|------------------|
| 0  |                  |
| 4  | i<br>j<br>k<br>l |
| 8  | m<br>n<br>o<br>p |
| 12 |                  |
| 16 |                  |
| 20 | a<br>b<br>c<br>d |
| 24 | e<br>f<br>g<br>h |
| 28 |                  |

physical memory

# Traducción de direcciones

## Ejemplo

3. Suponga que se va a obtener de memoria la letra **h** a un registro de la CPU (llamado **%eax**) para lo cual se usa la siguiente instrucción:

```
mov VA, %eax
```

Donde **VA** es la dirección virtual donde se encuentra **h**. ¿Cual es el formato correspondiente a **VA** y **PA**?

|    |   |
|----|---|
| 0  | a |
| 1  | b |
| 2  | c |
| 3  | d |
| 4  | e |
| 5  | f |
| 6  | g |
| 7  | h |
| 8  | i |
| 9  | j |
| 10 | k |
| 11 | l |
| 12 | m |
| 13 | n |
| 14 | o |
| 15 | p |

logical memory

|   |   |
|---|---|
| 0 | 5 |
| 1 | 6 |
| 2 | 1 |
| 3 | 2 |

page table

|    |                  |
|----|------------------|
| 0  |                  |
| 4  | i<br>j<br>k<br>l |
| 8  | m<br>n<br>o<br>p |
| 12 |                  |
| 16 |                  |
| 20 | a<br>b<br>c<br>d |
| 24 | e<br>f<br>g<br>h |
| 28 |                  |

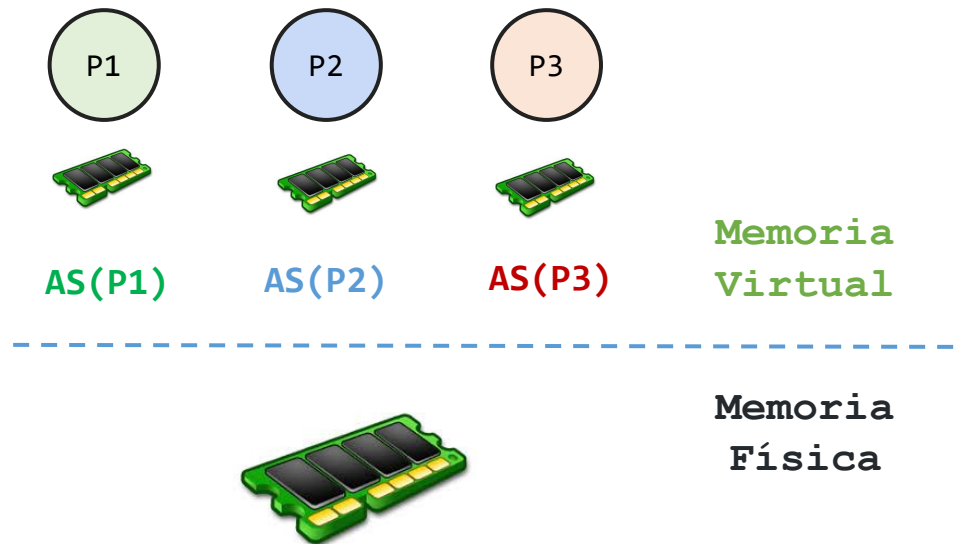
physical memory



# Procesos y espacio de direcciones

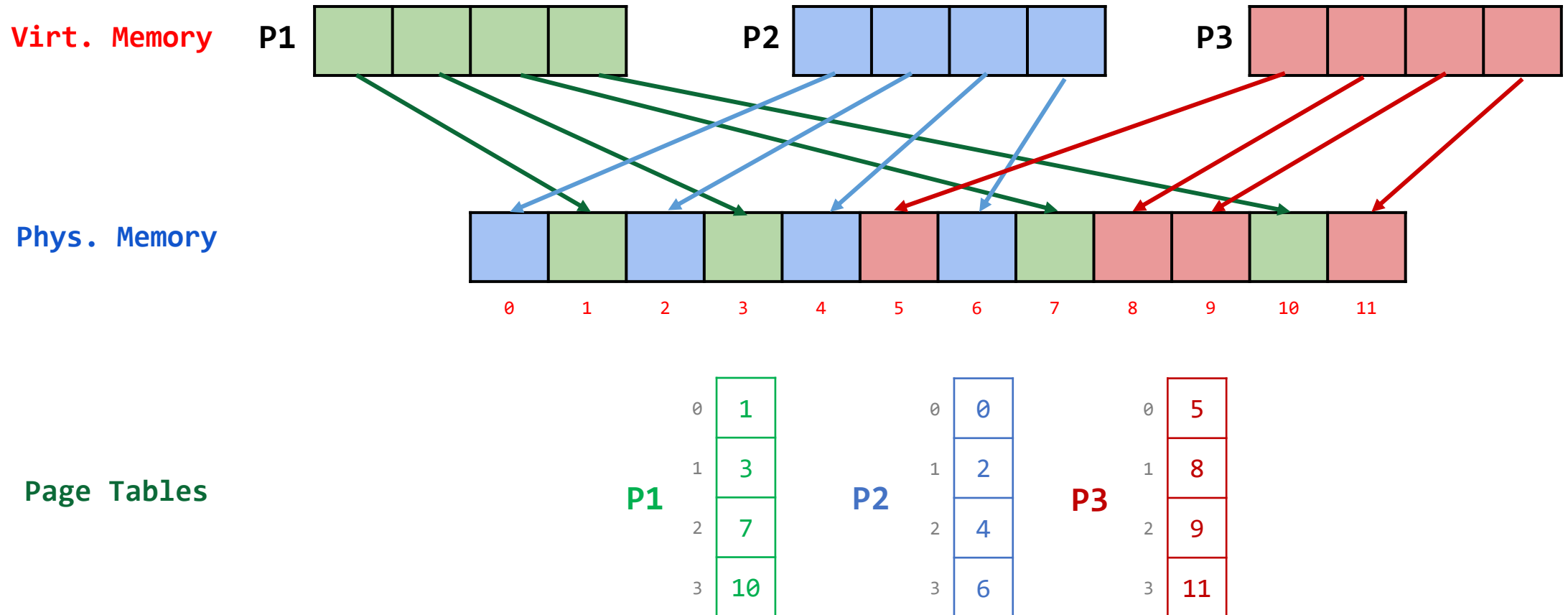
## ¿Que pasa cuando se ejecutan varios procesos?

- Se crea una tabla de página separada para cada proceso.
  - No hay manera de acceder a la memoria física de otro proceso.
  - En un context switch, el registro del MMU es apuntado a la dirección base de la tabla de página del proceso que se encuentra en ejecución.



# Procesos y espacio de direcciones

¿Que pasa cuando se ejecutan varios procesos?



# Procesos y espacio de direcciones

## ¿Que tan grandes pueden ser las páginas?

Las tablas de página **pueden ser muy grandes**. El siguiente ejemplo muestra esto

### Ejemplo:

Suponga que se tiene una memoria virtual (Address Space) con las siguientes características:

- 32 bits de direcciones.
- Páginas de 4KB
- Asuma que cada PTE tiene un tamaño de 4B

Responda las siguientes preguntas:

1. ¿Cual es el formato de las VA en este caso?
2. ¿Cual es el tamaño de la tabla de página (PT)?

# Procesos y espacio de direcciones

## Ejemplo:

Suponga que se tiene una memoria virtual (Address Space) con las siguientes características:

- 32 bits de direcciones.
- Páginas de 4KB
- Asuma que cada PTE tiene un tamaño de 4B

Responda las siguientes preguntas:

1. ¿Cual es el formato de las VA en este caso?
2. ¿Cual es el tamaño de la tabla de página (PT)?

# Procesos y espacio de direcciones

## Ejemplo - Pregunta 1

### Datos:

- $n(VA) = 32$ .
- $size(page) = 4KB$
- $size(PTE) = 4B$
- $n(offset) = ?$
- $n(VPN) = ?$



$$2^{n(offset)} = 4K$$

$$2^{n(offset)} = 4 * 2^{10}$$

$$2^{n(offset)} = (2^2) * (2^{10})$$

$$2^{n(offset)} = 2^{12}$$

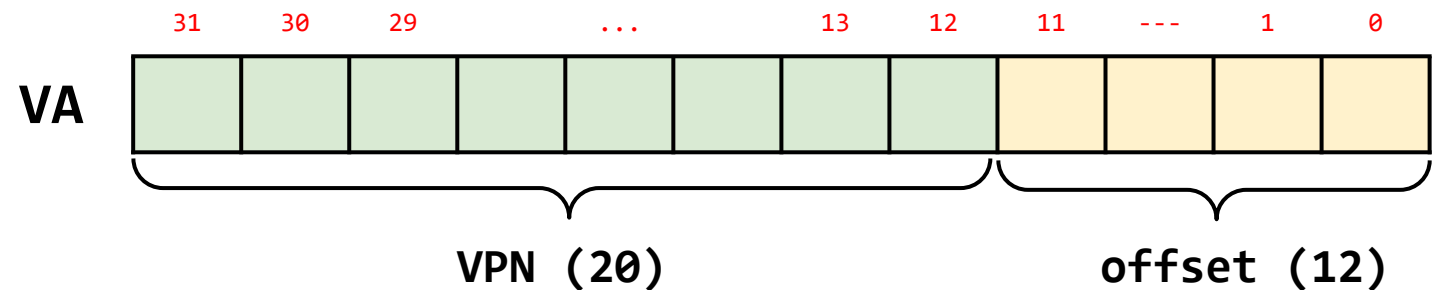
$$n(offset) = 12$$

$$n(VPN) + n(offset) = n(VA)$$

$$n(VPN) = n(VA) - n(offset)$$

$$n(VPN) = 32 - 12$$

$$n(VPN) = 20$$

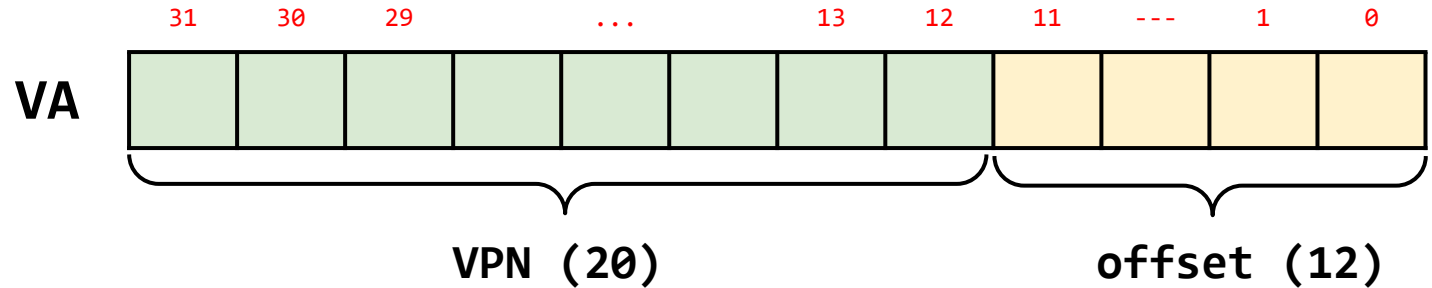


# Procesos y espacio de direcciones

## Ejemplo - Pregunta 1

### Datos:

- $n(\text{VA}) = 32$
- $\text{size}(\text{page}) = 4\text{KB}$
- $\text{size}(\text{PTE}) = 4\text{B}$
- $n(\text{offset}) = 12$
- $n(\text{VPN}) = 20$
- $\text{size}(\text{AS}) = ?$
- $\text{num\_pages} = ?$

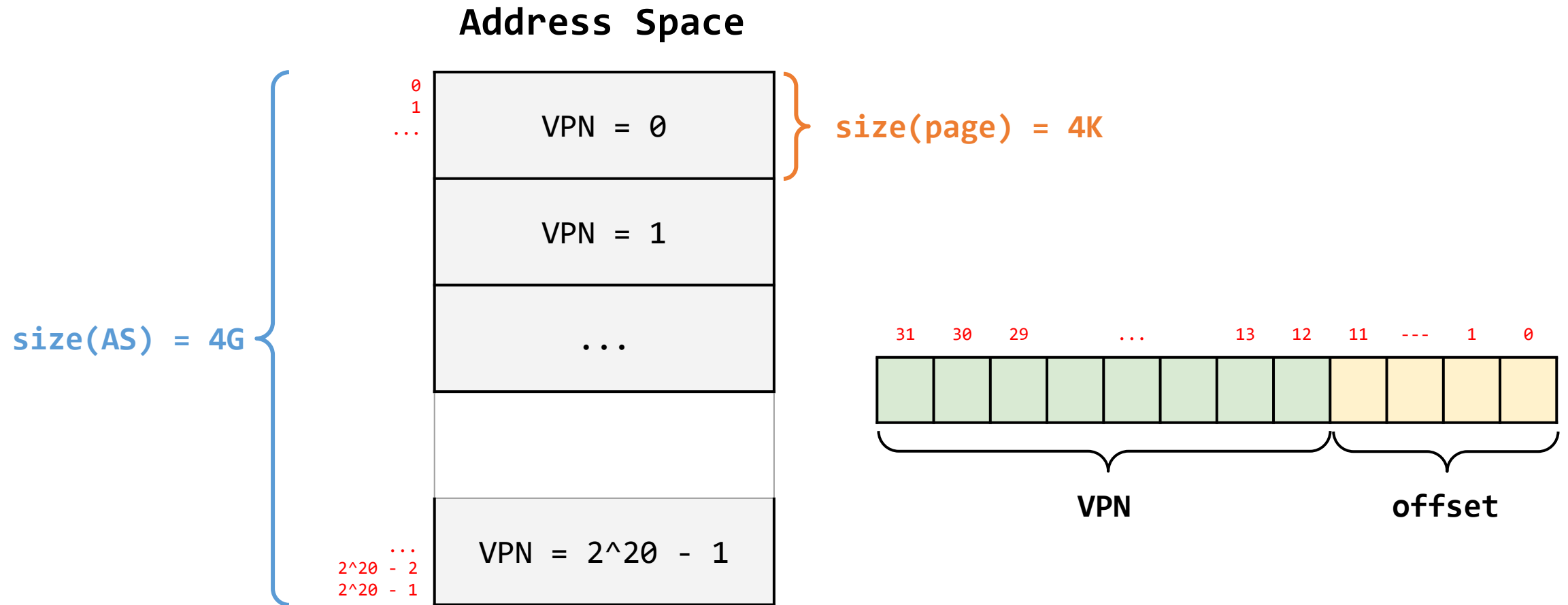


$$\begin{aligned}\text{size}(\text{AS}) &= 2^{n(\text{VA})} \\ \text{size}(\text{AS}) &= 2^{32} \\ \text{size}(\text{AS}) &= (2^2)(2^{30}) \\ \text{size}(\text{AS}) &= 4\text{G}\end{aligned}$$

$$\begin{aligned}\text{num\_pages} &= \text{size}(\text{AS}) / \text{size}(\text{page}) \\ \text{num\_pages} &= 4\text{G} / 4\text{K} \\ \text{num\_pages} &= 2^{30} / 2^{10} \\ \text{num\_pages} &= 2^{20} \\ \text{num\_pages} &= 1\text{M}\end{aligned}$$

# Procesos y espacio de direcciones

## Ejemplo - Pregunta 1



# Procesos y espacio de direcciones

## Ejemplo - Pregunta 2

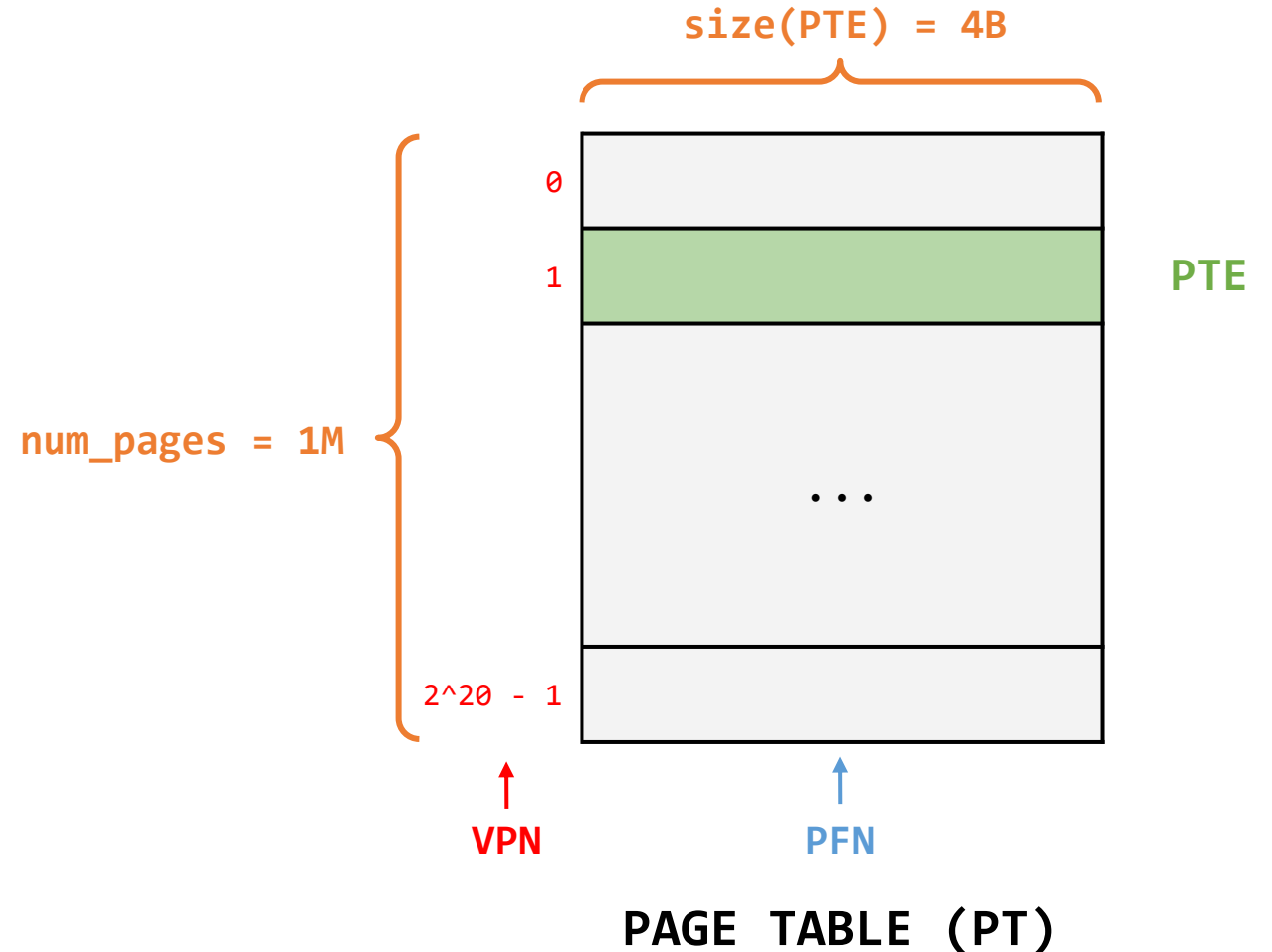
### Datos:

- $\text{num\_pages} = 1\text{M}$
- $\text{size(PTE)} = 4\text{B}$
- $\text{size(PT)} = ?$

$\text{size(PT)} = \text{size(PTE)} * \text{num\_pages}$   
 $\text{size(PT)} = 4 * 1\text{M}$   
 **$\text{size(PT)} = 4\text{M}$**

### Conclusión:

El tamaño de una tabla de página para un proceso es de **4MB** en este ejemplo.





# Procesos y espacio de direcciones

## ¿Que tan grandes pueden ser las páginas?

Si por cada proceso, se tiene un tabla de página de 4MB ¿Cual es el gasto de memoria de  $N = 100$  procesos?

$$N * \text{size}(\text{PT}) = 100 * 4\text{M} = 400\text{M} \quad (\text{Demasiado})$$

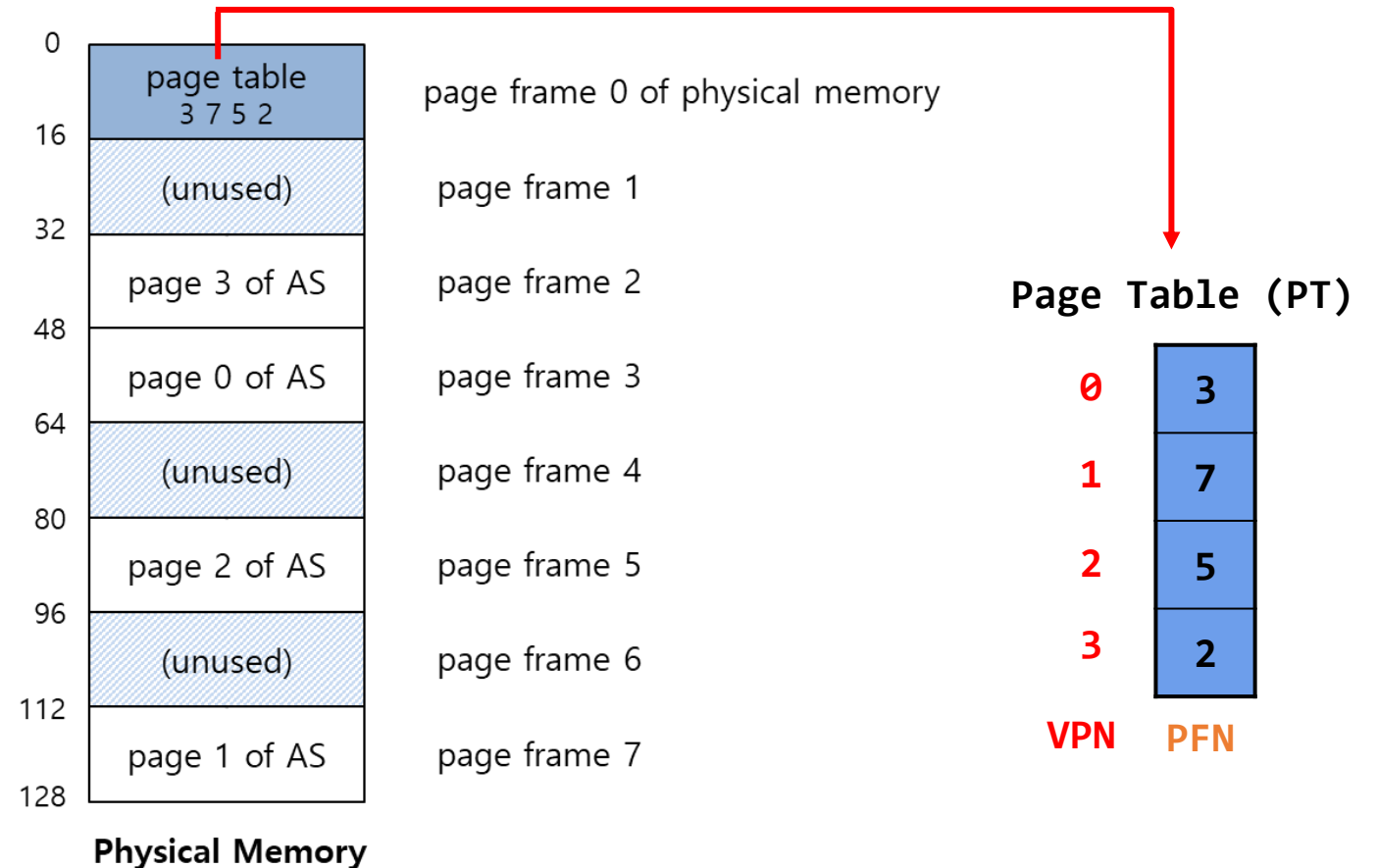
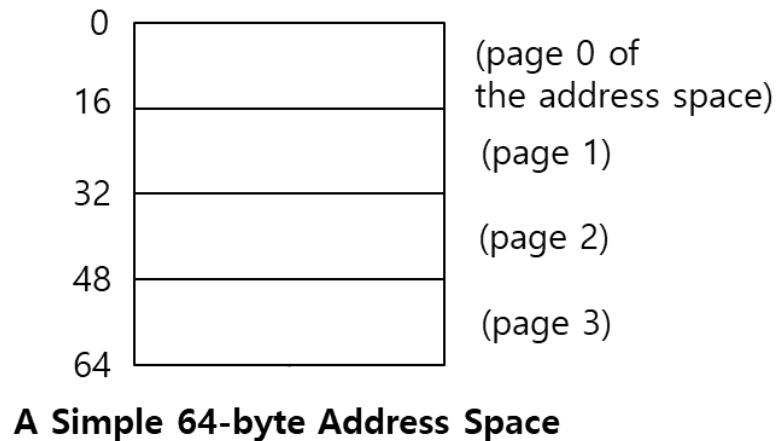
### Conclusión:

- Hay un enorme gasto de memoria.

# Procesos y espacio de direcciones

## ¿Donde se guardan las tablas de página?

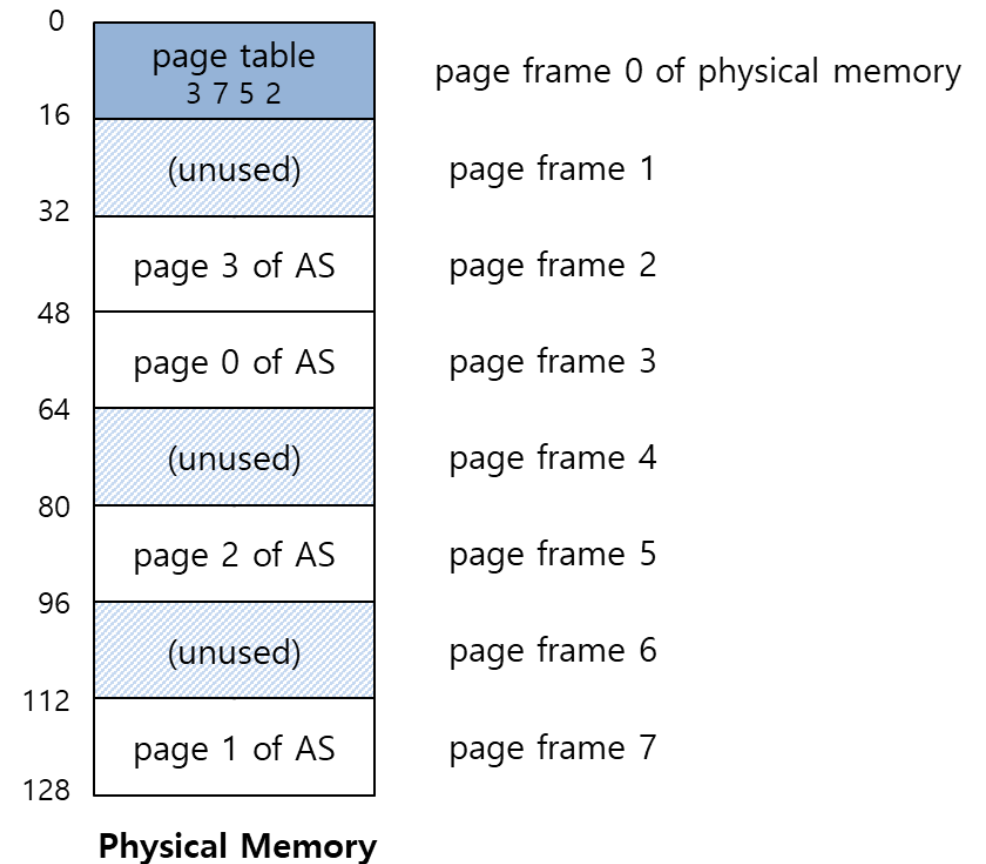
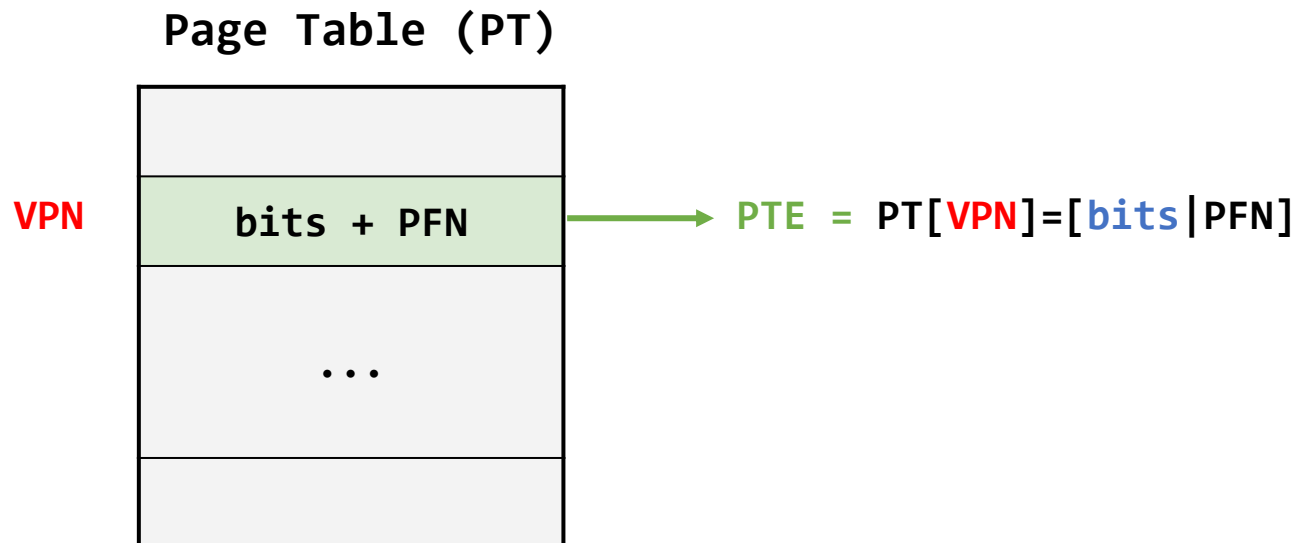
Las tablas de página son estructuras que se almacenan en memoria física en el **espacio kernel**.



# Estructura de una tabla de página

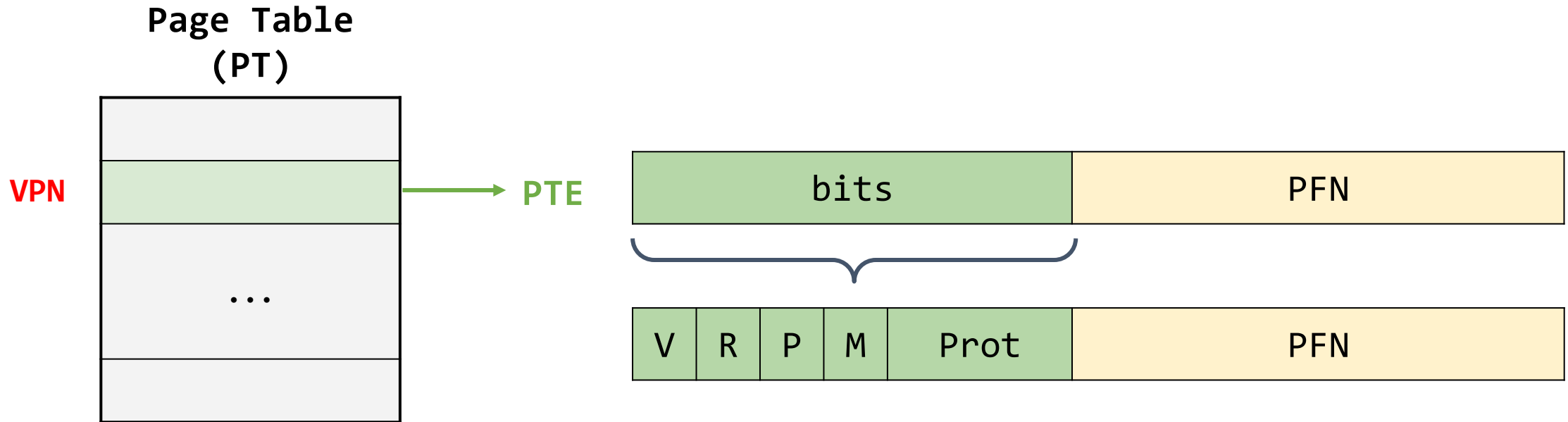
## Tabla de página (PT)

- **Estructura de datos** usada para mapear direcciones virtuales a direcciones físicas.
- **Forma más simple:**
  - Tabla de página lineal (Un arreglo).
  - El SO indexa el arreglo con el valor del **VPN** para obtener la **PTE**.



# Estructura de una tabla de página

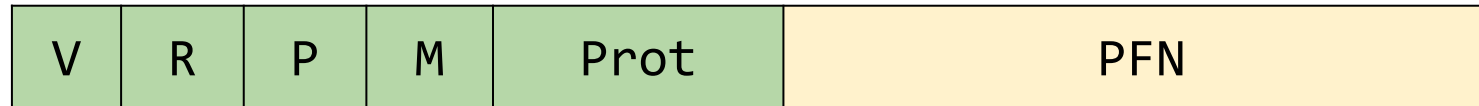
## Estructura de una PTE (Page Table Entry)



# Estructura de una tabla de página

## Estructura de una PTE (Page Table Entry)

- **Bit de validez (V):** Indica si la traducción es válida.
- **Bits de protección (Prot):** read - write - execute.
- **Bit de presencia (P):** Indica si la página está en la mem. física.
- **Bit sucio (M):** Indica si la página ha sido modificada.
- **Bit de referencia (R):** Indica si la página ha sido accedida.



Para comprender mejor cada uno de los **bits** mostrados descritos anteriormente, vamos a analizar el siguiente ejemplo tratando de explicar de manera gradual el rol de cada uno de los **bits** previamente mencionados.

# Estructura de una tabla de página

## Ejemplo

Suponga que se tienen los siguientes datos:

- Memoria virtual de 64 bytes dividida en páginas de 16 bytes.
- Memoria física de 128 bytes dividida en frames de 16 bytes.
- Las páginas de la memoria virtual contienen los siguientes segmentos del proceso se muestran en la **tabla 1**.
- En la **tabla 2** se muestra la tabla de página donde se especifica el mapeo entre la memoria virtual y la memoria física.

| page | segment |
|------|---------|
| VP0  | code    |
| VP1  | heap    |
| VP2  | free    |
| VP3  | stack   |

**Tabla 1.** Memoria virtual

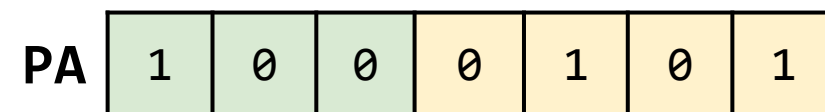
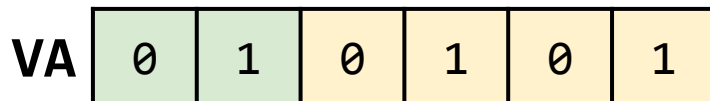
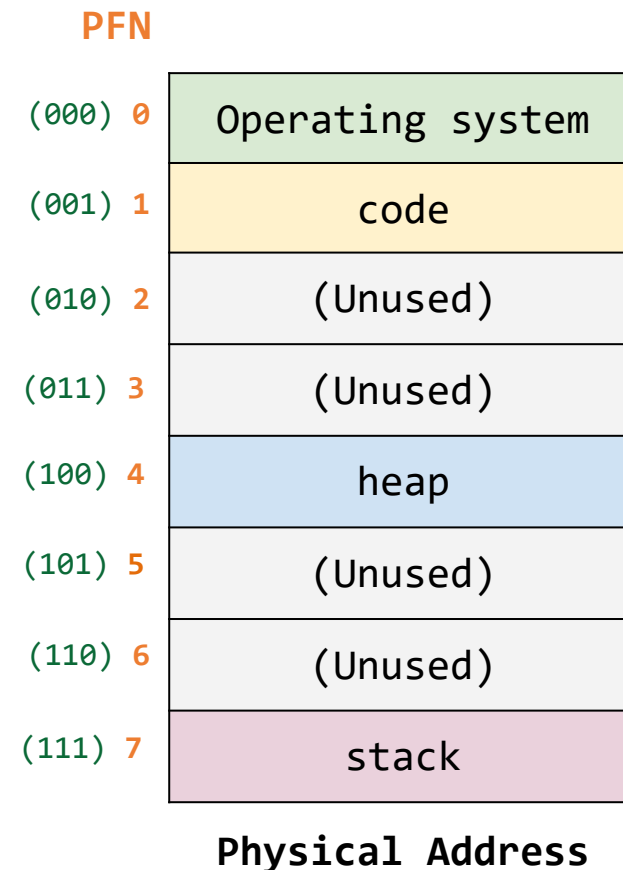
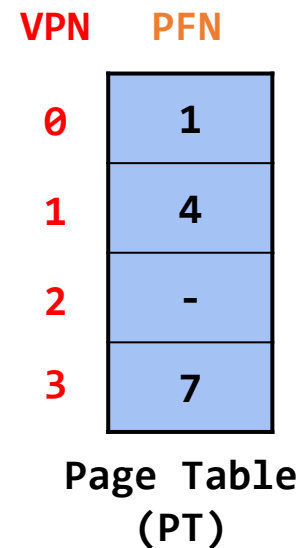
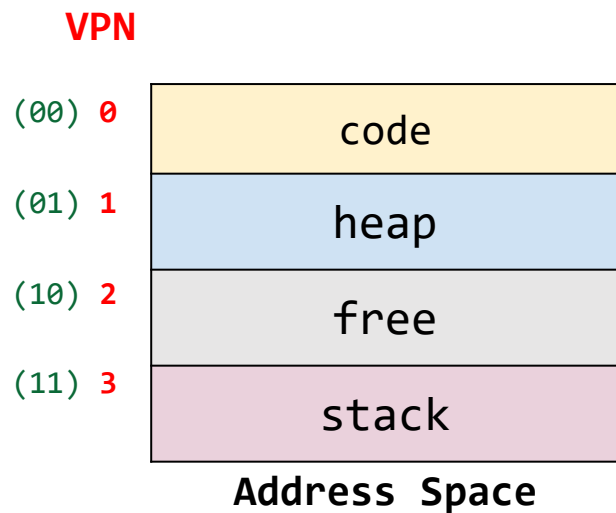
| page | frame     |
|------|-----------|
| VP0  | PFN1      |
| VP1  | PFN4      |
| VP2  | No valido |
| VP3  | PFN7      |

**Tabla 2.** Tabla de página

# Traducción de direcciones (II)

## Bosquejo de la memoria virtual, la memoria física y la tabla de página

| page | segment | frame     |
|------|---------|-----------|
| VP0  | code    | PFN1      |
| VP1  | heap    | PFN4      |
| VP2  | free    | No valido |
| VP3  | stack   | PFN7      |



# Traducción de direcciones (II)

## Valid (V)

Bit que indica si una **PTE** puede o no ser usada. Se evalúa cada vez que una dirección es usada.

| VPN    | Address Space |
|--------|---------------|
| (00) 0 | code          |
| (01) 1 | heap          |
| (10) 2 | free          |
| (11) 3 | stack         |

|   | V | PFN |
|---|---|-----|
| 0 | 1 | 1   |
| 1 | 1 | 4   |
| 2 | 0 | -   |
| 3 | 1 | 7   |

Page Table (PT)

| PFN     | Physical Address |
|---------|------------------|
| (000) 0 | Operating system |
| (001) 1 | code             |
| (010) 2 | (Unused)         |
| (011) 3 | (Unused)         |
| (100) 4 | heap             |
| (101) 5 | (Unused)         |
| (110) 6 | (Unused)         |
| (111) 7 | stack            |



# Traducción de direcciones (II)

## Reference (R)

- Bit que indica si la página ha sido accedida. Es llevado a **1 (operacion set)** cuando la página ha sido leída o escrita.
- Utilidad:
  - Permite rastrear el acceso a la página.
  - Determina la popularidad de la página.

## Ejemplo (continuación):

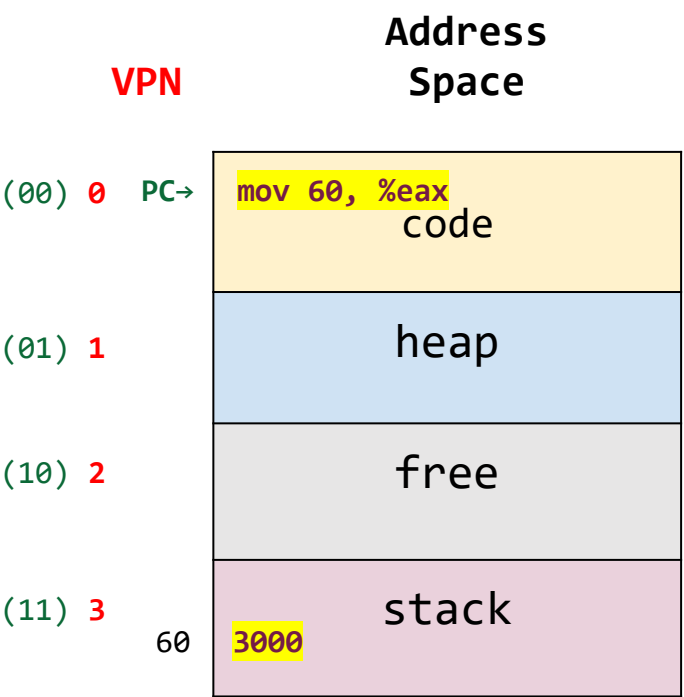
Suponga que se ejecuta una instrucción de lectura en la cual se accede a la dirección 60:

```
mov 60, %eax
```

¿Cual seria el estado del bit **R** para cada PTE?

# Traducción de direcciones (II)

## Reference (R)



|   | V | R | PFN |
|---|---|---|-----|
| 0 | 1 | 1 | 1   |
| 1 | 1 | 0 | 4   |
| 2 | 0 | - | -   |
| 3 | 1 | 1 | 7   |

Page Table (PT)

| PFN         | Physical Address     |
|-------------|----------------------|
| (000) 0     | Operating system     |
| (001) 1     | mov 60, %eax<br>code |
| (010) 2     | (Unused)             |
| (011) 3     | (Unused)             |
| (100) 4     | heap                 |
| (101) 5     | (Unused)             |
| (110) 6     | (Unused)             |
| (111) 7 124 | 3000<br>stack        |

# Traducción de direcciones (II)

## **Modify (M) ó Dirty (D)**

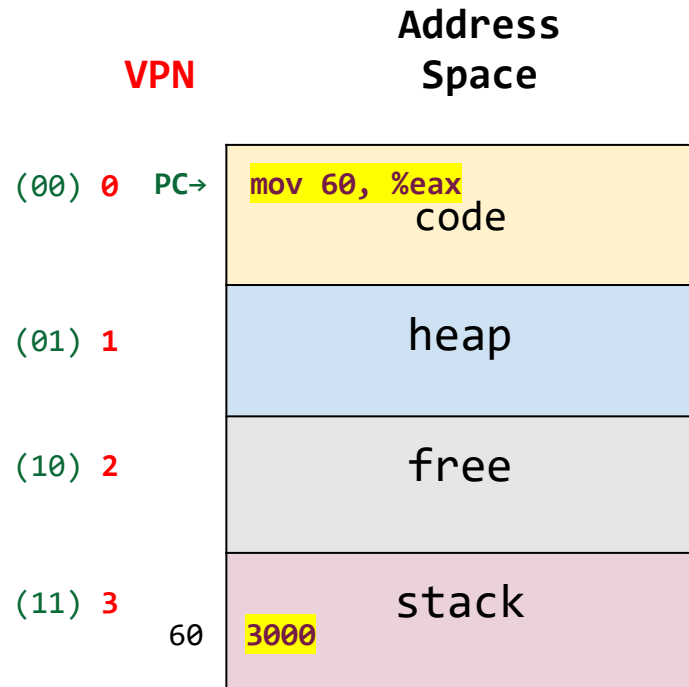
- Indica si la página ha sido modificada, es decir, que sobre esta se ha hecho una escritura (**write**).

## **Ejemplo (continuación):**

Suponga que una variable global es modificada. ¿Como quedaría el **bit M** para cada una de las **PTEs**?

# Traducción de direcciones (II)

Modify (M) ó Dirty (D)



|   | V | R | M | PFN |
|---|---|---|---|-----|
| 0 | 1 | 1 | 1 | 1   |
| 1 | 1 | 0 | 0 | 4   |
| 2 | 0 | - | - | -   |
| 3 | 1 | 1 | 0 | 7   |

Page Table  
(PT)

|         | PFN | Physical Address     |
|---------|-----|----------------------|
| (000) 0 |     | Operating system     |
| (001) 1 |     | mov 60, %eax<br>code |
| (010) 2 |     | (Unused)             |
| (011) 3 |     | (Unused)             |
| (100) 4 |     | heap                 |
| (101) 5 |     | (Unused)             |
| (110) 6 |     | (Unused)             |
| (111) 7 | 124 | 3000<br>stack        |

**Nota:** En este ejemplo, **VPN=0** agrupa código y variables globales. En un sistema real, las variables globales residen en un segmento de datos separado (**R/W**) y el código tiene permisos **R/X** con **M=0**.

# Traducción de direcciones (II)

## Protection (Prot)

Bit que controla cuáles operaciones son permitidas (**R/W/X, User/Kernel, etc.**)

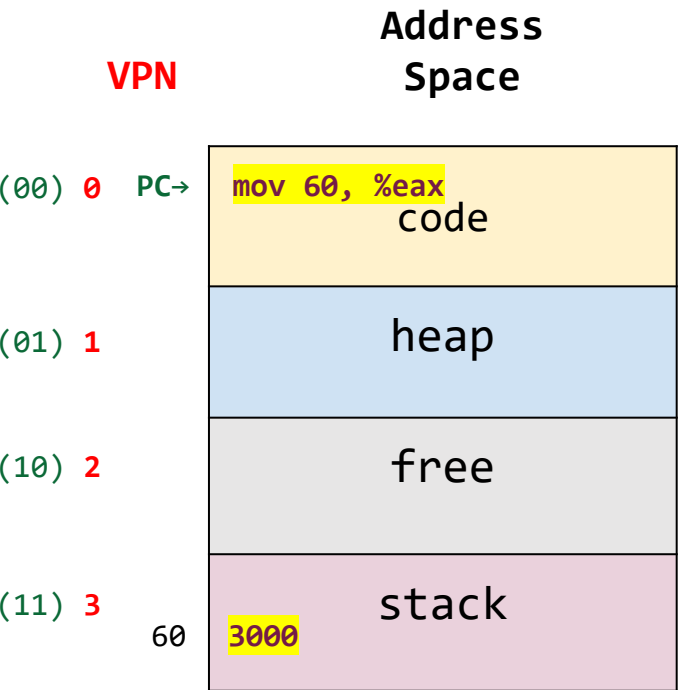
## Ejemplo (continuación):

Agregue a cada PTE los bits asociados a la protección, asumiendo por ahora que:

- **R/X = 00**
- **R/W = 01**

# Traducción de direcciones (II)

## Protection (Prot)



|   | V | R | M | Prot | PFN |
|---|---|---|---|------|-----|
| 0 | 1 | 1 | 1 | 00   | 1   |
| 1 | 1 | 0 | 0 | 01   | 4   |
| 2 | 0 | - | - | -    | -   |
| 3 | 1 | 1 | 0 | 01   | 7   |

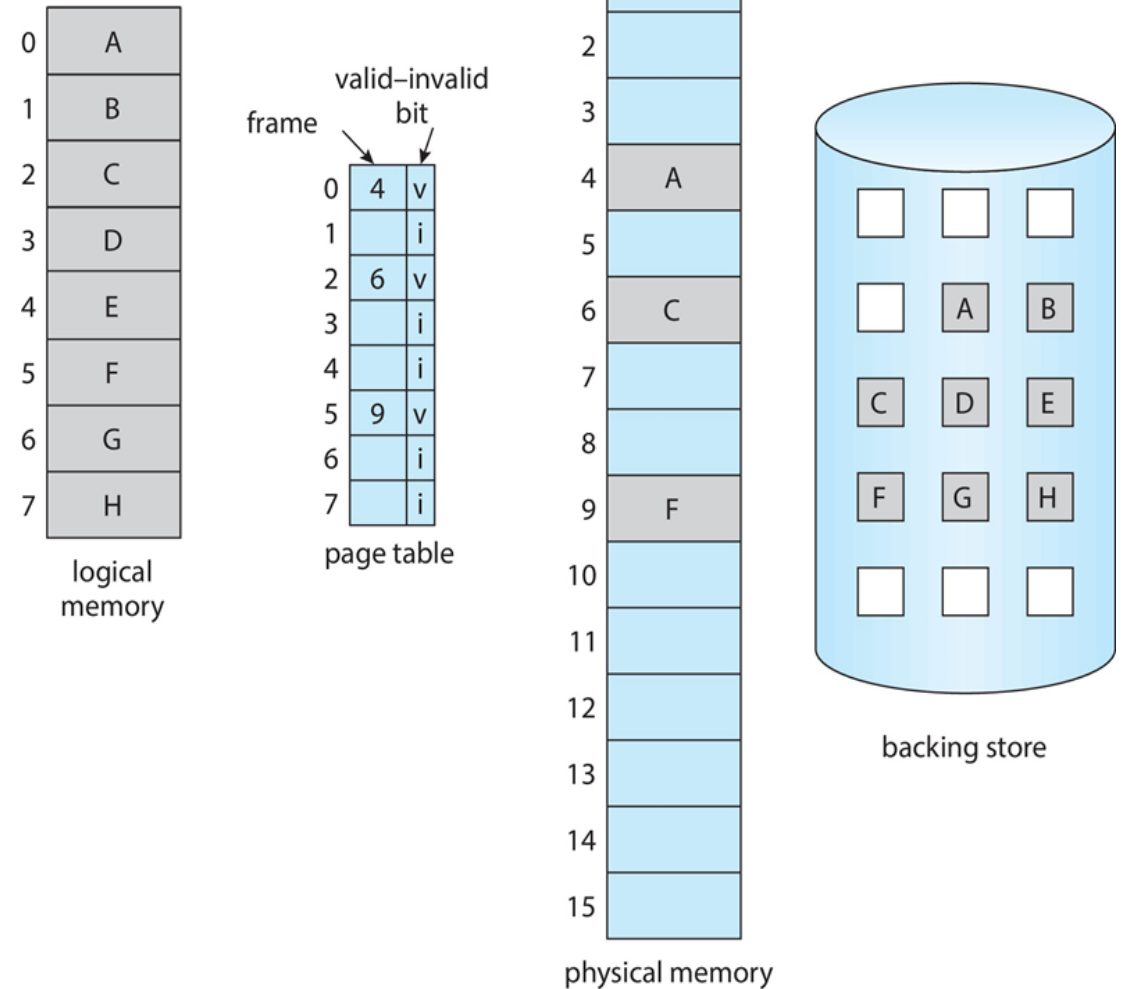
Page Table  
(PT)

|         | PFN | Physical Address                  |
|---------|-----|-----------------------------------|
| (000) 0 |     | Operating system                  |
| (001) 1 |     | <code>mov 60, %eax</code><br>code |
| (010) 2 |     | (Unused)                          |
| (011) 3 |     | (Unused)                          |
| (100) 4 |     | heap                              |
| (101) 5 |     | (Unused)                          |
| (110) 6 |     | (Unused)                          |
| (111) 7 | 124 | <code>3000</code> stack           |

# Traducción de direcciones (II)

## Present (P)

Indica si la página se encuentra en memoria física o en el disco.

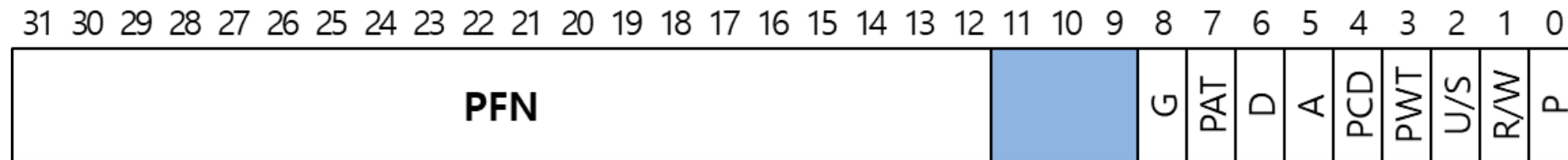


**Nota aclaratoria:** Este diagrama es tomado del libro de Silberschatz que usa **v/i** para **valido/invalido**; en este **v** equivale a **V=1** e **i** a **V=0**.

# Estructura de una tabla de página

## Ejemplo: PTE en x86

La estructura presentada corresponde al formato real de una entrada de tabla de páginas (PTE) implementada en arquitecturas x86.



An x86 Page Table Entry(PTE)

- **P**: present
- **R/W**: read/write bit
- **U/S**: supervisor
- **A**: accessed bit
- **D**: dirty bit
- **PFN**: the page frame number

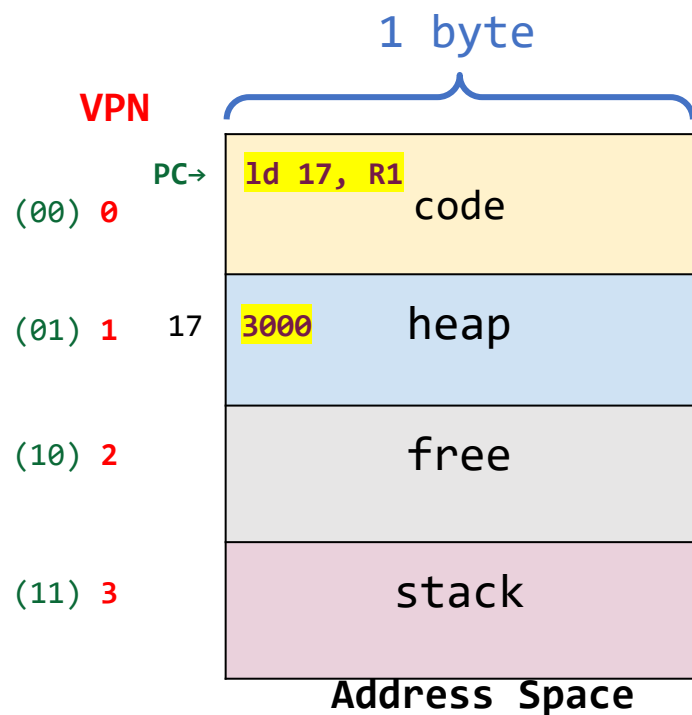
Entender la **PTE** permite implementar gestión de memoria (traducción, permisos, fallos de página) y adaptar el SO a distintas arquitecturas.



# Estructura de una tabla de página

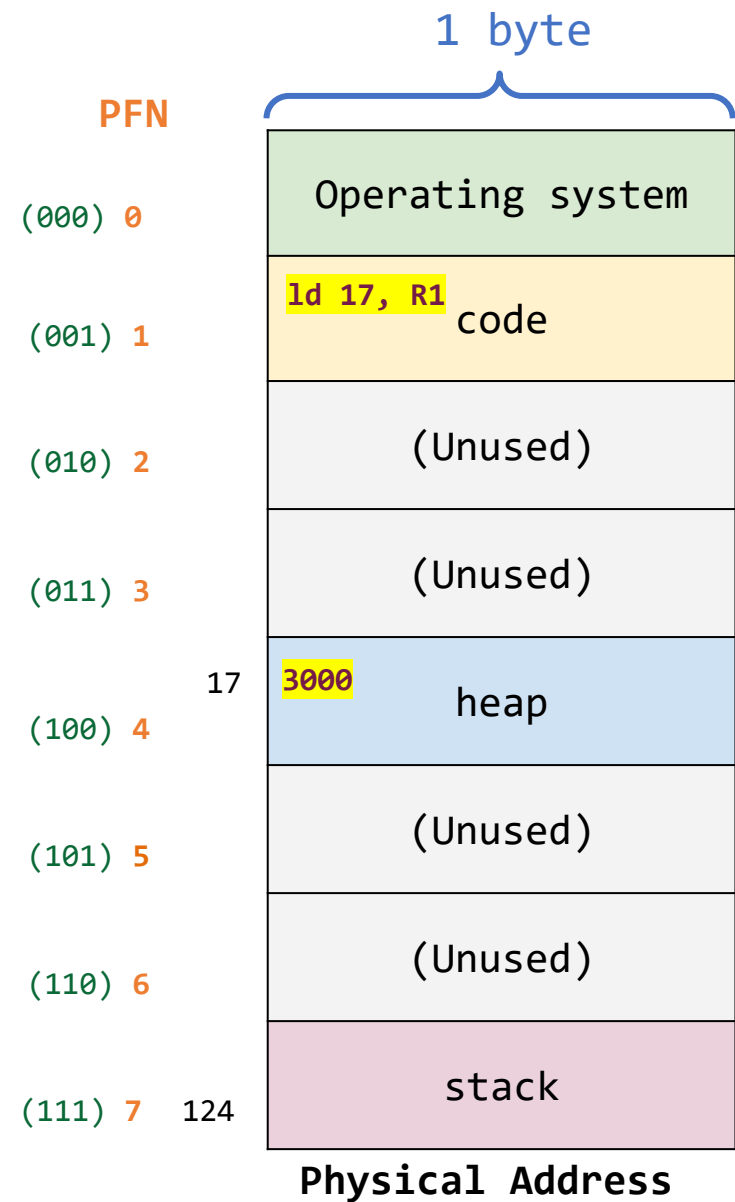
## Ejemplo

Suponga que se tiene el siguiente esquema:



|   | V | PFN |
|---|---|-----|
| 0 | 1 | 1   |
| 1 | 1 | 4   |
| 2 | 0 | -   |
| 3 | 1 | 7   |

Page Table (PT)



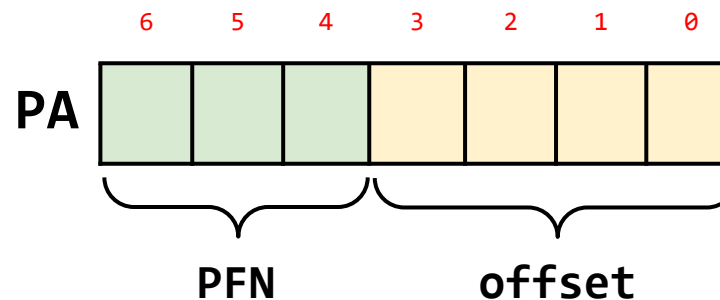
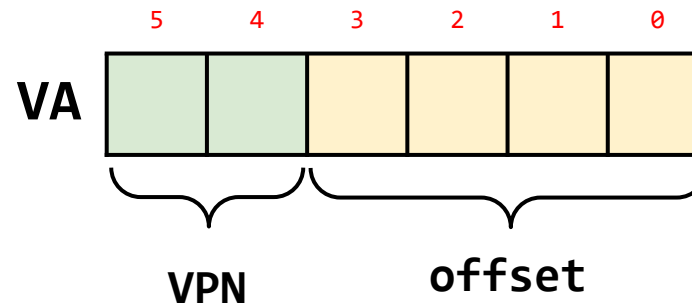
# Estructura de una tabla de página

## Ejemplo

Información del esquema:

| Memoria Virtual |      |
|-----------------|------|
| size(AS)        | 64 B |
| n(VA)           | 6    |
| n(VPN)          | 2    |
| n(offset)       | 4    |

| Memoria física |       |
|----------------|-------|
| size(PM)       | 128 B |
| n(PM)          | 7     |
| n(PFN)         | 3     |
| n(offset)      | 4     |



|   | V | PFN |
|---|---|-----|
| 0 | 1 | 1   |
| 1 | 1 | 4   |
| 2 | 0 | -   |
| 3 | 1 | 7   |

Page Table (PT)

# Estructura de una tabla de página

## Ejemplo

Suponga que se va a analizar la instrucción **ld 17, R1** y responda las siguientes preguntas:

1. Obtenga los valores de los campos de la dirección virtual.
2. Obtenga los valores de los campos de la dirección física.
3. Teniendo en cuenta la tabla de página mostrada.
  - ¿Cual es el tamaño de una de sus entradas?
  - ¿Cual es el tamaño de la tabla de página entera?

# Estructura de una tabla de página

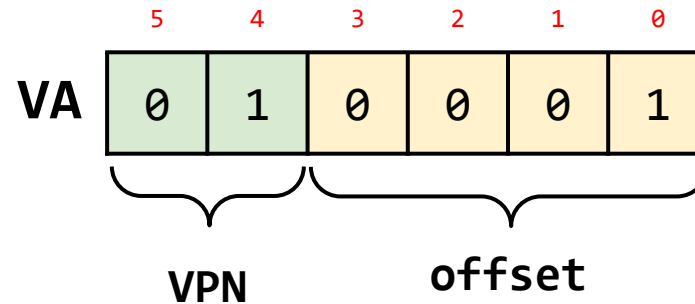
## Ejemplo

Suponga que se va a analizar la instrucción **ld 17, R1** y responda las siguientes preguntas:

1. Obtenga los valores de los campos de la dirección virtual.

$$VA = 17 = 10001 = 0x11 = 010001 = [01|0001]$$

- VPN = 01 = 1
- offset = 0001 = 1



# Estructura de una tabla de página

## Ejemplo

Suponga que se va a analizar la instrucción **ld 17, R1** y responda las siguientes preguntas:

2. Obtenga los valores de los campos de la dirección física.

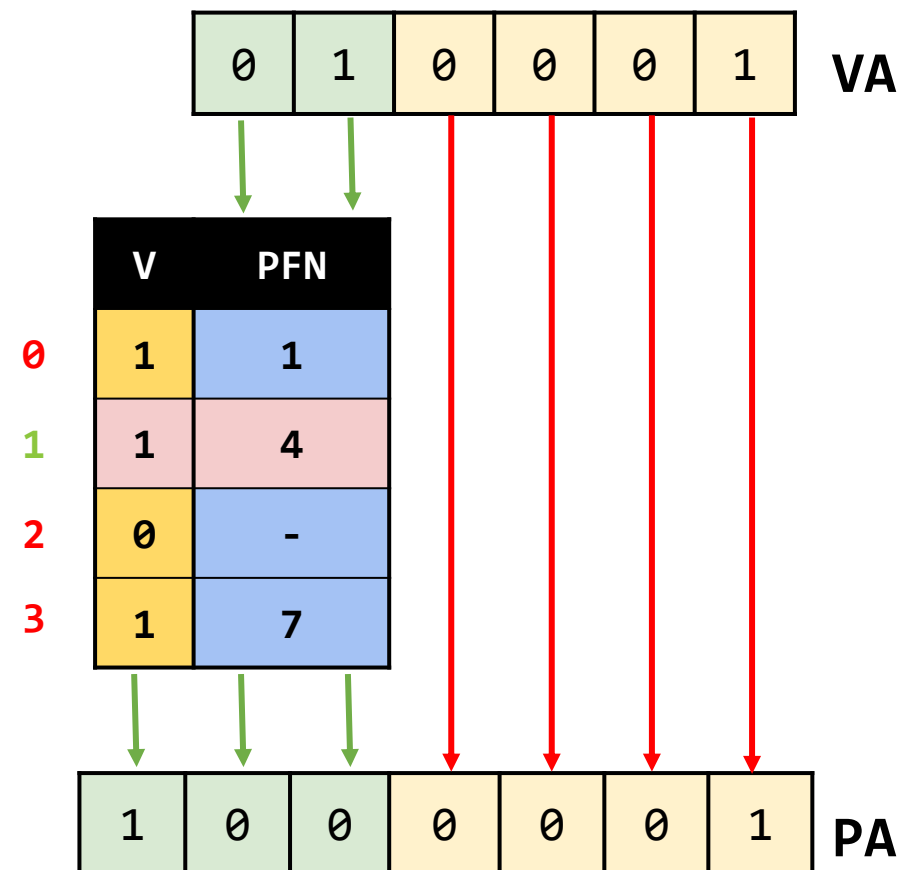
Cuando:  $VA = 17 \rightarrow VPN = 1$

$PFN = PT[VPN = 1] = 4 = 100$

$PA = [PFN | offset] = [100 | 0001]$

- $PFN = 100 = 4$
- $offset = 0001 = 1$

$PA = 1000001 = 65 = 0x41$



# Estructura de una tabla de página

## Ejemplo

Suponga que se va a analizar la instrucción **ld 17, R1** y responda las siguientes preguntas:

3. Teniendo en cuenta la tabla de página mostrada.
- ¿Cual es el tamaño de una de sus entradas?
  - ¿Cual es el tamaño de la tabla de página entera?

Page Table (PT)

|   | V | PFN |
|---|---|-----|
| 0 | 1 | 1   |
| 1 | 1 | 4   |
| 2 | 0 | -   |
| 3 | 1 | 7   |

4 bits

$$\text{size(PTE)} = \text{size(bits)} + \text{size(PFN)}$$

$$\text{size(PTE)} = \text{size(V)} + \text{size(PFN)}$$

$$\text{size(PTE)} = 1 + 3$$

$$\text{size(PTE)} = 4 \text{ bits}$$

$$\text{size(PT)} = \text{num\_pages} * \text{size(PTE)}$$

$$\text{size(PT)} = 4 * (4 \text{ bits})$$

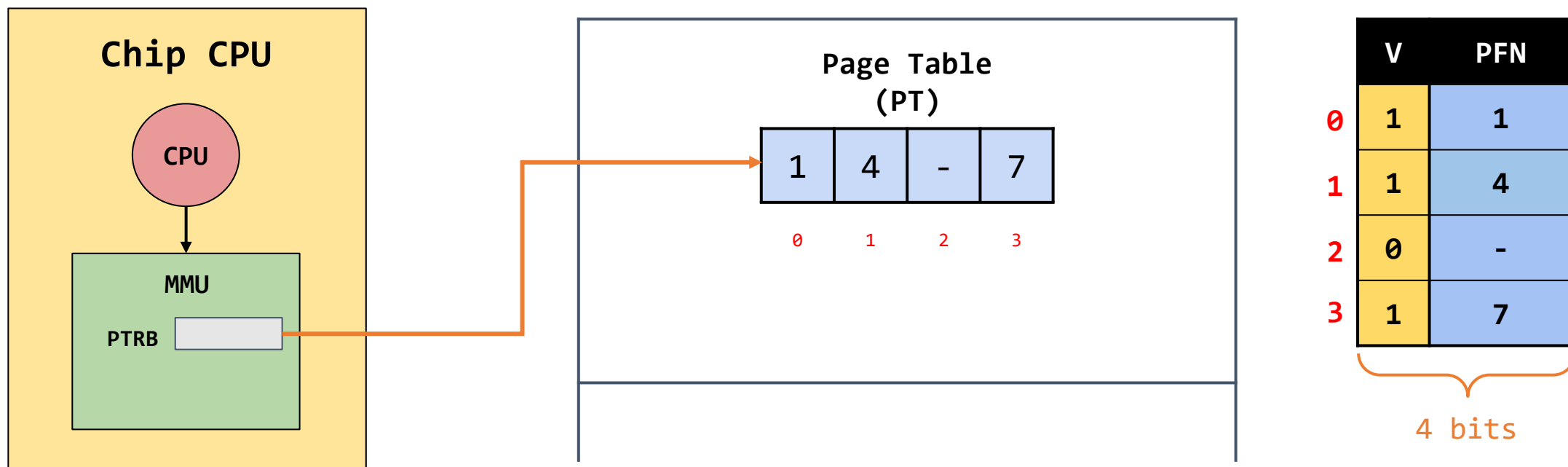
$$\text{size(PT)} = 16 \text{ bits}$$

# Estructura de una tabla de página

## Registro PTBR

**Problema:** El hardware debe hacer la traslación pero ¿Dónde está la tabla asociada al proceso?

- Las **tablas de página** residen en memoria física y son gestionadas por el kernel
- Para conocer la ubicación de estas se emplea el **PTBR (Page Table Base Register)**.
- El registro **PTBR** mantiene la dirección física de la tabla de página asociada al proceso.
- La **MMU** usa el **PTBR** para localizar la tabla de páginas durante la traducción de direcciones.

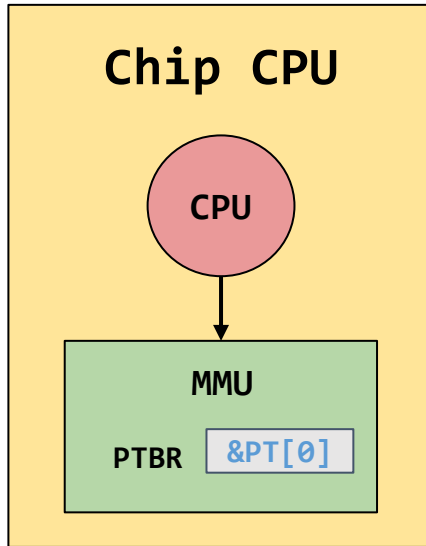


# Estructura de una tabla de página

## Registro PTBR

Suponiendo que **PTBR** = 4, entonces tenemos:

El **PTBR** apunta a la base de la tabla de páginas en memoria física. La MMU usa este valor como referencia para ubicar la entrada correspondiente durante la traducción de direcciones.



**PTBR**

4

$PTBR = \&PT[0] = 4$

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 |   |   |   |   |   |   |   |   |
| 1 |   |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |   |
| 4 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 5 | 0 | ? | ? | ? | 1 | 1 | 1 | 1 |
| 6 |   |   |   |   |   |   |   |   |

OS

Page Table (PT)

PFN0

Page Table  
(PT)

|   | V | PFN |
|---|---|-----|
| 0 | 1 | 001 |
| 1 | 1 | 100 |
| 2 | 0 | -   |
| 3 | 1 | 111 |

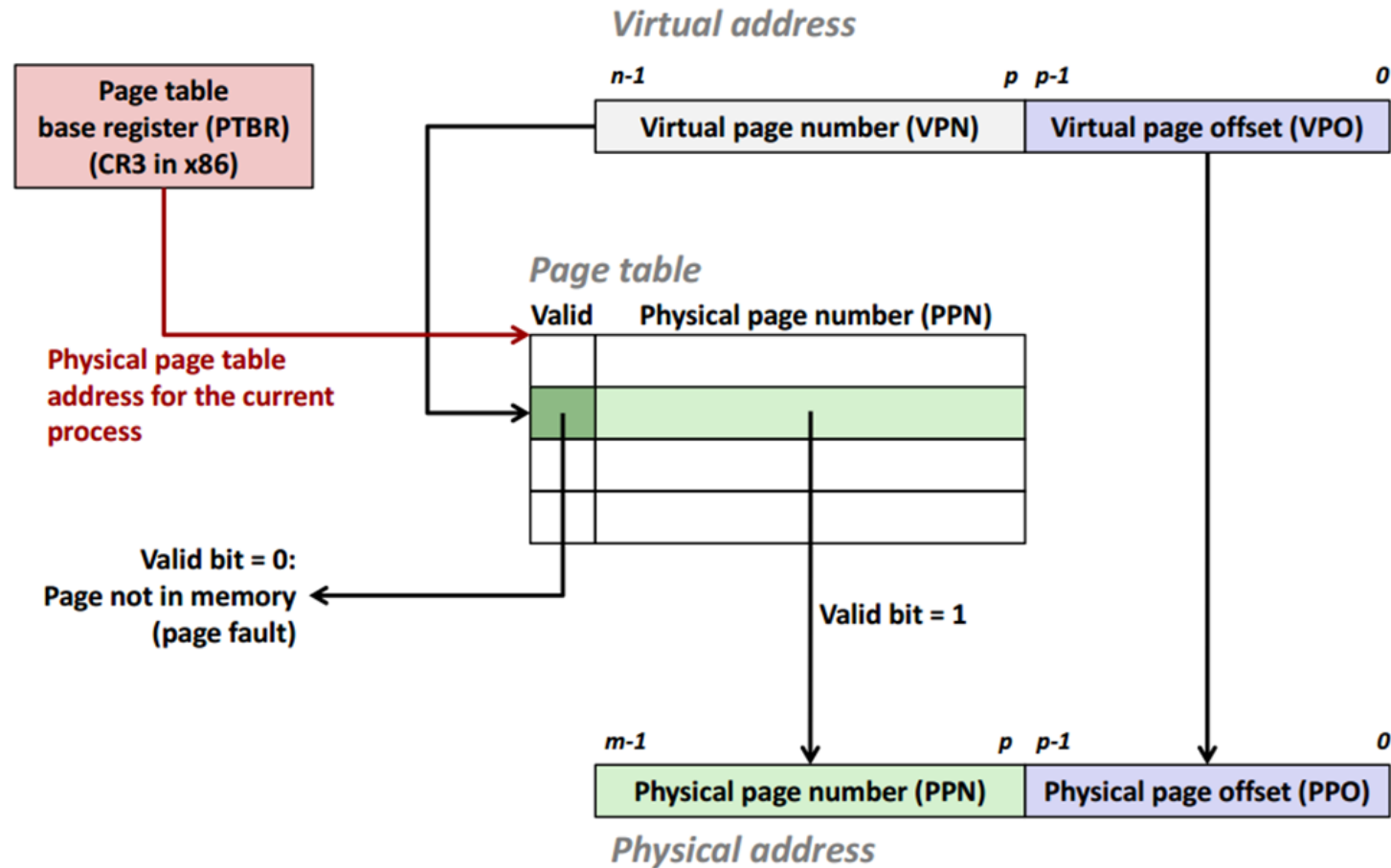
4 bits



# Traducción de direcciones usando paginación

## Hardware asociado

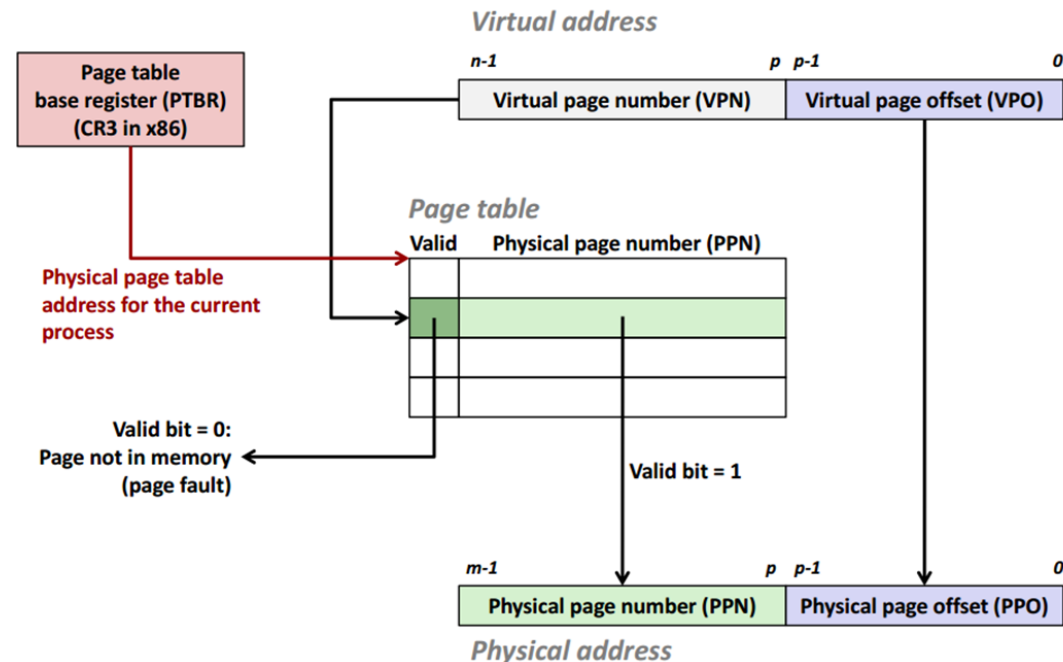
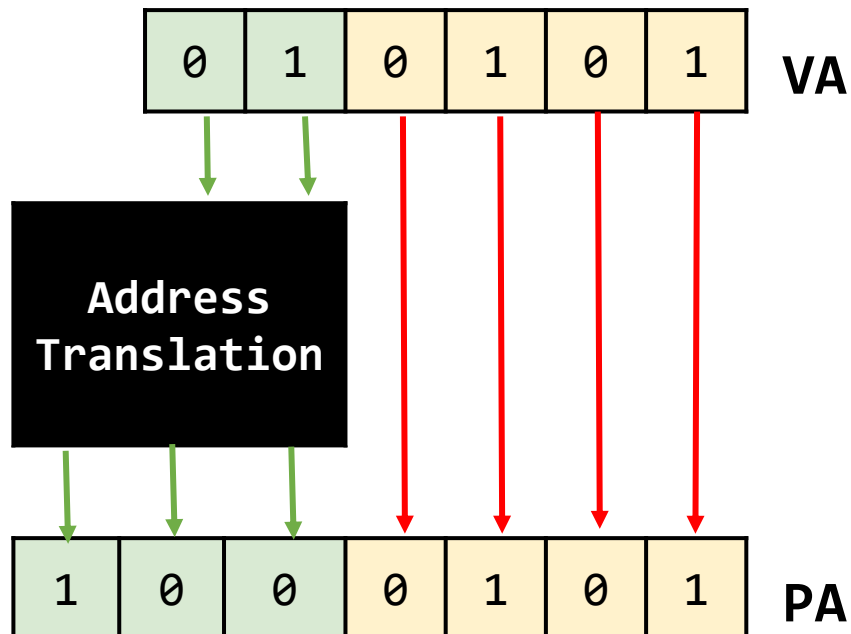
El hardware debe conocer la estructura exacta de la tabla de página (PT)



# Traducción de direcciones usando paginación

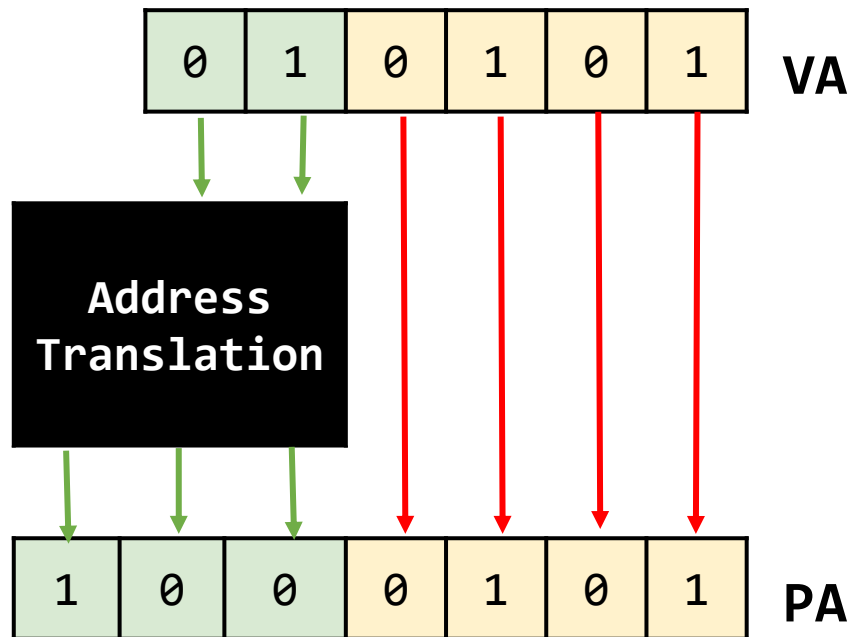
## Lógica implementada resumida

```
index into PT: PTBR + VPN*size(PTE)
Fetch PTE for VPN;           // Traer la PTE
if (VALID == 0)
    fault();
PA = (PFN << PAGESHIFT) | offset // Realizar la traducción
Fetch PA → R1;               // Cargar los datos desde la memoria física
```



# Traducción de direcciones usando paginación

## Ejemplo tomado del libro



Supongamos que se ejecuta la instrucción

```
movl 21, %eax
```

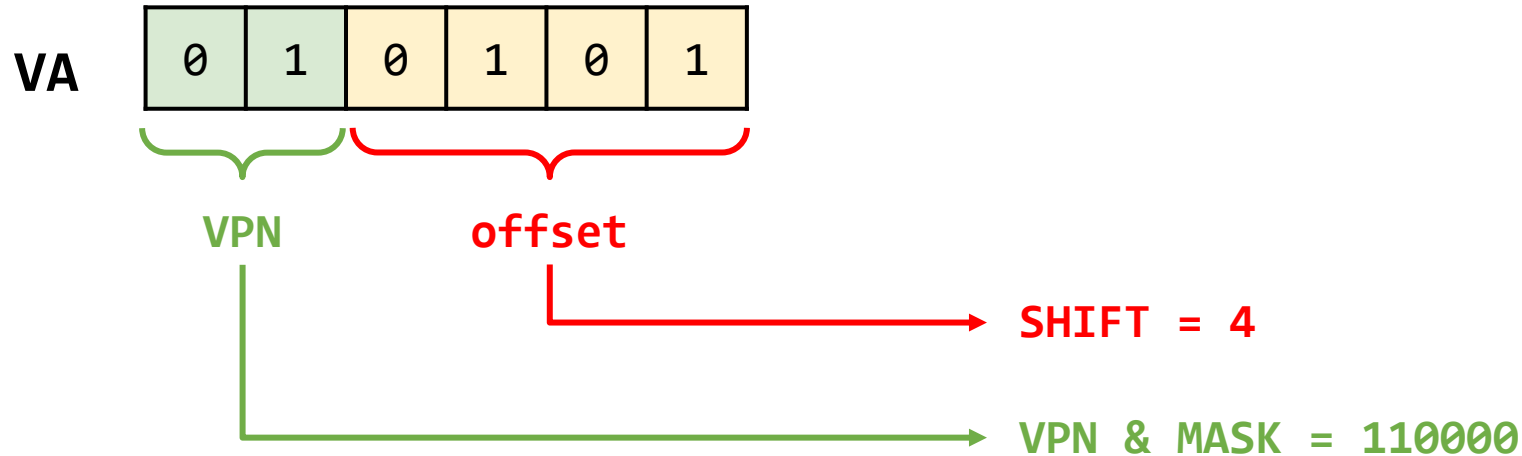
¿Cómo es que el hardware lleva a cabo la traducción de la VA (21) a la PA (117)?

1. Traer la PTE solicitada de la PT asociada al proceso.
2. Realizar la traducción.
3. Cargar los datos desde memoria física.

# Traducción de direcciones usando paginación

## Ejemplo tomado del libro (Análisis detallado)

1. Encontrar la localización del PTE deseado.

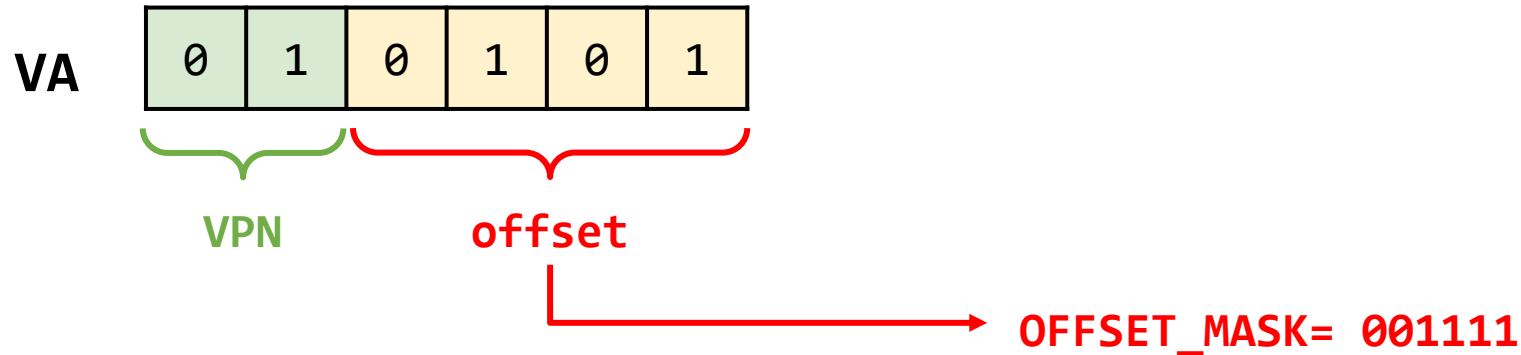


```
VPN = (VirtualAddress & VPN_MASK) >> SHIFT // VPN = 01
PTEAddr = PageTableBaseRegister + (VPN * sizeof(PTE)) // PTEAddr = &PT[1]
```

# Traducción de direcciones usando paginación

## Ejemplo tomado del libro (Análisis detallado)

2. Realizar la traducción.



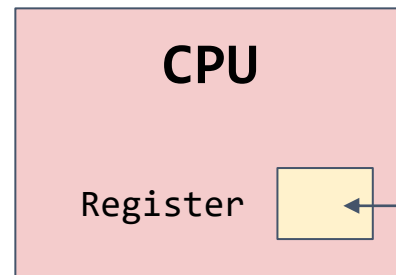
```
offset = VirtualAddress & OFFSET_MASK    // offset = 01001 & 001111 = 000101
PhysAddr = (PFN << SHIFT) | offset        // PhysAddr = (111 << 4) | 000101 = 1110101
```

# Traducción de direcciones usando paginación

## Ejemplo tomado del libro (Análisis detallado)

### 3. Cargar los datos desde la memoria física

```
Register = AccessMemory(PhysAddr)
```



data

| PFN     |                  |
|---------|------------------|
| (000) 0 | Operating system |
| (001) 1 | (Unused)         |
| (010) 2 | VP3              |
| (011) 3 | VP0              |
| (100) 4 | (Unused)         |
| (101) 5 | VP2              |
| (110) 6 | (Unused)         |
| (111) 7 | VP1              |

Physical Address

PA = 117 = [111|0101]

# Traducción de direcciones usando paginación

## Resumen protocolo completo

```
1  // Extract the VPN from the virtual address
2  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4  // Form the address of the page-table entry (PTE)
5  PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7  // Fetch the PTE
8  PTE = AccessMemory(PTEAddr)
9
10 // Check if process can access the page
11 if (PTE.Valid == False)
12     RaiseException(SEGMENTATION_FAULT)
13 else if (CanAccess(PTE.ProtectBits) == False)
14     RaiseException(PROTECTION_FAULT)
15 else
16     // Access is OK: form physical address and fetch it
17     offset = VirtualAddress & OFFSET_MASK
18     PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
19     Register = AccessMemory(PhysAddr)
```

# Posibles problemas

## Acceso simple a memoria

### Código

```
int array[1000];  
...  
for (i = 0; i < 1000; i++)  
    array[i] = 0;
```

### Compilación y ejecución

```
prompt> gcc -o array array.c -Wall  
prompt> ./array
```

### Código ensamblador resultante

|        |                           |                                               |
|--------|---------------------------|-----------------------------------------------|
| 0x1024 | movl \$0x0, (%edi,%eax,4) | # %edi = &array; %eax = i → array[i] = 0      |
| 0x1028 | incl %eax                 | # %eax = %eax + i → i = i + 1                 |
| 0x102c | cmpl \$0x03e8,%eax        | # 0x03e8 = 1000 → i - 1000 < 0 → sf = 1       |
| 0x1030 | jne 0x1024                | # Si no se llega a i = 1000 vaya al principio |



# Posibles problemas

## Acceso simple a memoria

### Código

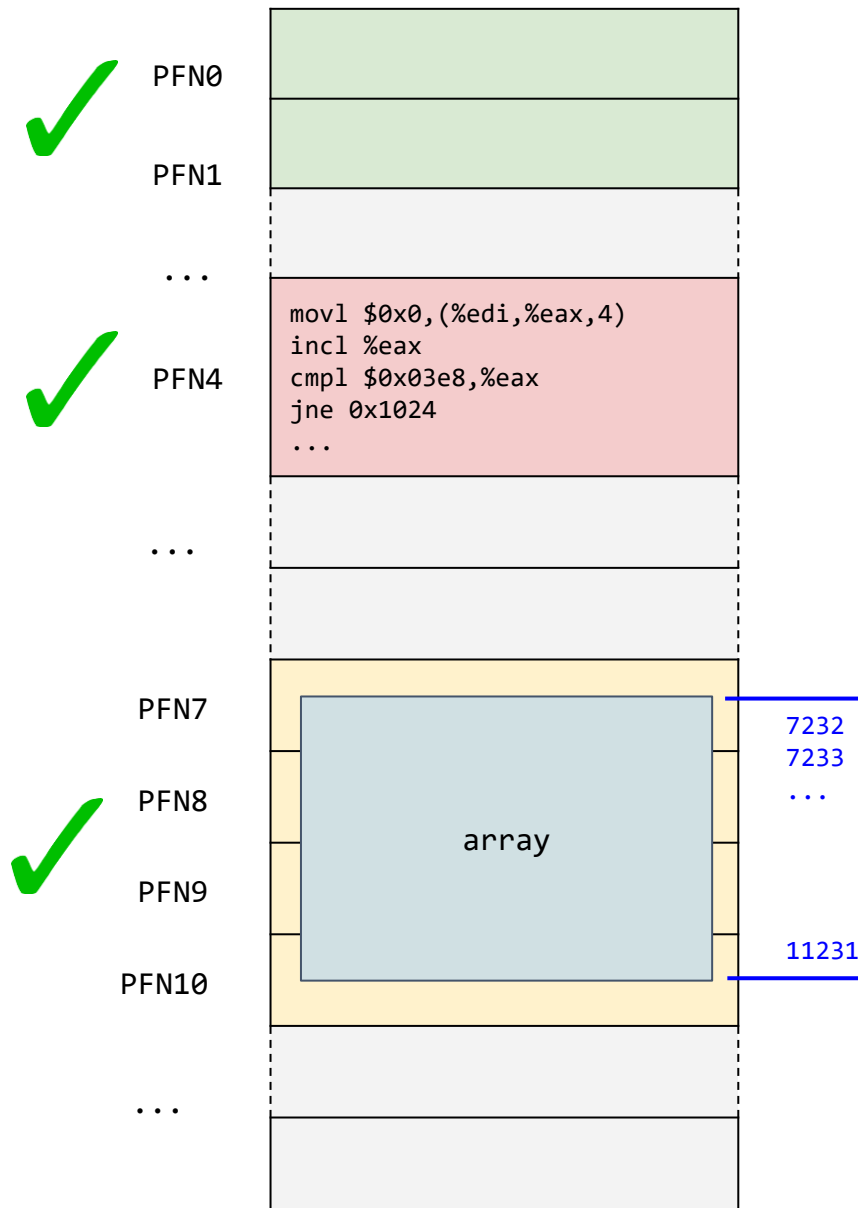
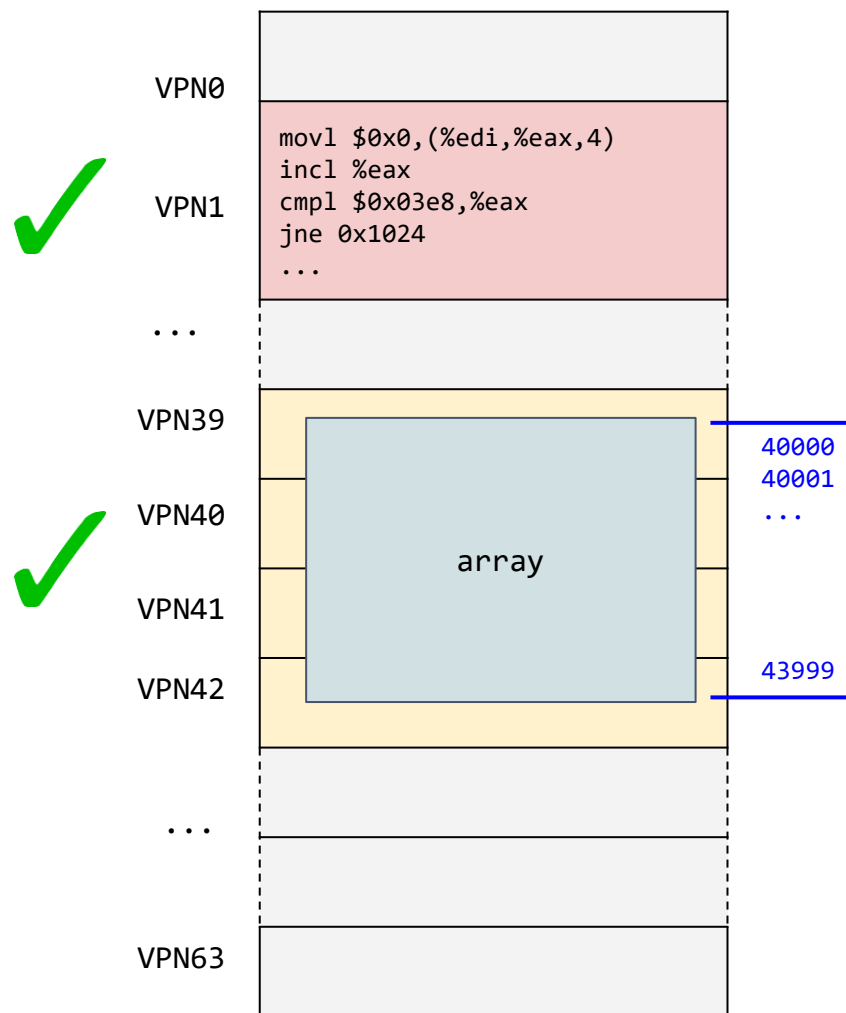
1. Espacio de direcciones de 64KB y tamaño de página de 1KB.
2. Tabla de página (tipo array) localizada en la dirección física 1KB (1024).
3. Del enunciado, el código está desde 1KB (1024), luego este se encuentra dentro de la página cuya VPN = 1
4. El tamaño del array es de 1000 enteros → 4000 bytes.
5. Se asume que el rango de direcciones del array es:  $40000 \leq \text{dir} < 44000$  ocupando las páginas virtuales 39, 40, 41 y 42.
6. Se asume el mapeo mostrado en la tabla a continuación.

```
0x1024 movl $0x0, (%edi,%eax,4)
0x1028 incl %eax
0x102c cmpl $0x03e8,%eax
0x1030 jne 0x1024
```

| VPN | PFN |
|-----|-----|
| 39  | 7   |
| 40  | 8   |
| 41  | 9   |
| 42  | 10  |

# Posibles problemas

## Acceso simple a memoria



# Posibles problemas

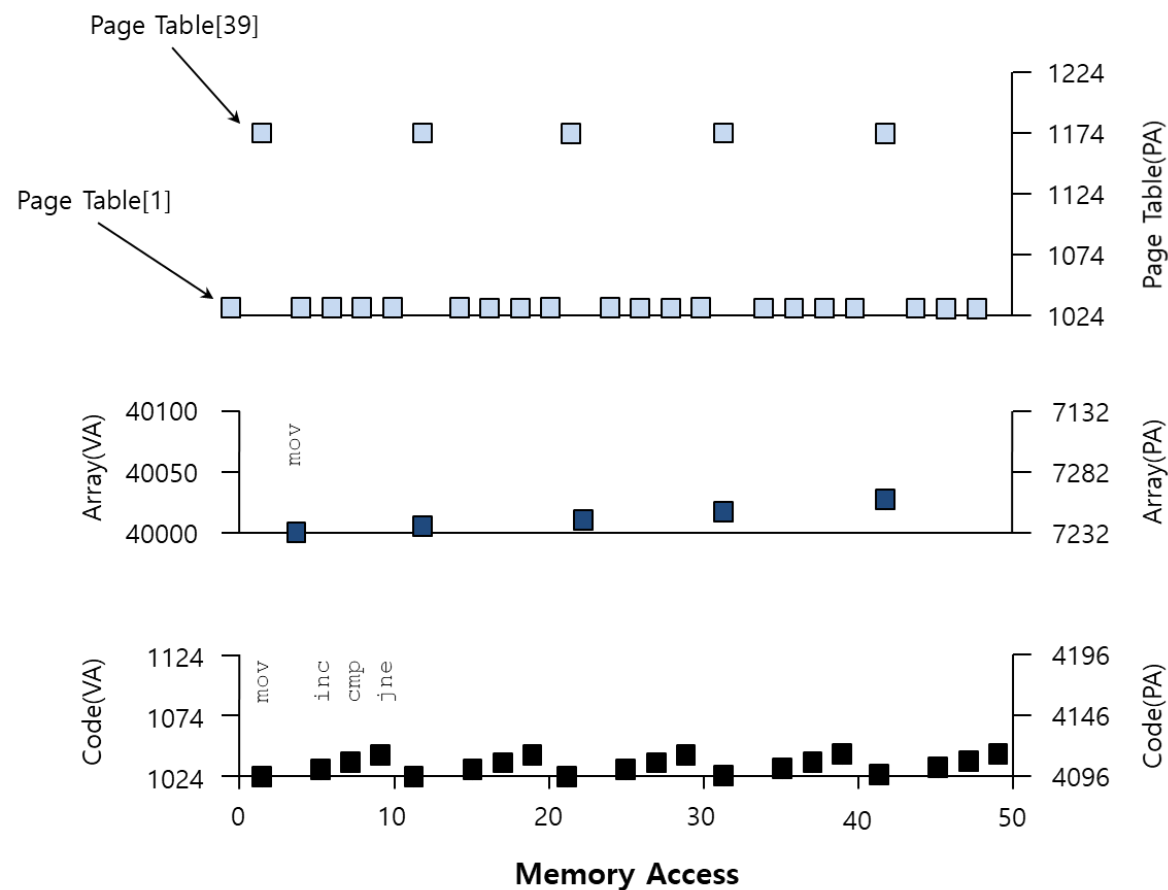
## Acceso simple a memoria

### Análisis de la traza de memoria

Para la ejecución del código del programa, cada **instruction fetch** genera dos referencias a memoria:

- Acceso a la tabla de página (PT) para obtener la dirección física en la que reside la instrucción.
- A la dirección física en la que se encuentra la instrucción para poder llevarla a la CPU para su ejecución

| Instrucción                            | Accesos a memoria |
|----------------------------------------|-------------------|
| <code>movl \$0x0, (%edi,%eax,4)</code> | $2 + 2 = 4$       |
| <code>incl %eax</code>                 | 2                 |
| <code>cmpl \$0x03e8,%eax</code>        | 2                 |
| <code>jne 0x1024</code>                | 2                 |



# Posibles problemas

## Paginación - Desempeño

- Para encontrar el PTE requerido se necesita saber la dirección de memoria dónde se encuentra la tabla de página.
- Por cada acceso a memoria se requiere que el SO realice **una referencia extra a memoria**.

# Referencias

- **Operating Systems Three Easy Pieces - Chapter 18: Paging: Introduction** ([website](#): Capítulo [18](#))
- Videos de youtube: Clase 8 - Paginación ([link](#)).
- Página de Remzi H. Arpaci-Dusseau ([link](#))
- Operating System Concepts - Tenth Edition ([website](#))
- Operating Systems Notes ([link](#))
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/1-Overview.md>
- <https://myaut.github.io/dtrace-stap-book/kernel/proc.html>
- <https://myaut.github.io/dtrace-stap-book/index.html>
- <https://www.technologyuk.net/computing/computer-software/operating-systems/operating-system-utilities.shtml>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/simple-threads/>
- <https://courses.cs.duke.edu/spring19/compsci590.1/slides/>
- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>
- <http://pages.cs.wisc.edu/~bart/537/lecturenotes/titlepage.html>
- <https://www.scs.stanford.edu/22wi-cs212/>
- <https://flint.cs.yale.edu/cs421/papers/x86-asm/asm.html>